

VECTOR

Conference Bumper Issue

- *APL in Business* at Swansea 42
- *APL 94* at Antwerp 53
- Lescasse on PLUS III Programming 68
- Reiter on Fractals in J 86
- Pearson on Causeway 126
- Gay on Bracket-Axis Simulation 150
- 16-page Educational Supplement 17



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.11 No.2 October 1994

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, in workspaces from I-APL, APL*PLUS, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the Vector TrueType font, available free from Vector Production), and Winword-2.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1994-95

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	50	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates are £250 per full page, £125 for half-page or less (there is a £75 surcharge per advertisement if spot colour is required).

Deadlines for bookings and copy are given under the Quick-reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO6 4JJ. Tel: 0439-788385. Cserve: 100331,644

Contents

		Page
Editorial: Monsieur Langlet's Enterprise	Anthony Camacho	3
APL NEWS		
Quick Reference Diary		5
News from Sustaining Members	Gill Smith	13
The Education Vector	Ian Clark	17
REVIEWS SECTION		
APL Product Guide — Updates	Gill Smith	33
Les APL Etendus (par Bernard Legrand)	Adrian Smith	38
RECENT MEETINGS		
<i>APL in Business</i> at EBMS, Swansea		42
Opening Plenary	Jon Sandles	44
Duncan Pearson's DDE and DLL Workshop	Ray Cannon	48
Croeso e Cymru and the APL Welsh Hwyrnos	Diane Whitehouse	50
<i>APL 94</i> at Antwerp: Roundup	Anthony & Sylvia Camacho	53
Reflections	Ray Cannon	59
Binary Algebra Workshop (Foil)	Gérard Langlet	60
GENERAL ARTICLES		
Windows Development with APL*PLUS III (Part II)	Eric Lescasse	68
Chaotic Behaviour Revisited	Gérard Langlet	82
Fractals RYIJ	Clifford A. Reiter	86
One Font to Rule Them All	Adrian Smith	105
The Random Vector Understanding Statistical Theory Through Simulation	David Appleton	109
TECHNICAL SECTION		
Hackers' Corner: Making Noises with APL	Adrian Smith	118
Technical Correspondence		123
Causeway: a Technical Architecture	Duncan Pearson	126
No SX please, we're APLers	Ray Cannon	143
Bracket Axis Simulation for APL*PLUS II/386	Allan Gay	150
Index to Advertisers		159



APL Products and Services

Internet: apl2@vnet.ibm.com - CompuServe: go IBMAPL2 - Fax: 1-408-463-4488 - Tel: 1-408-463-APL2(2752)
APL Products and Services, M46/D12, IBM Corp., Santa Teresa Lab, 555 Bailey Ave, San Jose, CA 95141 USA

Editorial: Monsieur Langlet's Enterprise

by Anthony Camacho

The philosophy of science is the study of how it is that we discover the truth about the world, and in particular those general truths that allow us to foresee, to some extent, the consequences of our actions. Without such foresight morality is impossible since morality consists in acting so as to avoid doing harm. This shows that knowledge is primary.

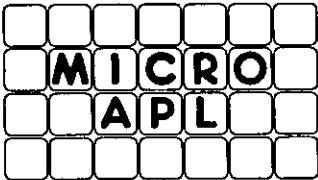
The standard approach to the problem of discovering laws about the world is to try to guess at what might be a law and then try to disprove it. If serious, well thought out attempts to disprove a hypothetical law have failed to find fault then we may accept it. Most scientific experiments are involved in these attempts to disprove a hypothesis.

The hypothesis is, of course, a model. It is a symbolic description of the world and carries with it, in the meaning of its symbols, the clue to what tests would disprove it.

As I understand it Gérard Langlet, apart from his other work, suggests that, at the lowest level and the finest detail, the way the world works must be *simple*. Complexity can only arise with structure and structure is built of parts which are combinations of the lowest level and finest details.

If the physical world is composed of quanta and time too is non-continuous then the models of the world which assume a continuum are inappropriate for dealing with quantised time and space.

Gérard suggests that, instead of forming our hypothetical models to match the world, we should build up structures from the simplest possible elements — bits. When these are elaborated to the level where the real-world counterparts could be observed then we can seek for such real world counterparts. A perfect match, that defies all attempts at disproof, between such a model and the real world would be a real breakthrough. I am not sure how close the *APL Theory of Human Vision* gets: getting close enough to consider the question is exciting enough.

—		APL Preferences	▽
APL.68000			
Platforms		Versions	
☒ RISC System/6000		☐ Level I	
☒ Apple Macintosh		☒ Level II	
☒ Commodore Amiga			
☒ Atari ST			
APL for GUIs			

MicroAPL Ltd., South Bank Technopark, 90 London Road,
LONDON SE1 6LN, UK

Tel: 071 922 8866 Fax: 071 928 1006
Applelink: microapl Internet: microapl@applelink.apple.com

Quick Reference Diary 1994

Date	Venue	Event
October 28th	IEE	BAA Meeting: Vendor Forum
June 4th-8th 1995	San Antonio, Texas	APL95

British APL Association meetings are normally held in the IEE, Savoy Place.
Nearest tube outlets: Temple or Embankment.

APL Training Courses for 1994

Level	Company	Days	Dates
Intro APL	MicroAPL	1	Nov 9
Intro APL2	MicroAPL	1	Oct 5, Nov 30
APL*PLUS	MicroAPL	1	Nov 23
Advanced APL	MicroAPL	1	Nov 16
APL GUI Design	MicroAPL	1	Oct 19, Dec 14
Programming Techniques	MicroAPL	1	Oct 26

If you would like to have your courses or seminars listed in Vector, please contact Gill Smith with the details.

Dates for Future Issues of VECTOR

	Vol.11 No.3	Vol.11 No.4	Vol.12 No.1
Copy date	2nd Dec 94	3rd March 95	2nd June 95
Ad booking	9th Dec 94	10th March 95	9th June 95
Ad Copy	16th Dec 94	17th March 95	16th June 95
Distribution	January 95	April 95	July 95

APL

Developers

If you are an APL developer seeking a challenging and rewarding career opportunity, please consider joining our organization. SunGard Capital Markets (formerly Devon Systems International) is a world wide supplier of financial software for the derivatives market. Our company was founded in 1983 and our primary product is a trading support system for future, options, and interest rate products.

With over 500 clients world wide, the company and the product are well recognized as the leader in its field. The system is developed using Manugistics APL*Plus II and runs on most supported hardware platforms. We are currently undertaking a major effort to incorporate an SQL database and GUI front-ends using APL*Plus III, C++, MS Access and other new technologies. We are seeking experienced, motivated APL developers who are interested in real-world applications development. With SunGard Capital Markets, a unique opportunities exist to apply APL skills as well as to learn new technologies whilst serving a very sophisticated client base consisting of international banks and financial institutions.

Requirements for the position are a minimum of 3 years of APL development experience using APL*PLUS II or Dyalog APL. Familiarity with SYBASE (or other mainstream SQL), C or C++, Windows programming techniques and/or Windows-based software development environments such as Visual Basic or Microsoft Access would be a DEFINITE PLUS.

Salaries are highly competitive and relocation packages are available.

If you think you would be interested in this unique challenge, and would like to join our team of highly capable software developers in Philadelphia, please send your CV in confidence to:

SunGard Capital Markets
Suite 900
510 Walnut Street
Philadelphia, PA 19106
ATTN: Barbara Foley

APL95 CALL FOR PARTICIPATION

"APL Does It Today"

SIGAPL's annual international conference for 1995 will take place between June 4 and June 8 in San Antonio, Texas. The technical theme of the conference is meant to emphasize practical use of APL. This breaks down into two sub-themes:

- APL95 as a showcase for highly-successful use of APL in the real world
- APL95 as a source of practical education in modern computing techniques using APL.

Of course, our technical program will be open to all forms of APL, J, and related languages, and to all the traditional topics and formats of presentation — but we will be placing special emphasis on material which relates to practical application.

The key to a successful conference is effective publicity, and to make the publicity effective we need to include details of what will happen at APL95. That's why you'll see target dates included in all the sections below. Some dates are very firm (paper review, for example, is a serious business that has to adhere to a schedule) — others are more relaxed. But the message about dates is this: We can do more to get a good audience for you if you let us know what you want to do at APL95 in plenty of time.

The conference language is English; there will be no translation facilities at the conference and we request that all technical material be in English. However, we are more than happy to cooperate with prospective presenters who are not English speakers and will work with you on pre-conference translation of presentations.

We invite proposals from all APL practitioners for a presentation in any of the following formats:

Tutorials and Workshops

It's relatively easy for people to begin using APL; there are several companies who offer introductory courses, and there are textbooks. But, once the initial skills are achieved there are fewer resources for people to advance onto the next rung on the ladder and become contributors of major value to their employers and clients.

The annual APL conference is a great concentrator of APL skills; indeed it's the one event of the year where nearly all the major players of APL will meet under one roof. At APL95 we want to use them to their best advantage for practical, up-to-date and relevant training — in the form of tutorials and workshops.

One way that you can help us with this is to offer to lead a tutorial or workshop session on the topic of your choice. Some of the things that we have in mind are:

- Event-driven applications programming
- Object-oriented programming
- Large APL applications (high code volume or high data volume)

You will certainly be able to think of others.

Another way you can help us is by telling us the tutorial and workshop topics which would make APL95 irresistible to you as a delegate. If enough people say they would attend a session on a particular subject, we'll do everything we can to get the best possible presenters. Please let us have your tutorial and workshop proposals by January 1, 1995.

Reviewed Papers

Reviewed papers have long been the backbone of the conference, and we intend this to continue. We invite original technical papers on all aspects of APL. These papers will be subject to double-blind peer review, and those which are selected for presentation will be printed in the APL95 Conference Proceedings, which will be an issue of APL Quote Quad — distributed to the conference delegates, SIGAPL members and libraries world-wide.

In order for us to begin to structure the technical program we would like prospective authors to send title(s) and/or abstract(s) to us as quickly as possible (preferably before November 1, 1994).

Draft papers for review must be received no later than December 1, 1994; you may send a draft paper even though you have not sent a title or abstract earlier than this. Draft papers should not exceed 6,000 words.

Informal Posters

Poster sessions are becoming increasingly important at SIGAPL's conferences; we will continue to build this aspect of the program at APL95.

Posters are a great way to present your ideas informally and receive feedback from an interested audience. In order to maximize the impact of posters we would like you to submit your proposals in advance of the conference — although you may just turn up at the conference with a poster if you prefer.

The only restrictions on the format of a poster are that it fit onto a metre-square wall area, must be original work, and that it must be a technical presentation (advertisers are expected to book space in APL95's exhibit area). Posters are not subject to peer review. We invite poster presenters to prepare a one-page summary of the ideas in their poster which may be distributed during their poster session; in addition, we may be able to find room for poster summaries in future editions of Quote Quad.

We are happy to receive advance poster submissions until May 1, 1995.

Panel Discussions

What are the hot topics for APL in 1995? How does APL fit in with the ever-changing outside world? Panels have always been the format we choose to explore these topics, and we see panels as an important part of being at APL95 rather than reading about it later. Some of the things we already know we want to explore in panel sessions are:

- The reunification of APL
- Which paradigms of the outside world are passing fads, and which should we absorb into APL?

No doubt you can think of more. We want to hear from you — because a well-prepared panel has far more value than one which is produced late in the day. We'd like your panel proposals by January 1, 1995.

Software Exchange

SIGAPL invites you to send new and useful APL/J software to the APL95 Software Exchange. New GUI software is especially welcome. Software from prior APL conferences is also welcome if it's been significantly improved.

APL95 conference software will be available in three ways:

- 1 The Conference Package: software received by April 5, 1995 will be available for purchase at APL95 in San Antonio.
- 2 The APL95 Supplement: software received between April 5 and June 8, 1995 will be mailed at no charge to all who purchase the Conference Package at APL95.
- 3 The Conference Package, plus the APL95 Supplement above, will be available by about October 1995 from SIGAPL, the BBS\APL, and the Waterloo archive.

Send software for the APL95 Software Exchange on a 3.5" disk to:

Dick Holt, 3802 N. Richmond St., Arlington VA 22207, USA

Software in APL, J, and related languages is invited. English is preferred, but other languages are also welcome. Your material should include an ASCII file that briefly describes what the software does, and what other software/hardware is needed to use it. Please include permission to distribute any copyright software.

Other

What have we forgotten? APL95 is wide open to all of your ideas — whether you are a SIGAPL member or not (and if you're not we'll be very happy to tell you how you can become one). If there's a topic that you'd like to be a part of APL95 that you haven't seen above — let us know, and we'll do all we can to fit it in.

If there's a way you want to present your ideas that doesn't fit into these categories — let us know that as well.

Above all, APL95 is going to be YOUR conference. The Organizing Committee and the Program Committee are here to turn your ideas into reality — we look forward to your responses!

Contact Information

Two Internet distribution lists have been set up for APL95. "APL95_info@ACM.org" will be used for broadcasting information to the subscribers. To subscribe, just send a one-line message to "mailserv@ACM.org"; that single line should read "subscribe APL95_info your_e-mail_address". The other list is "APL95_conf@ACM.org", to be used for initiating conversations with APL95 Committee members. Someone will reply to your message as soon as possible.

Site Information

San Antonio boasts a variety of attractions, from international shopping and dining along the tree-shaded Riverwalk to the historic Alamo, where a band of 189 Texan volunteers defied an army of thousands in 1836.

The flavour of old Texas is everywhere. Stroll through the narrow streets and shaded patios of La Villita, a restored Mexican village. Shop for handicrafts and farm-fresh produce at the colourful stalls of El Mercado. Relive the Old West at the Cowboy Museum. Enjoy exhibits of contemporary Mexican artists at the Mexican Cultural Institute. Even the giant new amusement park Fiesta Texas has a distinctive Texas flavor. Ride the Dining Train, tour art museums, or spend the day at Hemisfair Park, site of the 1968 World's Fair. Go back in time to the missions of San Antonio. Buy a cowboy hat and bring sun-screen; the weather will be hot and dry. ...YEE HA!

Get yourself and your interests registered!

Please complete and mail the enclosed form to:

APL95,
P. O. Box 210367,
Bedford,
TX 76095 USA



The Definitive APL for Windows

The screenshot shows a window titled "Grid, ToolBar and StatusBar Objects". It features a menu bar with "File" and "Edit". Below the menu is a toolbar with icons for file operations and editing. A text field contains "Clos Fourtet, Jul" and a checkbox is checked. To the right of the text field is a button with a magnifying glass icon. Below the toolbar is a table with columns for months and numerical data.

	May	Jun	Jul	Aug	Sep
Chateau Cole-Daugay-ex-Made	212	1000	154	631	
Chateau Coutet	681	206	837	709	
Chateau Cure-Bon-La-Madeleik	383	953	948	390	
Chateau Faurie-de-Soutard	12	192	984	245	
Chateau Fonplégade	258	234	102	220	
Clos Faurtet	599	228	319	700	
Chateau Franc-Mayne	152	486	861	814	
Chateau Franc-Pourret	431	259	371	394	
Domaine du Grand-Faurie	677	514	729	721	
Chateau Grand-Mayne	686	987	151	671	

At the bottom of the window is a status bar with the text "Ready ..." and three buttons labeled "Ins", "Caps", and "Num".

Experience counts

Since we launched Dyalog APL/W at APL92, nearly two years ahead of our nearest competitor, our customers have developed hundreds of successful industrial-strength GUI applications. With the benefit of their experience, we have enhanced and refined Dyalog APL/W into the mature, stable and above all useful Windows development tool that it is today.

Not only does Dyalog APL/W provide the most comprehensive set of GUI facilities available for any APL, but you can be confident that your workspaces will run *unchanged* on Unix workstations under OSF/Motif.

With Version 7.0 we have added support for Visual Basic Custom Controls, a powerful built-in Grid object, Numeric, Currency, and Date fields, ToolBar, StatusBar and TabBar objects, automatic context-sensitive Hints and Tips, Metafiles, MDI, 3-D Forms and Controls, a fully customisable Session, an ODBC interface, *Namespaces* for encapsulation, and a host of other improvements.

That's why Dyalog APL/W remains the professional choice. For a free trial copy, contact Dyadic or your local distributor today.

News from Sustaining Members

Compiled by Gill Smith

Dyadic Systems Ltd

Dyadic is extremely pleased with the way the new version of Dyalog APL was received at APL94. Delegates who attended Dyadic's Vendor Forum (and most did) heard John Scholes give the first presentation of "Namespaces" outside the UK. The reaction from the audience, most of whom were unaware of this new development, was extremely positive.

Namespaces allow you to organise your functions and variables into logical units and sub-systems, just as the DOS directory structure allows you to organise your files. In Dyalog APL, the active workspace is also known as the root namespace; rather like the root directory in DOS. Objects that you create at this level are held in the root namespace, just like files in your root directory. (If you don't want to use namespaces, you can define all of your objects in the root namespace, just as before.) However, you can also create a namespace (analogous to a sub-directory) in which you can place other objects and indeed other namespaces. There is no limit to the number of namespaces you can have, nor to the depth they are nested.

Namespaces do not just allow you to organise your workspace better, they also act as different contexts for the execution of code. When a function runs in a namespace, the names of variables and other functions to which it refers are local to that namespace, and do not conflict with objects with the same names in other namespaces or in the outer workspace. This means that, for the first time in APL, you can write a self-contained utility or a complete application that can safely be copied into any other workspace and run without fear of name conflicts. Namespaces also make it extremely easy to implement large application workspaces that contain logically separate sub-systems, without the need to resort to overlay files and similar techniques.

Namespaces provide benefits to all APL developers, but are of particular appeal to those developing GUI applications. This is because GUI objects (Forms, Buttons, Grids and so forth) are also namespaces themselves. This means that you can store all of the callback functions, and any data variables that they require, inside the object to which they apply, rather than having them swirl around in the workspace. Namespaces provide a much needed solution for the requirement to pass information between callback functions, and between

successive invocations of the same callback, without the use of global variables (i.e. variables that are global to the workspace as a whole). Instead, such variables become static variables, that are local to an object (and thus directly accessible from functions running within that object) but do not pollute the workspace as a whole. Furthermore, as GUI objects are namespaces, you can write a completely self-contained GUI widget that contains all the required APL code and data, together with its GUI components, and save it as a single object that can subsequently be copied into any active workspace and run. The user sees only what the object does; not how it does it, a fundamental tenet of object-oriented programming.

Dyadic intends to use namespaces as the basis of further development towards object-oriented programming, and to provide the advantages (but not the complexities) of this approach for Dyalog APL users in the future.

Insight Systems

So far, the second half of 1994 has left us breathless. After announcing SQAPL for SequeLink at APL93 in Toronto, we slowly sold about a dozen evaluation systems during the next twelve months. A year later, many of these customers are buying 20- to 50-user production systems. The first five beta copies of SQAPL 2.0 with ODBC support went to new customers in Helsinki, London, Melbourne, New York, and Rio de Janeiro. New customers are queuing up, and redistribution agreements are now more or less in place with three major APL vendors. The rumours regarding the death of APL seem to be premature, and the non-APL middleware market seems to be taking off like a rocket too, so we have never been busier.

Dyadic shipped SQAPL/EL with version 7 of Dyalog APL for Windows. The *Entry Level* version contains all the functionality required to get started with simple ODBC drivers for local databases such as dBase or Access. By the time this is printed, we should have completed the *Professional Edition*, which contains a number of additional features to increase programmer productivity, and take advantage of performance options in advanced ODBC drivers, especially when using remote databases. We are now concentrating on assisting Manugistics in the packaging of both the /EL and /PE versions of SQAPL for APL*PLUS III, and working with Soliton to provide SQAPL for Sharp APL under Unix. We will also provide a version of /PE for J version 2 from Iverson Software, so you can get started on J using a familiar tool for database access. Upon demand, we will make the tool (SQJ?) available for J under OS/2 and Unix, too. Need a copy of SQK? Please call us!

A second generation of servers, based on SequeLink version 2.5, will provide ODBC access to Unix APL servers (and hopefully also APL2 under MVS) by the end of the year. We are also starting work on a local ODBC driver for APL, which will give direct access to Dyalog APL/W and APL*PLUS III without the use of middleware. This product will also be available before yearend, if all goes well (we need it for a project of our own before that).

The idea of an ODBC driver for a local or remote APL interpreter may seem a bit strange. ODBC is for SQL databases, and the most natural way to call a development tool like APL under Windows would be via DDE or as a DLL? There are dozens of APL applications containing their own databases, which suffer from the fact that their internal databases are inaccessible to mainstream tools. With our ODBC APL drivers, we will provide tools to help you write simple SQL interpreters, so you can let people go into Microsoft Access, connect to APL (which you sold as a high-performance multi-dimensional DBMS) and say:

```
select high, low, close
  from curddb
 where market='NYC'
   and currency='JPY'
   and date>19940101
   and frequency='WEEKLY'
   and totalling='AVERAGES'
```

Although the underlying database may be a four-dimensional table with daily rates, we can return a two-dimensional table with massaged data, pretending that we have an SQL table containing weekly averages. This idea can be extended to all kinds of analyses — so APL can contain infinitely large SQL tables. The ultimate virtual database is the one which allows queries like:

```
select result from apl where expression='2+2'
```

Once more, a change of Internet address. Our address is now insight@inet.uni-c.dk.

Manugistics Inc

Manugistics released version 1.2 of APL*PLUS III for Windows almost a week ahead of schedule. We feel this version further positions APL*PLUS III as the strategic choice for Windows development. Major enhancements found in this release include support for Visual Basic Custom Controls, calling 16-bit DLLs directly, new object classes for MDI application development and more control structures; including :continue, :leave and :goto.

Currently, there are several beta sites testing the ODBC interface developed by Insight Systems for APL*PLUS III. Reports so far have been very positive and we soon hope to simultaneously release two versions of the product — one designed specifically for the single/ad-hoc user, the other designed for the commercial APL developer who needs to provide an ODBC enabled product with more functionality. For more information on ODBC for APL*PLUS III, please contact Manugistics, Insight Systems or your local APL*PLUS dealer.

The Manugistics troop returned safe and sound from APL94. In keeping with the conference theme of "The Language and its Applications", a commercial application developed in APL*PLUS III by GEIS called LEX2000 was presented during our vendor forum along with demonstrations of using VBXs and ODBC. Our software drawing winner was Daniel Jonusz of Mobil Oil Corporation in Princeton, NJ and APL*PLUS Sport Radios were won by three other conference attendees.

A very exciting synergy is developing between our mainframe based enterprise-wide planning product and the APL*PLUS Network Sharefile product. Two new sites began using Network Sharefile in the past few weeks with more to come. Network Sharefile allows multiple copies of the APL*PLUS Sharefile system to communicate with one another. A user can be running an application that appears to be using one Sharefile database while a number of Sharefile databases in virtually any location may actually be in use. The benefits of Network Sharefile include better performance and better data security and integrity.

THE EDUCATION VECTOR

October 1994

Editor Ian Clark

This Education Vector has been reprinted from VECTOR Vol.11 No.2. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Anthony Camacho, 11 Auburn Rd, Redland, BRISTOL, BS6 6LS Tel: 0272-730036.

Contents

Editorial	Ian Clark	18
Jot-Dot-Floor	Ian Clark	20
J-ottings 3	Norman Thomson	24
J: Bibliography	Donald B. McIntyre	28
Cows and Bulls - A Solution	Anne Wilson	29
More about Fractional Series	J.R. Sutton	30

Ian Clark
IAC/Human Interfaces,
9, Hill End, Frosterley,
Bishop Auckland,
Co. Durham DL13 2SX.

Tel: 0388-527190

Email: clark.i@applelink.apple.com or 100021.3073@compuserve.com

Editorial

by Ian Clark

Welcome to another edition of Education VECTOR. This goes to press just before APL'94, so whatever enlightenment emerges during the symposium will just have to wait for the next issue.

A teacher friend I saw recently congratulated herself on getting through the first week of term. Her face wasn't grey and the lines on it were fading (I thought) so she must have had a good summer holiday. With the unions all set to continue their boycott of the compulsory testing of the National Curriculum and the Government still striving to re-introduce the eleven-plus in one form or another, with amateur markers to swell the ranks of amateur inspectors, no doubt all riding to work on trains directed by amateur volunteer signalmen, this has got to be the Golden Age of the Amateur Volunteer.

Nowhere is this more evident than the teaching of Mathematics. A recent report on the sorry state of mathematics teaching seems to lay the blame squarely on Primary Schools. Primary teachers might justifiably wonder what it is the country really wants of them. To be the saviours of the next generation? Or to serve as whipping boys for the undesirable side-effects of a society governed by market forces and other gods of darkness? If mathematical skills are to be costed on their current market value by sector, is it any surprise that they're tending to accumulate in the faster end of the financial sector and desert the junior end (and upwards) of the educational sector?

For years those further up the educational tree have been telling the primary schools that all they require of them are children able to go to the toilet without supervision, tie their own shoelaces and write their own names. Leave the fancy stuff to those better qualified to teach it. Now it emerges that primary teachers have to carry the can back for the state of Mathematics throughout the entire system.

This isn't the place to discuss the role of specialist teaching in primary schools. Nor the evidence for and against the existence of things called pre-mathematical skills. But I do think that those responsible for the staffing and remuneration of schools should not hide from criticism behind the honest attempts of others to put into effect the results of their own loose thinking. I remember seeing a report, at least six or seven years ago now, about the teaching of Mathematics in both primary and secondary schools in England and Wales. It furnished figures to

show that at least 70% of mathematics teachers had no recognisable qualifications in mathematics beyond O-level, and that perhaps 90% were teaching the subject far above the level at which they were technically qualified.

Now never mind the reasons for this state of affairs and why it cannot be otherwise. Never mind asking whether they are doing a good job. Never mind wondering how to compel them to do a better one, by performance related pay, say, or testing their pupils at frequent intervals with nationwide tests. The question we want to ask ourselves is — no, not whether we're feeling lucky, but whether we would tolerate such a situation in the teaching of French, say, or Spanish.

Now I have a shameful admission to make. I hardly know a word of Spanish (except one or two I learnt from reading Hemingway, which it seems I don't pronounce correctly anyway). Yet, if Spanish were to be treated as Mathematics is at present, I could be recruited to teach PE, RE or any other 'E' for which I might be qualified, then directed to teach Spanish throughout the school. Would any blame attach to me if some official Spaniard came along and castigated my teaching (I think that's the word)? In a fair and just society, not at all. The blame would lie with the headmaster, or rather with those who made up the rules which forced the headmaster to contemplate such a ludicrous step. And ultimately, in a democracy, that's all of us. The headmaster might turn round and say he couldn't get anyone who really knew Spanish for love or money. Market forces, again. But what a slur on our society, and those who lead it.

The government have shown time and again in other spheres that it is possible to manipulate market forces. In fact they've spent billions doing so. Let them do so in the sphere of Mathematics teaching, if we all agree it's so important for the future economic health of the country.

Jot-Dot-Floor

by Ian Clark

A guy from an IBM laboratory (which will be nameless) once offered me for analysis dozens of reels of magnetic tape containing the record of two years' work by a team of 20 APL programmers using a central APL service. On the tape was captured every keystroke during that time. Nothing had been done with the tapes beyond elementary counts of keystroke by type, but it emerged that in all that time, Domino ($\boxdot$) had been used precisely twice.

Once was a mis-key. The other was when the programmer wanted a nice blobby character for printing a histogram.

This doesn't say much for the depth of theory brought to bear on the APL programming in that particular location. Perhaps they'd all have been better off using BASIC. Now as every educational APL user knows, Domino is one of the most impressive primitives of all. It not only inverts matrices, it divides one matrix by another, like this: $Y\boxdot X$.

In fact $\boxdot X$, the matrix inverse of X , is just a special case of $I\boxdot X$, where I is the Identity Matrix (with 1s down the diagonal, 0s elsewhere), just as $\div X$ is a special case of $1\div X$. Get your free copy of I-APL out and try it. And while you are about it, you can verify that $\div X$ is in a sense a special case of $\boxdot X$, by defining X as a one-by-one matrix of some given number, yes, like this:

```
X←1 1p99
```

Yes that's all very well, but is there a practical use for it? Is there any mileage in it, I think they say nowadays (or was that last year?) Well, there's a lot of mileage in it, both literally and metaphorically. Consider this, now.

Petrol is expensive these days. To fill my car with petrol generally costs me over £20 a time. Do you know how much it costs for a litre? What's that in gallons? If it weren't for the scale still stuck to the pumps in some of the more backward parts of the country I'd never know. In fact I'm invited implicitly to trust the pump to deliver what it says it's doing. For all I know they've got a little icon on their coloured screens saying "muggins" which they touch whenever they're certain I'm not the weights and measures man.

Nobody likes to pay lots of money for things when they don't know how much it really costs. You know what you're being charged, of course, but do you know

what you're getting for it? Now it may come as a surprise to the oil companies, but when you buy petrol you don't really want so many litres of dangerous smelly fluid. What you really want is so many miles for your car to go. So how many miles are you really getting for a tankful of petrol costing £20?

The measure of how good a car is at turning petrol into distance travelled is called its (fuel) consumption. The manual that comes with the car will give you a figure to expect, generally in miles per gallon. But it varies. A mile uphill takes more petrol than a mile downhill. Just how much more you only begin to appreciate when you see the petrol actually being sprayed in the carburettor as somebody puts their foot on the accelerator pedal. More importantly, the more your car needs servicing the more petrol it tends to use — to go a set distance of course. So how can you possibly find out your car's consumption, here and now?

A car-manufacturer's manual once told me, in all seriousness, to fill up a jerrycan with exactly one gallon of petrol, and then to drive around until the tank runs dry and the engine stops (it's no good relying on the petrol gauge on the dashboard showing "empty", the book said). Once the car goes phut and stops, get out and pour in the gallon of petrol. Write down the tachometer (milometer) reading — most important! Then drive around steadily on level roads without stopping and starting if at all possible until the engine stops again. Read the tachometer once more and subtract the first value from it. You'll get a number, say 35.5 (miles). This is actually your car's fuel consumption in miles per gallon!

Why not get your class to ask their dads to do this the very next weekend? Why should teachers always be the ones to bear the burdens of others' loose thinking? Then, when they're all back in class the following Monday, and you've administered motivational therapy to the defaulters, tell them all there's a smarter way to measure fuel consumption, without wasting time and petrol.

Each time you buy some petrol, note down the tachometer reading (X) and the quantity bought (Y). Plot the two series of numbers, Y against X, on a graph. Draw the best straight line through the points (it's best to have four or more) and hey presto, your car's consumption is the gradient of the straight line!

With APL you don't even have to plot the graph and "guess-timate" the line. Suppose X and Y are the two sets of readings. For test purposes try:

```
X←1.1 2.1 3.1 4.1 5.1
Y←0.88 1.68 2.48 3.28 4.08
```

In fact you can compute Y by:

```
Y←X×.8
```

Now $Y \boxminus X$ gives the gradient of the best straight line through the set of points having the coordinates:

$(X[1], Y[1]), (X[2], Y[2]), \dots$

Try it — you'll see 0.8 as the result.

You can try adding a bit of noise to Y — varying it up and down a little to suggest figures got from the real world. That's the great thing about APL. It's so easy to generate test data, which brings dry calculations to life. We'll leave noise generation as an exercise here (something for a future issue of EV).

Wait a minute though — this line we've fitted happens to pass through the origin (0,0). This is not what we want. The best straight line doesn't necessarily go through the origin, because you can't expect to start recording petrol purchases with a zero mileage reading. Not even with a car straight from the factory.

However you can still use "domino" ($\boxplus$) to fit the correct line, but you must modify X slightly. Instead of just a sequence of distances, we want a 2-column matrix, with 1's all the way down column 1, and the original X becoming column 2. Thus, instead of X being:

```
1.1 2.1 3.1 4.1 5.1
```

we want X to be:

```
1 1.1
1 2.1
1 3.1
1 4.1
1 5.1
```

To make X into this, enter:

```
X←X◊.*0 1
```

Now:

```
Y⊖X
```

gives a result consisting of a pair of numbers: 0 0.8 i.e. the intercept and the gradient, which together define the line. Instead of 0 the first number might turn out to be a very small but non-zero number in "scientific notation", e.g.:

```
5.846505E-10 0.8
```

which is really only saying that when we calculate a number which ought to be 0, using an approximating algorithm which doesn't know that, we'd be very lucky to get exactly 0.

How on earth does it work? $Y \div X$ is an example of "matrix division", of which scalar division is a special case. In other words, if L is the one-by-two matrix (0 0.8)

$$L = \begin{bmatrix} 0 & 0.8 \end{bmatrix} Y \ X$$

is just solving for L the equation:

$$L \cdot X \ X = Y$$

by (matrix) dividing throughout by X.

In APL, matrix division is done by a process called polynomial regression. A special case of this is linear regression, better known as "drawing the best straight line" (the least-squares best fit). The APL tutorial by Alvord and Thomson explains it all in detail.

So it really is some use being smart and knowing some mathematics! It can save you having to drive around all day going nowhere and risk coming to a dead stop in an awkward place.

British Industry - Please Take Note.

A sample workspace *PETROL*, which performs this activity, is provided on the Freeware Disk of IAPL/Mac, the Macintosh version of I-APL. Write to the Editor, EV.

An APL Tutorial, by Alvord and Thomson, is available from I-APL Ltd, 11 Auburn Road, Bristol BS6 6LS.

J-ottings 3

by Norman Thomson

The aim of J-ottings is to identify and articulate some fundamental hurdles which the practised APL user must clear in order to achieve full, or even passing fluency in J. The similarities between APL and J do much to disguise the importance of some of the fundamental differences. It is all too easy to start out in J and then give up, particularly in the absence of commercial pressures to drive the learning process along. To stop thus is to miss savouring the delights of travelling towards the high peaks of Roger Hui's remarkable interpreter. The foothills are steep, but the view from the top is truly spectacular! Not only does J contain, as primitives, many things which have to be explicitly programmed in APL, but after a while the user begins to sense it having a kind of intelligence of its own, giving meanings through ruthless logic to combinations of symbols which may well have never been part of its author's visions.

Quite apart from the, at times subtle, differences between APL and J there is much in J itself that is "similar but different" which makes the exercise of focusing on "look-alike pairs" a valuable aid in learning the language. What follows highlights two pairs of conjunctions, namely $\&$ and $\&.$ (with and under) and $@$ and $@.$ (atop and agenda).

The first three result in compositions, that is if u and v are verbs (u and v are the standard shorthand for "verbs" in the J literature) then $u\&v$, $u\&.\ v$, $u@v$ are single verb entities, so that a reference to say a left argument means the left argument of the composition and not one of the components.

In *Jottings-2* the "sums-of-squares-about-the-mean" verb was given as

```
ss=.+/@*:@(-mean)
mean=.+/%#
```

This uses $@$, the conjunction which is called atop, or perhaps more expressively "sequence". By reading its definition from left to right the verb ss is interpreted as "the sum of squares of mean deviations", which is executed by performing mean deviation ($-mean$), square, and sum in sequence. This much is just like APL.

It is important to appreciate the need for the parentheses in the definition of ss , particularly since in APL a final parenthesis on the right is necessarily redundant unless vector notation is being used. Parentheses in both J and APL commonly

appear to the left due to verbs and functions having long right scope. For example parentheses are required to ensure that an expression like $(1/2)+3$ means 3.5 rather than $1/2+3 = 0.2$. Since the atoms of tacit programming are verbs, and adverbs and conjunctions have long left scope, parentheses tend to appear to the right rather than to the left. Without parentheses the definition of `ss` becomes

```
ss1= .+/@*:@-mean
```

whose meaning is given by the parenthesising

```
ss1= .((+/@*:@- )mean
```

At this point a very important principle called *rank inheritance* must be introduced if the `@` conjunction is to be understood fully. Rank inheritance says that in a composition `f@g` the verb `f` inherits the rank of the result of `g`. Thus in the verb `ss1` with a vector argument, the result rank of `mean` is 0, and this rank is "passed along the line" to the remaining verbs which therefore operate at scalar level.

```
ss1 1 4 1 2
```

is executed by calculating the mean (it is 2) which becomes the right argument of the hook `(... )mean` for which `1 4 1 2` becomes the left argument since no explicit left argument is present. The contents of these parentheses are a sequence `(... )@-` and so `-` is applied first between `1 4 1 2` and `_2` to give `_1 2 _1 0`. This becomes the argument of the "sum-the-squares" sequence (composition) within the inner brackets, however rank inheritance decrees that this composition be applied at scalar level to give the final result `1 4 1 0`.

The conjunction `@:` is a variant of `@` which allows infinite rank. Thus substituting `@:` for `@` in `ss1`

```
ss2= .+/@*:@:-mean
```

restores the value of `ss2 1 4 1 2` to that of `ss 1 4 1 2`, namely 6.

The difference between `&` (with) and `&.` (under) is highlighted by the most common application of the latter, namely multiplication using logs. In terms of frequency of usage this is to with/under what matrix multiplication (in the mathematical sense) is to inner product. Define

```
v0= . +&(10&^.)
```

so that `v0` is the "add-base10-logs" verb and `2 v0 3` is 0.69897.

Now define

```
v1=. +&. (10&^.)
```

which is the "add-base10-logs-and-take-antilogs" verb so that $2 \text{ v1 } 3$ is 6.

A delightful illustration of the use of "under" occurs in one of Donald McIntyre's APL93 tutorials. In essence the problem is to calculate the value of the expression $\text{sqrt}(1 - \text{sqrt}(1-x^2))$. This can be done correctly by using the verbs *: ("square"), %: ("square root"), -. ("1 minus"), and constructing correctly but mundanely the verb sequence

```
%: @ -. @ %: @ -. @ *:
```

However spotting that the first and last verbs in this sequence are inverses leads to the equivalent composition

```
(-. @ %: @ -. ) &. *:
```

The parentheses, although syntactically unnecessary, help to indicate the next abbreviation which can be achieved because the first and last verbs in the parenthesised composition are also inverses, since -. is self-inverse. The composition can thus be reduced to

```
(%: &. -. ) &. *:
```

Here is another illustration which requires appreciation of the concept of rank-0 and rank-1 cells of a matrix such as $1.2 \ 3$. This matrix may be conceived either as 6 rank-0 items or 2 rank-1 items which are what we would ordinarily call rows. The rank-1 items are "box"-ed (for which the verb is <) along rank 1 by the expression <"1 where the " symbol is the rank adverb. $1 = <"1 \ m$ is thus a pair of three-item lists.

```
-----
| 0 1 2 | 3 4 5 |
-----
```

The inverse of "box" is "open" given by >, thus $(*: &>) 1$ is the matrix m squared

```
0 1 4
9 16 25
```

(*:&.>)1 is the squares of the two three-item lists.

```
-----
| 0 1 4 | 9 16 25 |
-----
```

The relation between the conjunctions @ and @. is not at all like that between the conjunctions & and &..

@. is strongly associated with the conjunction *tie* which produces a gerund, a word chosen by J's inventors to underline the fact that what a *tie* ties is verbs but the result is a noun. The name gerund may at first appear misleading since the resulting noun bears no relation to gerund in its common usage in English as an "-ing" word. In J programming practice verbs are tied together into gerunds to provide the sort of choice which is rendered in other languages by the case statement, and the corresponding select clause is whatever follows @. . This too is a conjunction with a gerund (that is a string of tied verbs) as left conjunct and the selecting index in origin zero as its right conjunct. Thus + ` - ` % @.2 selects the verb "divide" (remember zero origin!).

The combination of tie and @. called agenda appears frequently in recursively defined verbs, e.g.

```
fac=. 1: ` ([*fac@<:.)@. *
```

Monadic * is signum so either the verb 1: is chosen if the argument is 0, or "argument times fac (argument following decrement-by-1)" if the argument is greater than 0.

g `: 0 means "append" the results of all the individual verbs comprising it, e.g.

```
g=. + ` *
(g `: 0) 4
4 1
```

thus providing the function arrays which are absent from APL.

Observe that the colon is attached to the tie, not to the 0. Replacing 0 with 3 gives a variation which provides a neat way of generating $n(n+1)$:

```
(g `: 3)4
20
```

: 3 means "evoke" the gerund in the sense of inserting the component verbs in turn and expanding the noun in the manner of \$ (APL p) to as many items as required. The expression above thus means $4+4*4$.

Similarly

```

      (- \ % \ : 3)4
3
that is 4-4%4, and
      (+ \ % \ : 3)4 8
6

```

that is $4+8\%4$, with the second 4 arising from wrap-around.

J: Bibliography

by Donald B. McIntyre

- [1] A.N. Whitehead, *An Introduction to Mathematics*, Home University Library, New York and London (1911).
- [2] Alan Mayer, Editorial Comment, Vector, Vol.10, No.3 (January 1994)
- [3] Donald B. McIntyre, Disk with J script files and texts, APL93, Toronto (August 1993). Available from I-APL Ltd., 56 The Crescent, Milton, Weston-super-mare, Avon, BS22 8DU.
- [4] Donald B. McIntyre, *Amendment: "A change for the better"*, Vector, Vol.9, No.3 (January 1993)
- [5] Herbert Hellerman, *Digital Computer System Principles*, McGraw-Hill Book Company, New York (1967) p.55 and (1973) p.53.
- [6] Kenneth E. Iverson, *J Introduction & Dictionary*, Iverson Software Inc., Toronto, J Version 7 (1993)

Cows and Bulls — A Solution (Education Vector, July, 1993)

From: Dr Anne D. Wilson, York.

Herewith my solution to Cows and Bulls, when the computer guesses. The problem is one of reducing the number of possibilities using the 'hits'. I used this as the design of the function:

- evaluate all possibilities [4] – [8]
- drop those with wrong bull score [16]
- drop those with wrong cow score [18] – [19]

```
[0] FROM BULLCOW PICK;ABC;BC;FREQ;GUESS;POSS;TRIES;X
[1] ABC←FROM+ 'ABCDEFGHIJKLMNPOQRSTUVWXYZ'
[2] TRIES←0
[3] * This ran out of space POSS←ABC[1+(PICKρFROM)τ-1+1, FROM*PICK]
[4] X←{(PICK,19)ρ'((FROM*PICK)ρ(FROMρ'))}
[5] X←X,[2]((PICK,5)ρ 5 0 ρFROM*φ-1+1,PICK)
[6] X←X,[2]((PICK,10)ρ')/ABC,[2]
[7] X←'((FROM*PICK).PICK)ρ'((X), '((FROM*PICK),0)ρ' ' ' '
[8] POSS←AX
[9] loop:→{0 1 =1ρPOSS)/lerr2,lok
[10] * A random choice for the next guess
[11] 'Is it ',(GUESS+,POSS[?1+ρPOSS;]),' Enter BULLS (hits) and COWS (near)'
[12] →(2*ρBC+□)/lerr1
[13] →(PICK←+/BC)/lerr1
[14] TRIES←TRIES+1
[15] * Keep only if the number of BULLS is correct
[16] POSS←{BC[1]=POSS+, =GUESS)/POSS
[17] * Next look at COWS + BULLS
[18] FREQ←+/ABC.=GUESS
[19] POSS←{+/BC)=+/(+/[2]POSS.=ABC){((1+ρPOSS),ρFREQ)ρFREQ)/POSS
[20] →loop
[21] lerr1: 'Try again - you have made a mistake'
[22] →loop
[23] lerr2: 'You have made a mistake - I'VE WON'
[24] →lend
[25] lok: 'After ',(ρTRIES),' tries I reckon it is ',POSS
[26] lend;
```

Sample run, where solution is ABCD:

```
4 BULLCOW 4 Is it DBBA Enter BULLS (hits) and COWS
(near) □: 1 2 Is it AABD Enter BULLS (hits) and COWS
(near) □: 2 1 Is it ABCD Enter BULLS (hits) and COWS
(near) □: 4 0 After 3 tries I reckon it is ABCD
```

Editorial comment: This is an excellent solution to the general $m \times n$ problem. It works provided $FROM * \tau^{-1} + PICK$ is less than 100000 (though this could be extended by minor modification of lines [4] to [8]). For the larger problems, of course, you need lots of megabytes of RAM!

More about Fractional Series

by J.R. Sutton

In Jot-Dot-Floor (Education Vector 11.1, pp. 31-35) the author refers to the series:

$$\sum_{n=0}^{N-1} X^n = (X^N - 1)/(X - 1) \quad 1$$

and asks why it doesn't work for fractional N. Start with the case $X=1$:

$$\sum 1^n = 0/0$$

but the RHS can be evaluated by taking the limit $X=1+\delta$ as δ approaches zero. So

$$1 + 1 + 1 + \dots = N$$

which is just the count of the terms in the sum. It looks reasonable that a fractional number of terms will count to the fraction. (Much as $N=0$ counts the empty series.)

In the general case the RHS of equation 1 can be evaluated just as easily for fractional (or irrational or transcendental) N as for integers so, whatever the fractional series means, we can define its sum as whatever the RHS evaluates to. For example with $X=4$ and $N=2.5$ the sum is $31/3$. Staying with $X=4$ and various N we have:

N	Sum	N	Sum
1	1	1.5	2.333..
2	5	2.5	10.333..
3	21	3.5	42.333..
4	85	4.5	170.333..

Negative values of N are not a problem. For $N=-1$

$$(X^{-1} - 1)/(X - 1)$$

evaluates to $-1/X$ and for the general case of negative integer N to

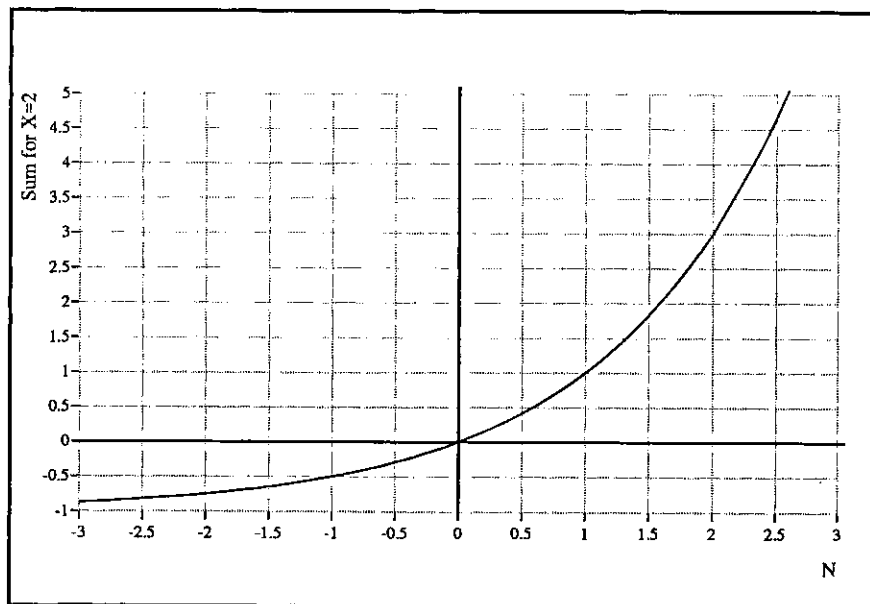
$$-\frac{1}{X} - \frac{1}{X^2} - \frac{1}{X^3} \dots$$

This can be understood as follows. To go from $N=2$ to $N=1$ take away the term with $n=1$. To go from $N=0$ to $N=-1$ take away the term with $n=-1$. Fractional negative N just combines this concept for negative N with that for fractional N .

The graph shows the sum for $X=2$ and a continuous range of N from -3 to 2.6 as an example. The shape of the curve is, of course, exponential.

```
)load rain
xx+ ^3+0.1*0,156
```

```
chset ('style' 'boxed,xyplot,nomark,grid')
chset ('yint' 0)('xint' 0)('xcap' 'N')('ycap' 'Sum for X=2')
chset ('nib' 'medium')
chplot xx,[1.5] ^1+2*xx
View chclose
```



Dr J R Sutton
16 Overland Road
Mumbles
SWANSEA SA3 4LP



RENAISSANCE DATA SYSTEMS

Enlightenment Thru Information Processing

Specializing in Books and Software on APL, J
and other Curiosities of merit.

HAVE YOU GIVEN A COPY OF IAPL OR J TO A TEACHER OR STUDENT?

Name: _____ Date: _____

Street: _____

City: _____ State: _____ Postal Code: _____

Country: _____ Telephone: _____

For a copy of our most recent catalog,
please send a self-addressed, legal-
sized envelope (stamped if from U.S.)
or mail this form to:

Renaissance Data Systems
Park West Finance Station
P. O. Box 20023
New York, New York 10025-1510

Learn APL

APL Shareware/Demos and Related Publications

APL and Mathematics

APL AS A TOOL OF THOUGHT Proceedings

Other Important Sources of APL Information.

APL History

APL References and Techniques

Special Subjects in APL

APL Interpreters and Software

J=: A Powerful Dialect of APL

Other Software for Use with APL

To Russia with Love and for APL92

Other Curiosities of Merit

**I-APL: An international voluntary project of individuals and
companies seeking to share their love of APL.**

APL Product Guide - Updates

compiled by Gill Smith

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

Pressure on space occasionally prevents us from printing the complete guide, however updates will always be listed. We do depend on the alacrity of vendors to keep us informed about their products. Anyone who is not included in the Guide should contact me to get their free entry — see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete APL Systems (Hardware & Software)
- APL Interpreters
- APL-based Packages
- APL Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

All contributions and updates to the APL Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO6 4JJ. Tel: 0439-788385, Email: 100331.644@Compuserve.com

Product Guide - Updates and Vendor Addresses

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Lingo Allegro	FRESCO Business Graphics	\$250	Fast and Easy Business Graphics DLL
	GDDME	\$1000	AP126 GDDM graphics emulation for Dyalog/W
	AP127	\$250	ODBC Interface for Dyalog/W
	FACS	\$1000	EMMA-like functions for SQL tables
	TOPR	poa	APL Code and Application Management for Dyalog APL/W
	Rumba Connection	\$250	Connect Mainframe APL to Dyalog APL/W using Rumba
	IRMA Connection	\$250	Connect Mainframe to Dyalog APL/W using Irma for Windows

APL CONSULTANCY

COMPANY	PRODUCT	PRICES(£)	DETAILS
Andrews	Consultancy	poa	APL programming and analysis, specialises in tree-processing algorithms.
Paul Chapman	Consultancy	poa	24-hr programmer: APL, C, Assembler, Graphics; PC, mini, mainframe and network.
Evestic AB	Consultancy	poa	Excellent track record from 10+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.

CHANGED OR NEW ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
Andrews	Dr Anne D Wilson	23, The Green, Acomb, YORK YO2 5LL Tel: 0904-792670
APL Interest Group, South Africa	Mike Montgomery	Private Bag X11, Rivonia 2128, South Africa Tel: +27 (11) 803 7200 Fax: +27 (11) 803 9134
Evestic AB	Olie Evero	Bertellusvagen 12A, S-146 38 Tuillinge, Sweden Tel&Fax: +46 778 4410
Insight Systems ApS	Morten Kromberg	Nordre Strandvej 119A, DK-3150 Hellebæk, Denmark Tel: (45) 42 10 70 22 Fax: (45) 42 10 75 74 Email: insight@inet.uni-c.dk
Lingo Allegro USA Inc.	Walter G. Fil	318 Half Day Road, Suite 297, Buffalo Grove, IL 60089 USA Tel: (312) 203-4926 Fax: (708) 459-8501 Cserve: 71303,3224

Mike Montgomery has asked us to include his address details as a focus for APL interest in South Africa. Will everyone who is interested please contact him!

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
ACM/SIGAPL	Donna Baglio	ACM, 1515 Broadway, New York, NY 10036 USA Tel: (212) 626-0806 Email: baglio@acm.org
Active Workspace Ltd	Ross D Ranson	Moulsham Mill Centre, Parkway, Chelmsford, Essex, CM2 7PX. Tel: (0245)-496647; Fax: 0245-496646.
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 589 5578 Fax: +61 3 589 3220
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: 31-3474-2337, Fax: 31-3474-2342
Andrews	Dr Anne D Wilson	23, The Green, Acomb, YORK YO2 5LL. Tel: 0904-792670
APL-385	Adrian Smith	Brook House, Gilling East, York. Tel: 0439-788385 Fax: 0439-788194 Email: 100331.644@compuserve.com
APL Bay Area Users Group		
APLBUG	Lewis H. Robinson (Sec)	1100 Gough St, Apt 14A, San Francisco, CA 94109, USA Tel: (415) 928-2058 Email: frgp21a@prodigy.com
APL Club Austria	Erich Gall	IBM Österreich, Obere Donaustrasse 95, A-1020 Wien, Austria
APL Club Germany	Dieter Lattermann	Rheinstrasse 23, D-59190 Waildorf, Germany. Tel: (+49) 6227-63469 Compuserve: 100332,1461
The APL Group Inc	Stuart Sawabini	644 Danbury Road, WILTON, CT 06897 USA. Tel: (203) 762-3933 Fax: (203) 762-2108
APL Interest Group, South Africa		
	Mike Montgomery	Private Bag X11, Rivonia 2128, South Africa Tel: +27 (11) 803 7200 Fax: +27 (11) 803 9134
APL People / Software	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
Association Francophone pour la promotion d'APL	Dr. Gérard Langlet	SCM, C.E. Saclay, F-91191-Gif sur Yvette, France. Fax: (33) 1 69 08 79 63
Atlantis Software	Arthur Whitney	1105 Harker Avenue, Palo Alto, CA 94301 USA
BACUS	Joseph de Kerf	Roolenberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS. Tel: 0272-730036. email: acamacho@clx.compulink.co.uk Reutemet (Sharp): ACAM
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS Tel: 0252-874697
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB. Tel: 071-404-5401. Compuserve: 100343,3210
Chicago SIG	Larry Mysz	836 Highland Drive, Chicago Heights, IL 60411 Compuserve:73040,3032
Cinerea AB	Rolf Komemark	Skyttegatan 25, S-193 00 Sigtuna, Sweden.
Cocking & Drury Ltd.	Romilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE Tel: 071-436 9481 Fax: 071-436 0524
CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 885168 Fax: +39 11 812 2652
CPCUG	Lynne Startz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850, USA. Tel: 301-762-9372.
David Crossley		187 Le Tour du Pont, Quartier Le Moura, 84210 ST DIDIER, France Tel: 90-68-08-87
CYBEX AB	Lars Wentzel	Gruvgatan 35B, S-421 30 V. Frölunda, Sweden. Tel: (46) 31-45 37 40. Fax: (46) 31-45 24 23.
Peter Cyriax Systems	Peter Cyriax	22 Hereford Road, London W2 4AA. Tel: 071-229-5344
Danish User Group	Per Gjerlof	Email: gjerper@inet.uni-cdk
Datatrade Ltd.	Ian Tomlin	1 & 2 Sterling Business Park, Salthouse Road, Brackmills, Northampton, NN4 0EX. Tel: 0604-760241
Dogon Research	Dick Bowman	2 Dean Gardens, London E17 3QP Tel: 081-520-6334 Email: bowman@apl.demon.co.uk
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein. Tel: +31 3474-2337
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL. Tel: 0256-811125 Fax: 0256-811130
E & S Associates	Frank Evans	19 Homedale Road, Orlington, Kent BR5 1JS. Tel: 0689-824741
Evestic AB	Olle Evero	Bertelliusvagen 12A, S-146 38 Tullinge, Sweden Tel&Fax: +46 778 4410

FinnAPL		Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland
General Software Ltd	M.E. Martin	22 Russell Road, Northholt, Middx, UB5 4QS. Tel: 081-884-9537
Grey mantle Associates	George MacLeod	Bartrum House, Ravens Lane, Berkhamsted, Herts, HP24 2DY Tel: 0442-878065 Email: 100412,1305@compuserve.com
Hartford CT Group	Bob Pomeroy	Mass Mutual Life, 1295 State St, Maildrop F465, Springfield, MA 01111 Tel: +1 (413) 788 8411x2838
H.M.W. Trading Systems	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA. Tel: 071-353-4212; Fax: 071-353-3325
HRH Systems	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 Tel: (703) 528 7624 Internet: dick.holt@acm.org
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics., LE16 8QL Tel: 0536-770998
IAC/Human Interfaces	Ian A. Clark	9 Hill End, Frosterley, Bishop Auckland, Co. Durham DL13 2SX Tel: 0398-527190. Email: clark.i@applelink.apple.com CS: 100021,3073
I-APL Ltd	Anthony Camacho (for queries, order forms) J C Business Services (for pre-paid orders only)	11 Auburn Road, Redland, Bristol BS6 6LS. Tel: 0272-760036 email: acamacho@clx.computelink.co.uk Reutersnet (Sharp): ACAM 56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept M45/D12, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: (408) 463-APL2 (~2752) Fax: (408) 483-4488 Email: APL2@vnet.ibm.com Cserve: GO IBMAPL2
Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
INFOSTROY	Alexei Miroshnikov	3 S. Tulenln Lane, St. Petersburg 191186 Russia. Tel:+7 812-3111611 Fax:+7 812-3153321 Email:aim@infostroy.spb.su
Insight Systems ApS	Morten Kromberg	Nordre Strandvej 119A, DK-3150 Hellebæk, Denmark Tel: (45) 42 10 70 22 Fax: (45) 42 10 75 74 Email: insight@inet.uni-c.dk
Interprocess Systems Inc.	Stella Chamberlain	11660 Alpharetta Highway, Suite 455, Roswell, Georgia 30076, USA Tel: (404) 410-1700. Fax: (404) 410-1773 Cserve: 70373,2676
Intelligent Programs Ltd	Mike Bucknall	9 Gun Wharf, 130 Wapping High St, London E1 9NH Tel: 071-265-1120
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel:(416) 925-6096 Fax: (416) 488-7559
JAD Software	David Crossley	580 Eyer Drive, #81 Pickering, Ontario, Canada L1W 3B7 Tel: (905) 837-1895 Fax:(905) 831-5172
Japan APL Association		23-2-302 Hiromichi, Adachi-ku, Tokyo 120, Japan
Kestrel Consulting	Mark Harris	Business & Technology Centre, Bessemer Drive, Stevenage, Herts SG1 2DX Tel:0438-310155 Fax:0438-310131
Lingo Allegro USA Inc.	Walter G. Fil	318 Half Day Road, Suite 297, Buffalo Grove, IL 60089 USA Tel: (312) 203-4926 Fax: (708) 459-8501 Cserve: 71303,3224
Melbourne APL Group	Harvey Davies	CSIRO Div Atm Res, Private Bag No.1, Mordialloc, Victoria 3195, Australia Tel: +61 3 586 7574 Fax: +61 3 586 7600 Email: hld@dar.csiro.au
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Hensage Street West, Aston Science Park, Birmingham B7 4AX. Tel: 021-359-5096. Fax: 021-359-0375
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 071-922 8866 Fax: 071-928 1006
Ellis Morgan		Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants. Tel: 0730-263843
New York SIG APL	Nestor Nelson	PO Box 138, NY 10185-0002, USA
Optima Systems Ltd	Paul Grosvenor	Alrport House, Purley Way, Croydon, Surrey CR0 0XY Tel: 081 781-1812 Fax: 081 781-1999
Potomac APL SIG	John Martin	Computer Sciences Corp, 1100 West St, Laurel, Maryland 20707-3587 Tel: +1 (301) 497-2699 Fax: +1 (301) 498-8260 Email:jam@acm.org
QB On-Line Systems	Phillip Bulmer	5 Surrey House, Portsmouth Rd., Camberley, Surrey, GU15 1LB. Tel: 0276-855880 Fax: 0276-855301
Renaissance Data Systems	Ed Shaw	P.O. Box 20023, Park West Finance Station, New York, NY 10025-1510, U.S.A. Tel: (212)-854-3078
The Rochester Group Inc.	Robert Pullman	50 S.Union St., Rochester NY 14507-1828, U.S.A. Tel: 716-454-4360. Fax:716-454-5430

Rochester APL	Gary Dennis	Sollton Associates, 1100 University Avenue, Rochester, NY 14607 Email: gsd@lpsalab.tor.sollton.com
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnetlibm.com
SE APL Users Group	John Manges	991 Creekdale Drive, Clarkston, GA30021 USA
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks., HP8 4QH.
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 Tel: +1 (818) 222-2759 Fax: (818) 222-9250
Sollton Associates	Laurie Howard	Sollton Associates Ltd, Groot Blankenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 646 4475 Fax: +31 20 644 1206 Email: ljh@sollton.com
SovAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region, Russia Email: askom@apl2.obninsk.su
Rex Swahn		8 South Street, Washington, CT 06793, U.S.A. Tel: 203-868-0131 Fax: (203) 868 9970
SWAPL	Stuart Yarus	PO Box 210367, Bedford, Texas 76095, USA Tel: (817) 577 0165 Compuserve: 73700.2545
SwedAPL	Gian Medri	Box 16181, S-103 24 Stockholm, Sweden Tel:+46 (8) 96 09 47
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: sl@ifi.unizh.ch
Sydney APLUG	Rob Hodgkinson	PO Box 1511, Macquarie Centre, NSW 2113, Australia Tel:+61 2 257 5313
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: (818) 222-2759 Fax: (818) 222-9250
Toronto SIG	Marc Griffiths	PO Box 384, Adelaide St Post Office, Toronto Ontario M5C 2J5, Canada Tel: (416) 532-0843 Email: marc@utoc.utoronto.ca Cserve: 76260,3314
Univware	Eric Lescasse	Tour Neptune, Cedex 20, 92086 Paris la Defense, France. Tel: +33 1 47-78-78-00. Fax: +33 1 40-90-04-11
Wickliffe Computer Ltd	Nick Telfer	76 Victoria Rd., Whitehaven, Cumbria, CA28 6JD. Tel: 0946-692589
Warwick University	Prof. Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel: 0203-523369
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: (203) 872-7806

Vector Back Numbers

Back numbers of Vector are available from:

British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO6 4JJ

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Les APL Etendus (par Bernard Legrand)

Masson, Paris, 1994 ISBN: 2-225-84579-4

reviewed by Adrian Smith

A Textbook on Extended APLs

APL2, APL*PLUS, Dyalog APL under Windows

First some statistics: this is a substantial and comprehensive textbook of APL comprising over 500 pages in 17 chapters; it covers all aspects of the APL language, and makes clear the differences in dialect between APL2 and the other two implementations, e.g. the '*Deux Philosophies*' (page G21) towards expressions like $A \ B \ C[2]$ where the '*Force de liaison*' differs in APL2.

All the chapters close with a set of exercises. These are nicely chosen to make you think, avoiding excessive repetition or trick questions which trip up the unwary reader and are liable to put him off APL altogether. It is nicely presented, with good indexing and a helpful set of summary tables showing (for example) all the possible reductions applied to the empty vector.

Coverage

Chapter A covers a little history, and introduces the various APL keyboards. Chapter B takes us through variables and assignment, simple arrays of various ranks, and vector notation. It makes clear the differences between *mixed* and *general* arrays, and also notes the index primitive $\square$ offered by APL2 and the extension to bracket indexing $MAT[(1 \ 2)(3 \ 4)]$ offered by Dyalog APL.

Chapters C and D are a straightforward introduction to the 'core' APL primitives (with a good discussion of reduction and behaviour on empty arrays), and the basics of function definition. Again, the different rules for forming the function header (such as the need to include $\{ \}$ around optional arguments in Dyalog) are well explained. Chapter E continues with a discussion of errors and the State Indicator (although I must take issue with Bernard's definition of *NONCE ERROR* as something specifically related to a failed interface call), and has quite a comprehensive comparison of the various tracing tools available in the different environments.

Chapters F and G are all about restructuring and transforming arrays. In general, Chapter F is 'classical' APL, with the addition of selective specification and the use (in APL2) of an axis modifier for $\uparrow$ and $\downarrow$. The discussion of dyadic transpose

is particularly thorough and supported with several helpful diagrams. Chapter G walks us through the use of the *DISPLAY* utility, and explains very clearly the meaning of the various indications it uses to hint at the type and rank of your data. It is also used to illustrate exceptionally clearly the different effects of *catenation* and *juxtaposition* on the structure of the resulting vector. This is (in my experience) one of the biggest problems for a 'flat' APLer to overcome in moving to an extended system.

Chapter H covers execute and (in some detail) format. Here the various APLs diverge dramatically, so the APL2 'format by example' is given a quite separate treatment to the use of *⎕FMT*. Both are again well illustrated with examples. Chapter I discusses operators, with some nice examples of inner products, and Chapter J sweeps up all the remaining math functions, with examples of *⊖* in curve fitting.

We now move on to the environment, with the various *⎕fns* (such as *⎕RL* and *⎕PW*) outlined in Chapter K and the basics of shared variables (including DDE from Dyalog APL) in Chapter L. Chapter M is a good basic introduction to file processing in all three APLs and Chapter N introduces the event-handling facilities. Unfortunately these differ so completely in the three environments that a sensible treatment is more or less impossible, and I doubt whether the coverage is sufficient to enable the reader to program these with total confidence and safety. Finally, Chapter O covers editing, from the ever-present *⌵* to the windowed facilities available with Dyalog.

Chapter P is a nice gentle introduction to the Dyalog approach to Windows development (unfortunately the book pre-dates the APL*PLUS III release) with some good examples and lots of screen-shots. Chapter Q has the obvious (but it needs saying) advice on documentation and housekeeping ("*Assurez l'hygiène de votre environnement*") and the book closes with some thoughts on 'Why APL Today'. Bernard takes many of the accusations levelled at APL and hits back at (for example) unreadability with some well-chosen counter-examples from Excel.

Summary

This is the first really good book I have seen which takes an integrated approach to the extended APLs, rather than grafting general arrays on to the back of an existing text. It deserves to be the Gilman & Rose of the 1990s. Someone should translate it, but please let that someone be an APLer with a smattering of French, not a professional translator who knows no APL!



SOLITON ASSOCIATES

SHARP APL: the high performance choice

SHARP APL is superior in the rapid development of mission-critical applications which meet the ever-changing demands of our customers.



***SOLITON provides SHARP APL
for MVS and UNIX with:***

- Superior productivity
- Ease in managing shared-file multi-user applications
- Powerful cooperative processing
- High performance DB2 interface
- Applications for end-users and programmers
- Responsive support services

For more information, telephone or FAX:

SOLITON ASSOCIATES LIMITED

44 Victoria Street, Suite 2100
Toronto, Ontario, Canada M5C 1Y2

Tel: (416) 364-9355 FAX: (416) 364-6159

In Europe

Soliton Associates Limited of Canada
Groot Blankenberg 53, 1082 AC Amsterdam
The Netherlands

Tel: +31-20-646-4475 FAX: +31-20-644-1206

In the U.S.

Soliton Associates Incorporated
1100 University Avenue, Suite 111
Rochester, New York, USA 14607

Tel: (716) 256-6466 FAX: (716) 256-6469

or via Internet to: sales@soliton.com

RECENT MEETINGS

This section of Vector documents all British APL Association meetings, and any other events of interest to the APL world. If you have recently attended any gathering which you feel would be interesting to Vector readers, please let the Editor have a brief note, and we will include it here. In this issue, we have notes (and photos - thanks to Ray Cannon) on the APL in Business Workshop, as well as first impressions of APL94 at Antwerp.



APL in Business at Swansea:
(front Row) Adrian Smith, Jon Sandles, Tony O'Hagan, Ellis Morgan,
Bob & Todd Brown, George MacLeod, Gareth Brentnall.

British APL Association Meeting *APL in Business* at EBMS, Swansea

Chairman's Impressions:

What you Saw was (more or less) what you Got

by Adrian Smith

After a lapse of a couple of months, it is interesting to reflect on what we set out to do at Swansea, and how close we came to achieving it. Of course the objectives were never totally stable — we were trying to do something quite new — but there were some fixed points in our agenda, and it is reasonable to ask:

- did we really show you the newest and best work being done under Windows on a PC?
- did you pick up any leading edge techniques of quantitative management?
- were you able to spend the afternoons taking apart the morning plenaries in a well-equipped laboratory full of fast PCs?
- was it a revitalising break from the daily routine?
- did you get to take away some useful code?

These were all promises that we made on the first page of the invitation: how well do you think we met them? So far, those of you who have written or e-mailed have all been remarkably positive (which makes me feel better), but such samples tend to self-select! I might yet get around to writing to everyone who came, to ask how we could do it better next time, but meanwhile, here are my views on what we did well, what we did badly, and what we left undone.

The Venue, Format and Organisation

Swansea was born out of our failed bid to run APL94 at York, and for the much smaller workshop format, I think it was close to ideal. We began by over-estimating the size of the gathering, which caused quite a few headaches as we cut back to match the bookings, but it would have been hard to run such an intensive 3-day workshop with many more people.

The EBMS was a good self-contained base, the only snag being the need to borrow facilities in the nearby computer lab for some of the more heavy-duty

afternoon sessions. However most people found their way around, and in the end most of the equipment co-operated, so this somewhat ad hoc approach stood the test — just.

Several people commented favourably on the basic structure of running a morning plenary, with everyone attending all the sessions. This had the great advantage of simplifying administration, as you could always be sure that everyone had got the message about the bus times, or the change of room in the afternoon. I hope the more esoteric statistical stuff didn't cause the old-time APL hacks too much mental indigestion — I thought Derek Bissell's talk on Control Charts was brilliant, but then again I learned my statistics by weighing Kit Kat bars! I honestly think that if we had gone for a straight-down-the-line Windows + GUIs agenda, we would have turned everyone off by the end of Day-1. It needed breaking up a bit, and talks like Kath Dowsland's on Genetic Algorithms were ideal.

I can take no credit at all for the excellence of the organisation! This was all set up by the Alans (Sykes and Mayer) and administered most capably by Haydn Williams and Pam. The coffee was on time, the signposting was pretty good, the overhead projector worked all the time — what more can you ask?!

The Main Topics

We took a quick poll around the room at the start of the first day (see Jon Sandles' notes on page 45), and most people had really come for the experience of Windows programming. We had already warned everyone (Vector 10.4) that this was not a jolly, and I think we went home feeling mentally quite battered by the amount of new stuff we had been exposed to in such a short time.

- Day-1 was short of 'talk and chalk' time for the main topic of programming the GUI APLs, so the afternoon workshop was something of a rabble. I had failed to appreciate the problems of class-control when you take 24 keen APLers, sit them down in pairs in front of a new toy, and try to get them to listen to you. In the end, Duncan and I gave up any attempt at formal teaching and just worked around the various groups as fast as we could. It was OK, but we did much better on Day-3 (see Ray Cannon's note on the DLL workshop) when we prepared more material in advance.
- Day-2 was about client-server, timeseries and graphics. I found Morten Kromberg's lecture on the basics of effective client-server architectures really superb, and I'm sorry I missed the afternoon workshop. The idea of a room full of chunky 486 'clients' all happily running database queries against Morten's dinky little SparcStation 'server' was remarkable enough. What's more it all worked!

- Day-3 was initially scheduled for Microsoft, who are anxious to tell the world about OLE2. In the end, we were too small a gathering for them, and instead we had John Scholes on Namespaces in Dyalog-7. I think that for many people this was actually a bonus, and namespaces were a hot topic for the rest of the conference. In the afternoon we had a much better organised workshop on DLLs; we need to remember how we did this one and use it as the pattern when we do this kind of thing again.
- by Day-4 everyone was pretty shattered, so we had a gentle 2-hour canter through some simple GUI applications, made sure everyone got the most up-to-date software (the workshops turned over quite a few bugs) and unwound gently ready for the trip home.

To summarise, I think that we delivered most of what was promised, but I was too deeply buried in running the thing to get a clear view. Let us know what you think, and the BAA may try to do it all again, better.

Opening Plenary:

An Introduction to Windows Programming with APL

Adrian Smith

reviewed by Jon Sandles

After being welcomed to Swansea by Alan Sykes, Adrian took to the stage decked in a wolf T-shirt (more of which later) and introduced himself. He gave a brief history of his work with and without Nestlé (and Rowntree Mackintosh before). He acknowledged and thanked Nestlé for supporting him in the past and the present. He talked about writing APL in a Windows environment and said that a running theme of the conference would be 'Causeway' — a project he and Duncan Pearson had undertaken to make Windows programming in APL simpler and more portable.

Adrian then asked everyone to introduce themselves — this technique (with a reasonably small conference anyhow) I would recommend to conference organisers in the future. Its effect was that everyone was more relaxed and more at ease, and you immediately knew who everyone was. You had a starting point for conversations and you also could approach some of the speakers before their talks. Adrian asked for name, what you did and what you aimed to get out of the conference. Here is an extract of who was there and what was said (other delegates that I missed/weren't there appear at the bottom of the list!):

Delegate	Job	Interests at conference
Jaakko Laakso Dr Kimmo Kekalainen	Statistics Finland Finnish Dyalog APL user	Gui programming/Causeway Experienced in Adrian's rain/here for Causeway Causeway Social IT correspondent! Inspiration and Gui/Causeway Windows/Causeway Stats/forecasting Looking for answers.....? Wants to teach us! Need Gui/Causeway for banks Here to tell us about Dyalog version 7 Gui/Causeway for APL*PLUS - Windows port Gui/Causeway for APL users. Gui/Causeway Gui/Causeway & to tell us about Client Server Finishing APL*PLUS III version of Causeway Gui/Causeway Gui/Causeway Gui/Causeway Gui/Causeway & to tell us about Stonehenge Gui/Causeway
Marc Griffiths Diane Whitehouse Gitte Christensen	Editor Toronto APL News Co-editor Toronto News Danish SIGAPL	
Nicholas Small Dave Parker Dave Phillips Moshe Sneiderovich Chris Hogan	Nat West Electricity board Cocking and Drury Mathematics, Melbourne HMW	
John Scholes	Dyalog APL	
Phil Last	HMW	
Jim Ryan Ross Ranson Morten Kromberg	Mobil R&D Self employed Insight systems	
Duncan Pearson	Compass R&D	
Dr Harri Heinonen R Naumanen Gordon Ramer Ray Cannon	Finnish Forest Service Chair FINAPL Ezer & Associates Commercial Union	
David Wiley	Karsten Manufacturing (PING Golf Clubs) Institute of Technology, Moscow	
A Samylovskiy		
Alan Hawkes Mark Caslin Paul Smith Tony O'Hagan Ellis Morgan	EBMS, Swansea Valuta Economics Self employed Statistician Self employed	Gui/Causeway Here to see BISSEL, not Gui! Stats/Currency modelling OR techniques APL & Aplers are fun! Planning/forecasting regression diagnostics Gui/Causeway SQL/APL/Causeway Business graphics / Rain / Causeway Gui/Causeway Gui/Causeway Gui/Causeway Gui/Causeway to support her husband's lifestyle! Forecasting, Gui/Causeway, Sell his book! Gui/Causeway, statistics C++ Windows to Gui/Causeway
Bob Brown Todd Brown George MacLeod	Vermont, USA Vermont, USA Self employed	
Gareth Brentnall Alan Mayer Anthony Camacho Sylvia Camacho	Mercia Software (Logol) EBMS, Swansea Editor Vector! Secretary BAPLA	
Andy Pole	Blackhawk LP	
Jon Sandles Michael Baas	Nestlé UK ADAPTA	

(Apologies for any omissions, mis-spellings, mis-quotes etc. — it was hard to keep up with everyone!)

Adrian (after cunningly killing 30 minutes by getting us to do all the work) began the workshop proper. He stressed the importance of the logical design of Causeway and told us the architecture will be formally fixed as from Friday (once we've fed back requests and bugs!).

Adrian was asked whether Causeway has enveloped the OO methodology. Adrian stressed that Duncan and himself had stayed well clear of producing a VB clone, Duncan acknowledged the influence of Ian Clark's *APLOMB*. Later on Adrian admitted that there were aspects of Causeway that were not unlike OLE2, the new Windows standard for application developers. (Is it not inevitable that if a tool is a good Windows programming tool then it will fall under the OO banner?)

Adrian reminded us that APL is a rapid prototyping tool and that this holds even truer in the Windows environment — if you have a full functional specification you might as well use C++, if not then prototype in APL. (Of course we never bother to rewrite in C++ once it's finished!) Causeway allows you to easily develop a Windows interface and lets you concentrate on the APL code beneath.

The Windows programming style is encompassed within the '*Microsoft Application Design Guide*' (read it and then read it again). Once you have realised the complexity in writing real Windows systems (what if no mouse? — tabbing — fonts — styles — Alt+keys etc.) you will realise there is a lot of work in putting together good Windows software. Causeway is meant to do a lot of this nitty-gritty automatically — leaving you to write the APL code (put away your SDK now).

Adrian told us how some objects automatically know how to behave in a Windows way using Causeway. A Close button will automatically know how to close a form, a PostScript object will already know how to draw a graph. Very little code is needed to enable basic Windows features.

Adrian introduced us to his new *APLthin* font with which he would like to take over the world. Along with Causeway this should help remove a lot of the incompatibility between Dyalog/Manugistics and other Windows APLs. He reassured the Finns in the audience that the Scandinavian characters were present and correct in their ANSI positions.

Message: If you have a standard front-end — have a standard font.

Adrian capped off a stirring performance with the classic Microsoft brains joke. (I can't reproduce it here — Bill has better lawyers than I have.)

You always walk from one of Adrian's talks knowing he's just sold you something and made you quite excited — but you haven't got the foggiest what.



Tony O'Hagan demonstrating to
Alan Sykes, George MacLeod, Alan Hawkes

Gareth Brentnall's Logol Workshop

notes by Jon Sandles

Gareth was showing TWIGS — a workspace that comes with the Logol suite that allows you to explore and experiment with some of the functionality built into Logol. It also contains some of the functionality that has NOT been built into Logol, because they would like you to find out for yourself why it doesn't work!! I played with lotsizing and bin-packing routines, and also the least squares Fourier fits.

Duncan Pearson's DDE and DLL Workshop

notes by Ray Cannon

This very successful workshop was composed of a set of printed exercises, notes and detailed descriptions of several supplied utility functions within the Windows Kernel DLL. It was taught in a room equipped with many 486 PCs completely set up with Excel, Word, Dyalog/W and APL*PLUS/III under Windows.

Although all the main points of Dynamic Data Exchange and Dynamic Link Libraries have been reported within Vector, there can be few readers who have had hands-on experience of them under both major vendors' APLs on the same machine.

Duncan's notes were both clear and concise, and highlighted the differences in use between the Dyadic and Manugistics offerings.

By supplying the correctly set up environment (hardware and software), the notes on what to do and the information sheets specifying the syntax details and other data, Duncan created the ideal environment for exploring DDE and DDL. We were not spoon-fed with complete scripts, but were given all the required information, and had to understand the implication of what we were doing to proceed.

The main advantage of a workshop such as this as against trying it at home (relying on, say, an article in Vector) is that we were not distracted by unrelated problems (such as incorrectly set up or incompatible software/hardware). Whenever I have tried playing at home with new features of this sort, I have always found I had a set-up different from the one used by the author of the guiding article, and have been constantly sidetracked finding out why things didn't work as the article said they should.

All in all I found this workshop one of the highlights of the conference. It alone fully justified my attendance.

Thank you Duncan.

Correspondence

From: Ellis Morgan

25 July 1994

I am writing to let you know how valuable I found last week's *APL in Business at Swansea*.

It was the first conference I have been to since I started working for myself, so I was apprehensive about both the cost and the week of lost earnings. I now feel that the time and money were well spent.

I am sure that the chance to see how others are using the extended capability of APL in today's desktop and client/server environments, the hands on training and practice in accessing windows resources and the software you distributed will add to the quality of my work by widening my repertoire while saving me time.

Thank you both for organising such a worthwhile and enjoyable conference. Thanks also to all the people at Swansea who looked after us so well and to the contributors, particularly those who put so much effort into the workshops.

Myrtle Farm
Winchester Road
Stroud, Petersfield
Hants. GU32 3PE

Croesoe a Cymru and the APL Welsh Hwyrnos

reflections by Diane Whitehouse

Over fifty attendees came to Wales to APL in Business from ten countries.

While the agenda was a serious one, uppermost in delegates' minds was that, as one self-proclaimed "hacker" said: "APL is fun and APLers are ... also fun" Here are some of the delegates' more weighty reasons for attending the conference: "I'd like to test fly some GUI ideas"; "I want to expose CAUSEWAY to some hard core techies and encourage them to break it"; "I'm starting from ground zero"; "I'm at Swansea because I'm being forced very reluctantly into this Windows thing"; "I know nothing so that's why I'm here"; "I want to turn my head around"; and "I'm hoping to acquire a set of relevant tools." Less seriously, perhaps, one veteran APLer commented, "I'm here to learn something about GUIs to support (my husband's) lifestyle."



**Jon Sandles, Adrian Smith, Alan Mayer, Duncan Pearson,
Anthony Camacho (asleep?), and Sylvia Camacho**

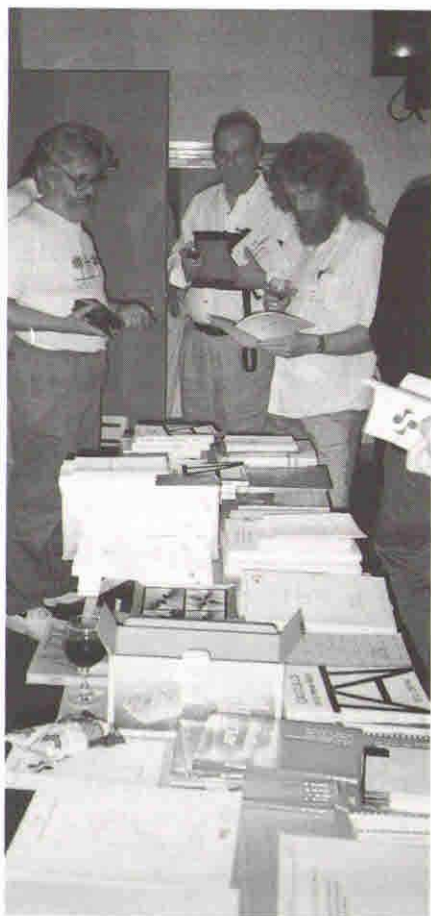
While this was a very intensive and exciting three-day conference, there was also plenty of opportunity for discussion, dialogue, and socializing. Following on from many of the papers and computer lab sessions, strenuous debate continued long into the night over pints of beer. Devotees extolled the virtue of local bitters at the local Rat and Carrot or the "Pub by the Pond" for just 89p a pint. (Forecasting presentations by both Andy Pole and Gareth Brentnall partly used beer consumption as models. In Gareth's case, he also referred to bulk paracetamol because "those of you who were in the bar late last night probably needed Paracetamol Bulk this morning.")

The Street Market

A more sober event was the APL street market, located on "EBMS Street". Among other items, there were on display plenty of British APL Association materials, a cunning APL *vade mecum*, and an excess of Toronto-based *Gimme Arrays!* In attendance, huddled over their portables, were Jon Sandles, Ellis Morgan, and Morten Kromberg offering previews of their wares (to be introduced in more detail later in the week). Some of the juiciest conversations centred on what is happening now in the various APL companies. And some of my personal favourites involved swapping local news and ideas with Danish, German, and Finnish APL newsletter editors.

At the street market, I spent some time with Jon Sandles of Rowntree Nestlé. Using Adrian Smith and Duncan Pearson's CAUSEWAY, Jon has developed a program called BLOCKBUSTER. It calculates how to distribute pallets across the floor of the new product warehouse to minimize loading stresses on the floor. In an influx of (presumably) Smith humour, "Holler" is the naming convention for notification of excess weight differential on any section of the warehouse floor.

Many other discussions took place on long strolls on the glorious Swansea Bay beach, around the newly-gentrified Swansea Docks, or along the seashore to the quaint village of Mumbles. Short car journeys into the surrounding hills brought a climate so pure that Putney's Kew Gardens are considering relocating their bedding grounds to this south coast idyll. We were lucky enough to share in, what was referred to as, the best summer weather in Wales since 1976...



The Bookstall at the Street Market
Anthony Camacho, Jim Ryan, Phil Last

Music was a great source of delight during the week. One colleague said how much he enjoyed the whistled baroque melodies, courtesy of Phil Last, every morning bright and early which were a pleasant reminder of Radio 3's morning programme, *Aubade*. Italian and Spanish discos abounded on campus. And Saturday night carousing in Swansea produced the most melodic, harmonious, but drunken, serenades: at 2am in the city's high streets, men's choirs competed with women's.



Part of the Crowd at Hwyrnos

Among the more formal social occasions organized at APL in *Business* was a dinner with speaker, Canon Don Lewis, who captured the spirit of the Welsh valleys with his stories of four-times married Welsh women. A Welsh eating, singing, and clog-dancing evening — a *hwyrnos* — added to the week's enjoyment, and brought out celebrity style in APL delegates from as far away as Finland and Russia. While we didn't have to wear leeks to get through the pub door, we almost all emphatically adopted local Welsh accents. Enamoured by the lilting refrains of local players reciting from Dylan Thomas's "Under Milkwood", we were, by the end of the week, greeting conference stalwarts, Haydn and Pam Williams, with the familiar, "And how are you today, my lovelies?" The answer is that we were all fine. The week was a great social, and intellectual, success. *APL in Business* was one of the most educational — and enjoyable — working conferences this delegate has attended in some years.

Diane Whitehouse was taking her mind off the more serious business of PhD-writing and newsletter production by reviewing the social aspects of APL conference attendance in Wales <dianew@utcc.utoronto.ca>.

APL 94 Roundup: Impressions of Antwerp

by Anthony and Sylvia Camacho

We delayed going to press by a couple of days to get this preliminary report to you. We hope to include a more comprehensive selection of photographs and notes in Vector 11.3.

The world having pressed heavily on us during the three months before the Conference, we left our booking very late. As always we had awkward questions such as how to deliver the delegate copies of Vector, and how to arrange for the British APL Association's promises to the Russian Fund to be fulfilled. To all our challenges Rita Peys and the Conference Office responded patiently, promptly, and in impeccable English. We should like to take this opportunity to thank her and her staff for their support.

The Highlights

You discover what was outstanding about a conference by listening to how you describe it to others. By this standard the highlight of APL94 was nothing to do with APL: it was the cultural evening — which Sylvia will long remember as the occasion "when I laughed so much I feared my contact lenses would be washed right out of my eyes". Take my advice, never miss a chance of attending a performance of "The Frivolous Raspberry". This duo did spoofs, take-offs, send-ups, pastiches and gross injustices to a huge range of music from Schubert Lieder to a 10-minute "Traviata" which contained bits of every opera you have ever heard in the best Anna Russell tradition ... They pretended to have forgotten the tune of the *William Tell* overture, and to re-construct it from Phil Benkard's humming!

As for the serious Conference business Sylvia was impressed by the energy and success with which the Japanese have been promoting APL, particularly among schoolchildren. Also the continuing work that Walter Spunde has received funding for in Australia; his down-to-earth advice on teaching undergraduates should be heeded. In addition the competence and enthusiasm of the Russian students makes the continued support of the APL community there seem well worth while. But the highlight was to watch and listen while Gérard Langlet issued challenges to several scientific disciplines with his inimitable verve, and of course ≠\.

The Venue

The Conference Venue, very imposing and elegant, was perhaps not quite in keeping with the informality of most delegates to these APL occasions. However, the opening plenary session was intriguing and very aptly reflected the artistic surroundings. The Province House is a very grand provincial government building. The plenary theatre was huge, with excellent facilities (except that, as usual, the screen display wasn't sharp enough). The other rooms were also comfortable and well-furnished. The corridors and open spaces were decorated with works of art, and there were attendants to prevent you from taking coffee where it might damage the decorations if spilled. The Province House closed early so the sessions at the end of the day could not be extended - people in distant hotels had to leave early to be sure they would not miss the bus. The meeting place was 15 minutes' walk away so, except for events that took place there, it was not much used.

The delegates stayed in five hotels. Some were within walking distance. The cheapest (where we stayed) was about half an hour away by bus; we were pleased with it. I am sure the facilities could have handled a conference of three times the numbers: there were 164 names in the list of participants and maybe a few people turned up at the last minute or for single days.

The Programme and Presentations

Fair comment on the programme would only be possible if three people between them attended every session. I fear my choice may have been unfortunate. APLers on the whole need more rehearsal than they get, and care too little about their audience. Many times someone from the audience had to ask for a larger font on the display or to focus the overhead projector.

The plenary sessions were mixed. The opening plenary was a brilliant survey of the loose connection between symbolism and ideas. It was full of superficial or deep ideas according to your point of view. "The consequences of the symbols are the symbols of the consequences." [L. von Bertalanffy 1965] For my part I cannot see how the insights of the artist can produce useful scientific advances. In hindsight one can say "Ah: he foresaw what Einstein discovered" but, like the prophecies of the I-Ching, you never know what they really foretell until it has happened.

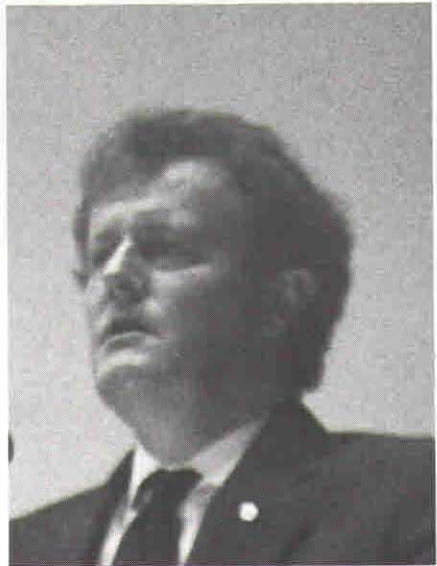
The panel on the future of APL was a disaster — a disconnected set of comments by clever people, some of which had some force — but no resolution of any disagreements and no possibilities of reaching any. One rather wondered whether Phil Benkard was right to say the future of APL is its past.



Arthur Whitney with Walter Spunde and Rob Hodgkinson



Joseph De Kerf opens the Conference



Alain Delmotte introducing the Programme

Other plenaries were sales pitches (Soliton, Arthur Whitney, Dyadic, Iverson Software, Manugistics and IBM) or history (APL in Japan and Origins of APL2).

The SigAPL Meeting

The open SigAPL meeting was introduced in a very upbeat way by Dick Bowman. SigAPL have done wonders. QuoteQuad appears on time or even early. The bank balance is rising. Membership has been stabilised (i.e. it is no longer falling). There is a location for APL95 (San Antonio). Regrettably there was nothing after the introduction. Nobody knew the bank balance or how many members there are or what procedure is used to choose a location for APL96.

The Banquet

The banquet, boisterous and with excellent food, was also the occasion for the presentation of the SigAPL Outstanding Achievement Award.

The restaurant was a long way outside Antwerp in a restored farmhouse. There was champagne (as at the Welcome reception), the food was excellent, and — as always — the company was marvellous. Stuart Yarus presented the SigAPL Outstanding Achievement Award to Donald McIntyre. The audience was obviously delighted by the choice, and Donald charmed them with his response. Apart from having done so much for APL and J, Donald is very well liked, has done a great deal unpaid, without hope or expectation of gain, and has no enemies. Many APLers are altruistic and like to see it recognised. It was also nice to hear the J work that Donald has done in the citation.

The banquet was very expensive at £52 a head. Indeed, the conference in general was a very expensive one and I suspect some APLers who would have had to pay for themselves may have been deterred by the price. I nearly was, myself.

The Arrangements

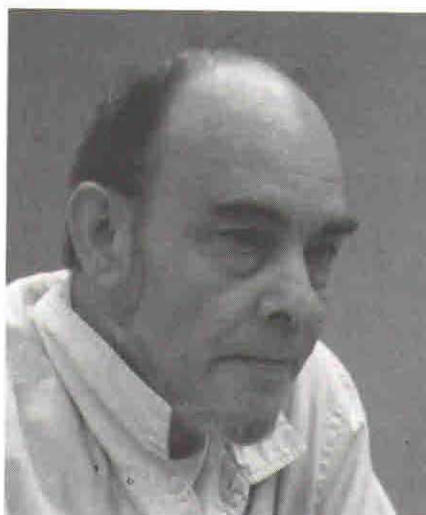
Practically everything ran very smoothly. Those in the know attributed this to Rita Peys and her secretary. We owe them our thanks and also thanks to Alain Delmotte for organising the programme.



The IBM Stand: Eke van Batenburg, Ray Polivka, and Dieter Lattermann



Pete Donnelly and John Daintree man the Dyadic Stand



Gérard Langlet at the Binary Algebra Workshop



Ilya Mironov (age 18), Ludmilla Lemagnen (organiser of the Russian Fund) and Victor Bargatchev (age 16)



Donald McIntyre accepting the Iverson Award



Michel Dumontier with an alternative solution

Reflections of Another

by Ray Cannon

I saw a lot of reflections of Antwerp, since it rained every day I was there. Luckily I did not go for the sight-seeing, but rather for the APL, or rather, the APLers. I made a few new friends, and met many old ones, but sadly many were missing.

Most of the papers covered the "same old ground" but there were a few papers breaking new ground, in particular Pavel Luksha's *"Probabilistic Studies of I CHING with the Help of APL"* and Gérard Langlet's *"The APL Theory of Human Vision"*.

As for the Vendors, in the Windows arena Manugistics and Dyadic were battling it out, both giving good accounts of themselves. IBM appeared to be retreating into its OS/2 corner while Iverson Software's J for Windows seems to be gaining ground. J Locales vs Dyadic Namespaces appears to be the next big fight, or is it between J and Manugistics over control structures?

The closing session was *"Acquisition and Processing Data from Normal and Cancer Cells"* by the Nobel prize winner Dr Ivar Giaever. As an APL consultant I found this session very interesting. More importantly however, it contained two messages to all APL vendors. If you want to sell APL (or J for that matter) to scientists doing research you must have direct IEEE support and you must have a Plot primitive. (This last point about a plot primitive was also made by Dr Walter Spunde in his paper on the *"The Impact of APL on First Year Mathematics"*.)

And finally a few key phrases.

"The consequence of the symbols is the symbols of the consequence."

"Peace in our time, stop the war between APLs."

Lots of "Little Trojan horses" in our code.

Vendors saying that their Windows Driver was easy to learn is a bit like the Nepalese Tourist Board saying:

"We have lots of very nice mountains, and they are all very easy to climb."

Binary Algebra Workshop (Foil)

by Gérard Langlet

These are the foils from Gérard's binary algebra workshop at Antwerp, reproduced from his masters.

Please read them in the order:

1	2
3	4

Binary algebra can be considered as the union of two isomorphous algebras :

- Logical algebra in which 1 is **True** while 0 is **False** :
1 is NOT 0 0 is NOT 1

- Modulo 2 integer algebra in which 0 and 1 are indeed numbers :

Then, 1 is $1-0$
while 0 is $1-1$

In both cases, the interpretation of 0 and 1 differs.

However, Boolean functions, especially in APL, are the **actors** for both algebras

Most arithmetic functions *either* disappear (becoming useless) or coalesce.

The monadic logical function NOT can always be expressed as $1 \neq \omega$ which, in modulo 2 integer algebra, is either $1+\omega$ or $1-\omega$ taken modulo 2.

The monadic SIGN
function vanishes :

SIGN ω is just ω

The monadic inverse
vanishes, as well as
arithmetic division
does :

$\div \omega$ is ω when ω is 1
and is forbidden if ω is 0

The case of the monadic
factorial is less clear :
if $! \omega$ is 1 when ω is 1, never
forget that $1 \equiv !0$ is
nothing but a convention
(as a limit of the Γ fn)

2-argument functions

$+$ and $-$ coalesce into
(plus/minus) modulo 2
which corresponds, in
logic, to the

EXCLUSIVE OR

(XOR for short, $\neq$ in APL)

Immediately, $+ /$ i.e. Σ
the discrete equivalent of

a defined integral $\int_a^b$

becomes expressible as

$\neq /$

but $\neq /$ also expresses $- /$

Since e and π may not be
directly defined in
algebras which only allow
to use *either* 0 or 1, there
are no π -times or
exponential monadic
functions.

HOWEVER, we shall see that
binary algebra may express
trigonometry and even
complex trigonometry in a
much better way than what
we know about conventional
arithmetics - which was
developed long before the
Computer Era, then is far
from fitting the capabilities
of the computer.

$\times$ (*multiplication*) as well
as L (function *minimum*)
coalesce into the Boolean
 $\wedge$ i.e. **AND**.

However, multiplication by 1
does not change much the
result.

And multiplication by 0 is
equivalent to a NO-selection of
the term.

0 raised to any positive power
 p is 0, while 1 raised to any
power is 1 : then, function
power vanishes, and,
consequently, non-linear
polynomials become linear.

So, for any problem which may have a solution, the fact that one will be able, from now on, to set the "equations" with 0s and 1s ONLY, instead of using conventional algebra, WARRANTS that one will also be able to find the solution(s), using linear algebra (and nothing else).

Of course, the convenient algebra will be IM2A i.e. (INTEGER) MODULO-2 ALGEBRA in which very few necessary operations will be processed using isomorphous logic.

Bitmaps are M2A matrices : Pixels (PELs in IBM jargon) are M2A bits.

The most important algorithm for image processing (after a quite long quest of references in a variety of fields - research and technology) appears to be the FFT and other discrete orthogonal transformations.

Outside image processing itself, the most-used algorithm is still... the FFT.

Apart from the ubiquitous FFT, the most-used algorithm in science is matrix inversion (especially for fitting).

This is the reason why I devoted the past years to develop MATRIX ANALYSIS research in M2A exclusively, giving a new eye to Gafols'work (1811-1832) thanks to APL, to topology and, of course, to the computer.

We shall consider, from now on, square matrices only, containing *either* 0s or 1s only.

Any non square matrix, and, even, any data structure with some bizarre planar shape containing INFORMATION may be embedded into a square matrix.

BOTH are related.

So, what is matrix inversion in bits ?

Then, what is a FFT in bits ?

A bit is a box which contains *either* 0 or 1, completely independently from the *meaning* of individuals 0s or 1s.

The behaviour of 0s or 1s becomes collective.

Matrix algebra is a collective algebra for INFORMATION.

Déclaration des Droits du Bit (Antwerp, 1994)

0. All bits contain information,
So, they deserve respect.

1. Every small bit is a quantum
of information.

Nobody has the right to crunch it.

Nobody has the right to smooth it.

Nobody has the right to truncate it.

Nobody has the right to average it
with it(s) neighbour(s).

Nobody has the right to sum it,
except modulo 2.

Bits like *SYMMETRY*

0 is a symmetric of 1

0 1 is a symmetric of 1 0

$(1 \neq 1 \ 0) \equiv \Phi \ 0 \ 1$

Matrix $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ is
symmetric

So is matrix $\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$

Matrix algebra is the united
crown of symmetry, whose
jewels dazzle when considered
modulo 2

The 16 Integer Modulo-2 matrices with shape 2 2 :

0 0	0 0	0 0	0 0	0 1	0 1	0 1	0 1	1 0	1 0	1 0	1 0	1 0	1 1	1 1	1 1	1 1	1 1
0 0	0 1	1 0	1 1	0 0	0 1	1 0	1 1	0 0	0 1	1 0	1 1	0 0	0 1	1 0	1 1	1 0	1 1
	MAI																
0 0	1 0	0 0	1 0	0 1	1 1	0 1	1 1	0 0	1 0	0 0	1 0	0 0	0 1	1 1	0 1	1 1	1 1
0 0	0 0	1 0	1 0	0 0	0 0	1 0	1 0	0 1	0 1	1 1	1 1	0 1	0 1	0 1	1 1	1 1	1 1

∇ R+MA

[1] R+32p 3 0 2 0 ◊ R+Rv 2 2 T,4 4 4 4 4 4 4 4 T 4539 2570 4539 24415 ∇

∇ R+MAI

[1] R+32p 3 0 2 0 ◊ R+Rv 2 2 T,4 4 4 4 4 4 4 4 T 4369 2655 48059 2655 ∇

0 1	0 1	1 0	1 0	1 1	1 1
1 0	1 1	0 1	1 1	0 1	1 0

0 1 0 1 1 0 1 1 1 1
1 0 1 1 0 1 1 1 0 1 0

Matrix inversion is performed,
for such matrices, by a simple
symmetry operation :

commutation of the elements in
the *main diagonal*,
independently from the fact
that the matrix is invertible or
not invertible.

Modulo 2 *strict* Matrix Inversion
of the 6 matrices with rank 2,
which form a *group*, already
give the clue for the equivalent
of the FFT (*which is linear*).

There **MUST** be an *involution*,
then assumed by a non-trivial
self-inverse matrix,
already for a 2-bit signal.

1 0 does not transform
0 1 anything (unit matrix)

0 1 reverses the couple
1 0 of bits (trivial NOT)

1 1 and 1 0 will do
0 1 1 1 the job.

No need for determinant
computation.

The determinant is a scalar
which must be *either* 0 or 1.

If 0, the matrix is *not-strictly*
invertible.

If 1, the matrix is *strictly*
invertible.

The commutation of diagonal
elements does not change the
determinant for a matrix with
rank 2.

So, this permutation does not
make a strictly-invertible matrix
from a non-strictly invertible
matrix (and conversely, by
symmetry).

1 1 and 0 1 will also do
1 0 1 1 the job,

not as orthogonal transformers,
but as **Trigonal** transformers.

(cfr. The Theory of Human Vision).

This is a very simple proof.

Other proofs were found :

a) by Boole Polynomials, once
the Cognitive Involution had
been found in Paritons,

b) by formal computing (in APL),

c) by the princip of invariance
(looking for the equivalent, in
bits, of a Gauss curve, *shape-*
invariant of the FFT).

The Neo-Numbers

A Neo-number is a square matrix $\in \{0,1\}$.

Neo-numbers may be added modulo 2, itemwise, so that the neutral element for addition of neo-numbers is an all-0 matrix.

The inverse operation of addition is itself.

$\neq$ will perform addition .

Neo-numbers may be multiplied modulo 2, as matrices, so that the neutral element for multiplication of neo-numbers is a unit matrix.

$\neq . \wedge$ will perform multiplication.

Consequently, any neo-number, raised to matrix-power $p+1$, reproduces itself, while the same neo-number, raised to matrix-power $p-1$, reproduces its (modulo2) strict matrix inverse.

Matrix inversion itself has then become a **linear** algorithm.

Some neo-numbers are also named *biterions*, when they exhibit *either* binary i.e. 2-fold circular symmetry (all self-inverse neo-numbers), or ternary i.e. 3-fold circular symmetry (genitons, i.e. symmetric Sierpinski matrices).

0 and 1, individually raised to any power, do not produce new results.

But neo-numbers, as **collective** patterns of 0s and 1s, raised to integral powers, DO :

When their (modulo 2) determinant is 1, power -1 makes sense and corresponds to the strict inverse matrix, and power 0 is a unit matrix.

Then, all finite neo-numbers are members of a *cyclic group* with order p , such that a neo-number, raised to power p , i.e. (modulo 2) matrix-multiplied $p-1$ times by itself, equals a unit matrix.

Let M be a neo-number the successive columns of which are the base-2 decomposition of the 16 following integers :

10153	36036	45591	4331
34151	15707	8376	10511
147	19736	15590	43989
43873	17927	48758	43736

Here are respectively the following modulo-2 matrix powers of this matrix, displayed in pixel mode :



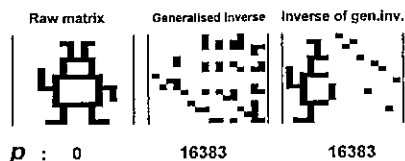
p equals 47523 ($p < 2 \star 1 \uparrow pM$).

M is then exactly equivalent to some complex number which would be inaccurately coded in "regular" complex arithmetics, i.e. in mathematical notation :

$$e^{2\pi i/47523}$$

The group of the 147523 successive matrix powers of M form a regular polygon which could not be distinguished from a circle, if displayed on a high-resolution video screen.

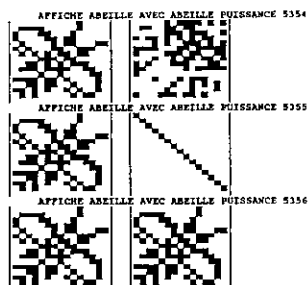
However, the matrix is coded in 256 bits, the equivalent of 2 regular complex numbers in regular double-precision arithmetics.



ROBY (°) is not invertible.

Its non-strict (generalised) matrix inverse is an invertible matrix, the strict inverse of which reproduces as much as it can the original matrix : a kind of *fuzzy* remembrance.

Much more than *quaternions*, the neo-numbers offer the possibility to code, with as much precision (density of information) as required, any point of the complex plane, in association with a very long-precision always-positive number, which would play the role of a common factor for all terms, as an equivalent of the module in *polar* representation of complex numbers.



(°) introduced at APL92, *Непербырз*

GENERAL ARTICLES

This section of Vector is intended for articles about the applications of APL, and non-technical (i.e. having a minimal content of actual program code) material about the development of the language.

"A Sea of Tribbles"

Due to technical difficulties (the complete failure of the mother-board on his PC) Peter Merritt has been unable to complete the main part of his article *"To Be, Or Not To Be – That is the Gazodenplatz"* in time for this edition of Vector. With luck, it should appear in the next edition. Apologies all round.

In the mean time, so as to prevent any more sleepless nights wondering what the message was, the decoded script is reproduced here.

Mary had a little lamb.
Its fleece was white as snow.
And everywhere that Mary went.
The lamb was sure to go.

(CLUE The message is a matrix of rank 2, shape 4 by 30. Note that the message contains both upper and lower case letters, but beside the space only one punctuation character, shown here as a full stop.)

With this extra help you should now be able to work out how the message was encoded, remembering of course that it was all done in APL. (It took me a little over an hour or so.)

Ray Cannon for and on behalf of Peter Merritt.

Windows Development with APL*PLUS III

by Eric Lescasse (Compuserve: 70731,3233) Part II of II

Introduction

The goal of this article is to show how to approach building Windows applications with APL. To do this, we will use APL*PLUS III for Windows, version 1.1, the new product from Manugistics, Inc. You will see that programming Windows applications, a task reputed to be more difficult overall than DOS application programming, is in fact

- within the reach of all APL developers, even the most occasional ones
- much more simple than developing in APL under DOS.

The resulting applications involve much less APL code, and are much more reliable and easier to maintain. Moreover, you still continue to have 100% of the traditional characteristics of APL at your disposal, in particular its unequalled interactive development environment, its great power for modelling and data processing, and its unsurpassed speed for processing large amounts of data.

Saving Windows

By querying the *def* property of an object, we can obtain a snapshot of the object in the form of an APL variable.

For example:

```
Time_def←'Time' []wi'def'
[]pw←60
Time_def
710430785 65539 19940305 162231800 OTime 8589934592 CForm 7
68 Pcaption Time C_Window 512 Pwhere 7.25 33.625 13 4
0 OTime 0 CButton 1024 Pcaption Time C_Window 512 Pwh
ere 4.875 14.5 1.5 10 HonClick Time OlTime 0 CLabel
2560 Pcaption 1994 3 5 16 22 13 400 C_Window 512 Pbord
er 1 Pwhere 1 1 1.2 20

ρTime_def
38
[]pw←80
```

Our form *Time* is thus saved in an APL variable named *Time_def*, which is a nested array of 38 elements. This variable can be:

- saved with the workspace
- stored in an APL*PLUS component file
- copied from one workspace to another, etc...

If we destroy our form *Time* by applying the method *Delete*:

```
'Time' ⍵wi 'Delete'
```

we can recreate it later with the following simple instruction:

```
'Time' ⍵wi 'New' Time_def
```

Even better, you can save only a part of a Windows form in this way, for example, a frame and all the radio buttons it contains, and re-install all of these sub-objects in another form, with one simple instruction like this:

```
'New_form.New_frame1' ⍵wi 'New' Frame_def
```

Let's Make Our Application More Lively!

We are going to modify our Windows application so it displays the time continuously in the label *lTime*. To do this, we need to use a new kind of object called a *Timer*, which is independent of our form:

```
'tTime' ⍵wi 'New' 'Timer' ('interval' 1000) ('onTimer'
'time')
```

The above instruction creates a Windows timer, gives it an interval property equal to 1000 milliseconds (one second) and indicates that the "callback" function to be executed at each expiration of the Timer is our function *Time*, which is already written.

Press **Alt-Tab** to make our window appear. We can see that the exact time now displays continuously in our window. Note that the button *Time* remains active and continues to operate correctly. To ascertain the execution speed of APL*PLUS III, let's modify the interval property of our Timer to give it a value of 10 ms:

```
'tTime' ⍵wi 'interval' 10
```

Press **Alt-Tab** to make the form reappear. We can see that the time is updated every hundredth of a second! Notice that, without closing the Time window, you can click in the APL session and continue to work without any slowdown!

The Window Editor]WED

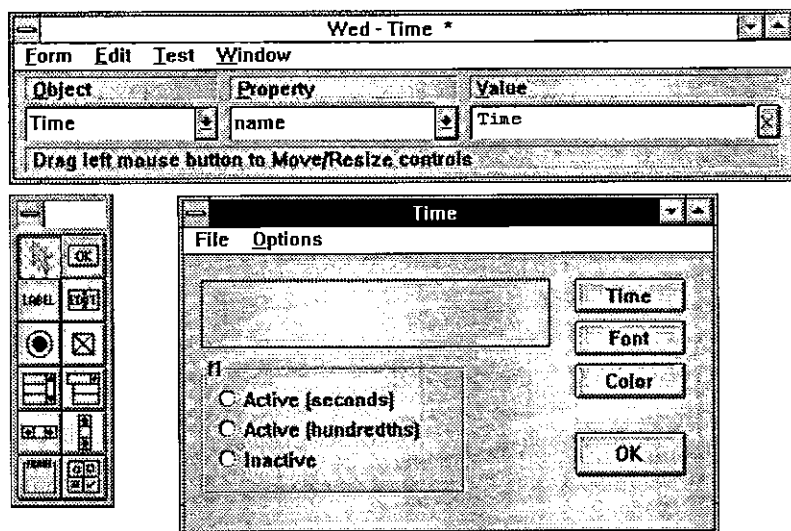
We will now make the following modifications to our window to complete it:

- move the Time button toward the top right part of the window
- Add a button Font
- Add a button Color
- Add a button OK
- Add a frame Timer, containing
 - a radio button Activate (seconds)
 - a radio button Activate (hundredths)
 - a radio button Inactive
- Add a menu with the selections File and Options.

The]WED editor is written in the form of a User Command so it is available in every APL workspace, even in a clear workspace. It is an interactive resource editor, that lets you create or modify windows, dialog boxes, or menus, letting you work mainly with the mouse.

`]Wed Time`

After using the editor to make our modifications, the screen looks like:



The]WED editor contains three windows:

- The top window, called *Wed*

This lets you choose the object (**Object**) that you want to work with (left combo box), the property (**Property**) that you want to modify or the event that you want to handle (middle combo box), and the value (**Value**) that you want to give it (input field on the right).

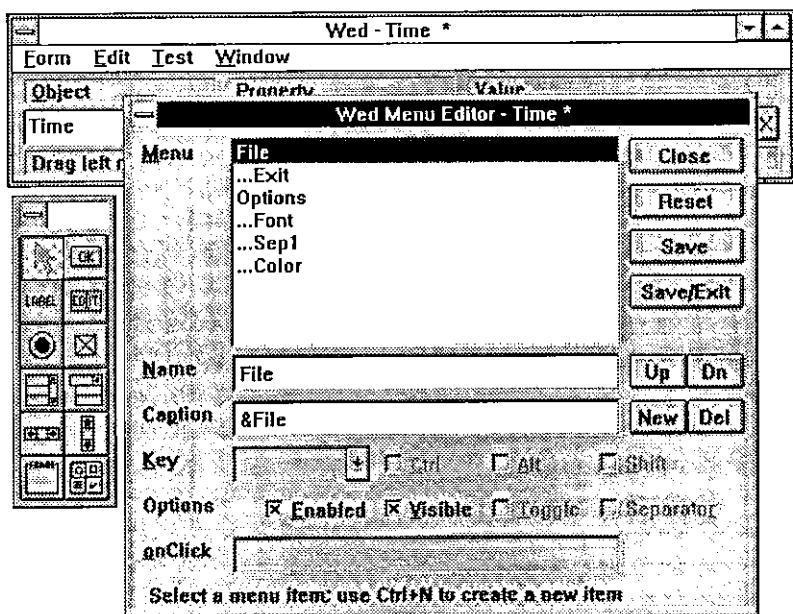
- The window at left (with no title)

This contains the tool palette, in which each icon represents a class of object that you can create in your window. By clicking on one of these buttons, then "dragging" with the right mouse button to set the size of the object in the form.

- The form being edited (in this case, the form *Time*)

After exiting from the]WED editor, you will find that you have a variable *Time_def* containing a description of our form.

The menus in our form have been created with the aid of the menu editor that you can invoke from within the *Wed* form by choosing the **Menu** option from the **Edit** menu or typing the "shortcut" key **Ctrl-M**.



From the]WED editor you can also directly define the "callback" functions associated with events that you wish to handle in your form, and also test the functionality of your form.

After exiting from]WED, we re-create our form with the instruction:

```
'Time' []wi 'New' Time_def
```

Having defined the menus for our form *Time* with the menu editor, let's see how we could have defined them with the system function []wi.

In the instructions that follow note that the periods (.) indicate the hierarchy of the menus and that the ampersand (&) indicates the character that will be underlined in the menus.

```
'Time.File' []wi 'New' 'Menu' ('caption' '&File')
'Time.File.Exit' []wi 'New' 'Menu' ('caption' 'E&xit')
('onClick' 'Exit')
'Time.Options' []wi 'New' 'Menu' ('caption' '&Options')
'Time.Options.Font' []wi 'New' 'Menu' ('caption' '&Font')
('Shortcut' 'F' 2)('onClick' 'Font')
'Time.Options.Sep1' []wi 'New' 'Menu' ('separator' 1)
'Time.Options.Color' []wi 'New' 'Color' ('caption' '&Color')
('shortcut' 'C' 2)('onClick' 'Color')
```

Now we need to write our "callback" functions *Font* and *Color*, as well as handle the button OK so it ends our application and closes our form.

This last task is very simple. It suffices to assign the value 2 to the *style* property of the OK button, with the following instruction:

```
'Time.OK' []wi 'style' 2
```

Click on the OK button and note that the window is closed. To reactivate the *Time* form, do this:

```
'Time' []wi 'Wait'
```

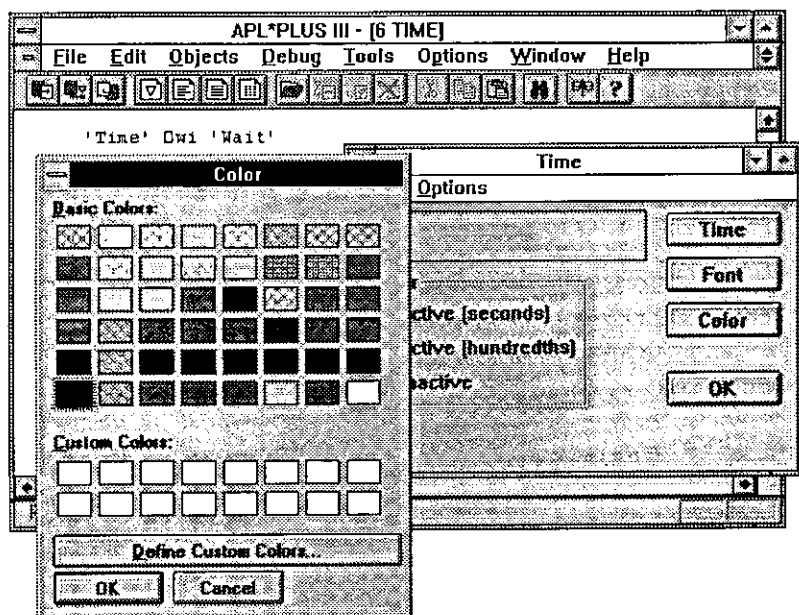
Now let's define the callback function *Color* like this:

```
▼ Color;A;[]wself
[1] []wself+ 'Time.lTime'
[2] :if ~0=A+WChooseColor []wi 'color'
[3] []wi 'color' (1>A)
[4] :end
▼
```

and copy the utility function *WChooseColor* and its subroutines into the workspace:

```
)copy c:\aplw\tools\windows WChooseColor WhwndOwner
SAVED 02/18/1994 08:40:15
)copy c:\aplw\tools\winapi D2C I2C W_Mem
SAVED 02/17/1994 18:15:39
```

Now click on the Color button. You will obtain a screen similar to the following:



Now let's bring the Font button in our form to life. We'll define the "callback" function *Font*, which modifies the font property of the *lTime* label, as follows:

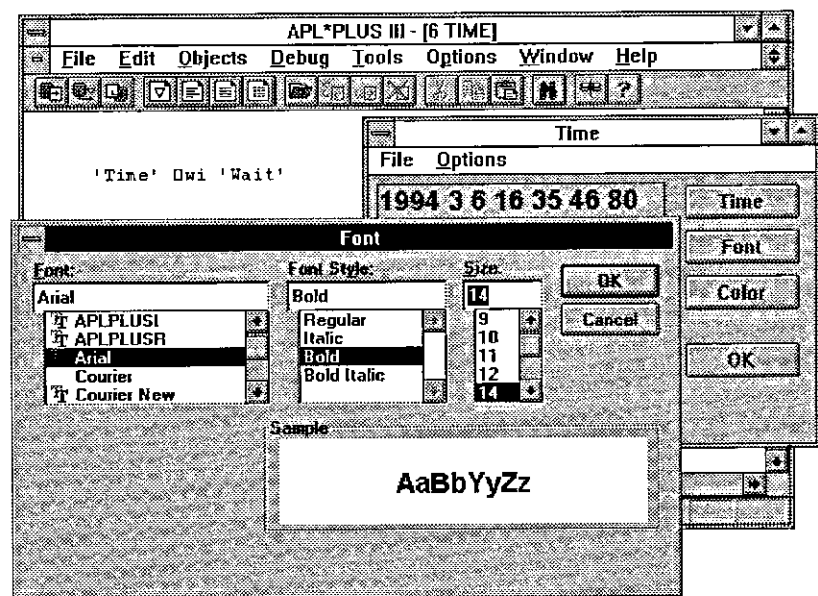
```
∇ Font;A;wself
[1] wself←'Time.lTime'
[2] :if ~0=A+WChooseFont wself'font'
[3] wself'font' (1>A)
[4] :end
∇
```

and copy the utility function *WChooseFont* and its subroutines into our workspace:

```
)copy c:\aplw\tools\windows WChooseFont
SAVED 02/18/1994 08:40:15
```

```
)copy c:\aplw\tools\winapi W2C C2I
SAVED 02/17/1994 18:15:39
```

Now, press the **Font** button in our **Time** form. A screen similar to the following will appear:



Activating the Menus

To make the menu options active, we set their *onClick* properties to execute the "callback" functions that are already created:

```
'Time.Options.Font' ⌕wi 'onClick' 'Font'
'Time.Options.Color' ⌕wi 'onClick' 'Color'
```

For the Exit option of the File menu to be able to close the form, it must apply the *Close* method to the form:

```
'Time.File.Exit' ⌕wi 'onClick' 'Close'
```

The "callback" function *Close* is defined as follows:

```

v Close;A
[1]  A←'Time' ⌕wi 'Close'
v

```

Handling the Radio Buttons

Using the J_{WED} editor, we have created a frame that we have called *fTimer*, and some radio buttons that we have named *Seconds*, *Hundredths*, and *Inactive*.

Let's associate with each of these buttons a "callback" function to handle a mouse click.

```

'Time.fTimer.Seconds' ⌕wi 'onClick' 'Timer'
'Time.fTimer.Hundredths' ⌕wi 'onClick' 'Timer'
'Time.fTimer.Inactive' ⌕wi 'onClick' 'Timer'

```

Note that we use the same "callback" function that, for simplicity, we have called *Timer*. This function is defined in the following fashion:

```

v Timer;w
[1]  'tTimer' ⌕wi 'Delete'
[2]  :select ⌕wi '.name'
[3]      :case 'Seconds'
[4]          w←'tTimer' ⌕wi 'New' 'Timer'
[5]          w ⌕wi 'interval' 1000
[6]          w ⌕wi 'onTimer' 'Time'
[7]          Time
[8]      :case 'Hundredths'
[9]          w←'tTimer' ⌕wi 'New' 'Timer'
[10]         w ⌕wi 'interval' 10
[11]         w ⌕wi 'onTimer' 'Time'
[12]  :endselect
v

```

You will notice that this function uses a "control structure" of the type *:SELECT* *:CASE* ... *:CASE* ... *:ENDSELECT* (the control structures, like the system functions and system variables of APL*PLUS III, can be written with either uppercase or lowercase letters).

This is an important innovation for APL that permits the language to be structured and brings it closer to traditional languages such as Visual Basic or C++. This also simplifies reading programs and facilitates maintaining APL applications.

The function *Timer* begins by cancelling the timer *tTimer* (line 1), then tests the name of the radio button that was clicked ('Seconds', 'Hundredths', or 'Inactive'). Notice that inside the callback function it is not necessary to specify the name of the object being processed in the left argument to $\square wi$.

The instruction *:select* tests which radio button has been clicked: if it is the button named **Seconds**, lines 4 through 7 are executed, and if it is button **Hundredths**, lines 9 through 11 are executed. The function *Timer* then creates a timer with an interval of 1 second if the user selected button 1, and an interval of 1/100 second if the user selected button 2.

Now we will modify our callback function *Time*, so it writes the data and time in a more agreeable form and so it does not show the milliseconds if the one-second timer is running.

Here is the new code for the function *Time*:

```

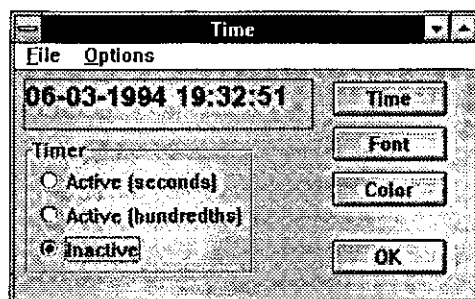
▽ Time;A;B;C;D
[1] A←'Time.fTimer.Seconds' □wi 'value'
[2] :if A
[3]     B←□ts[3 2 1 4 5 6]
[4]     C←'G<99-99-9999 99:99:99>'
[5]     D←100 100 10000 100 100 1001B
[6] :else
[7]     B←□ts[3 2 1 4 5 6 7]
[8]     C←'G<99-99-9999 99:99:99 999>'
[9]     D←100 100 10000 100 100 100 10001B
[10] :end
[11] 'Time.lTime' □wi 'caption' (,C □fmt D)
▽

```

Line 1 queries the value property of the first radio button (**Seconds**): the result will be 1 if the radio button is selected and 0 if it is not. Then we use the control structure *:IF ... :ELSE ... :END* to reduce the format of the time (eliminating the milliseconds) if the first radio button is selected.

On line 11 we use a "picture" type format instruction (which reminds one of those used by COBOL programmers) to format the date and time.

Our form now has the following appearance:



Low-Level Calls

APL*PLUS III allows you to:

- Call any other Windows application from your APL application
- Make calls to the hundreds of base functions in the Windows API
- Use any other commercially available DLL library and provide you with the services of the utilities it contains

The following section will show us how to do this.

Transforming Our Application into a Multimedia Application

By way of example, we are going to complete our callback function *Time*, so that our window beeps every fifteen seconds and plays chimes on the minute.

Calling a function in the base Windows API is done with the aid of the system function `▢wcall`. It is in general quite simple. You need to have the documentation of the Windows API at your disposal to know the list of available functions and their syntax.

Let's call the function Windows API function `MessageBeep` to emit a beep sound. Try the following instruction:

```
▢wcall 'MessageBeep' 0  
1
```

A beep sound is produced and the function `▢wcall` immediately returns the result of the function `MessageBeep`, which is 1.

Now we will show how we can as easily call an application written in another language from APL. To illustrate this point, we have written a very small Visual Basic application, that we have compiled and called MM.EXE.

This application uses the multimedia functions of Windows to generate a sound resembling chimes, a sound recorded in a .WAV file. It makes the sound TADA.WAV supplied with Windows.

To execute our Visual Basic application from APL:

```

      wcall 'WinExec' 'c:\vb\work\mm.exe' 'sw_show'
33

```

This instruction returns the instance number of the Visual Basic application. The sound TADA.WAV is then played on the sound card.

Now let's make the final changes to our function *Time* to transform our Windows application into a multimedia Windows application.

```

      v Time;A;B;C;D
[1]  A←'Time.fTimer.Timer[1]' wcall 'value'
[2]  :if A
[3]      B←ts[3 2 1 4 5 6]
[4]      C←'G<99-99-9999 99:99:99>'
[5]      D←100 100 10000 100 100 1001B
[6]  :else
[7]      B←ts[3 2 1 4 5 6 7]
[8]      C←'G<99-99-9999 99:99:99 999>'
[9]      D←100 100 10000 100 100 100 10001B
[10] :end
[11] 'Time.lTime' wcall 'caption' (,C ffmt D)
[12] :if A
[13]     :select B[6]
[14]         :case 0
[15]             A←wcall 'WinExec' 'c:\vb\work\mm.exe' 'sw_show'
[16]             :caselist 15 30 45
[17]             A←wcall 'MessageBeep' 0
[18]         :end
[19] :end
      v

```

To do this, we add lines 11 through 17. Note the nested control structures, and the possibility of using the *:caselist* option with the *:select* control structure.

B[6] represents the sixth element of our date/time vector, the seconds. In the case where it strikes the minute mark (*:case 0*), it calls our Visual Basic application to play the chimes, and in the case where the seconds are valued 15, 30, or 45 (*:caselist 15 30 45*), it only produces the beep sound.

Conclusion

Windows programming becomes something very simple with the object-oriented tools of APL*PLUS III. Very few of the languages or tools on the market let you develop under Windows as easily and as rapidly. Even Visual Basic, which we have used for several months, proves to be more laborious and slower overall than APL*PLUS III. C is undoubtedly faster in execution, but consumes much more time (and therefore costs) in development.

APL is simultaneously becoming more innovative and better structured. The examples in this complete application show clearly that it is now possible to develop APL applications using few APL primitives and yielding programs that are easily readable and maintainable.

What gives APL its remarkable power remains, as always:

- its remarkable interactivity
- its great powers in handling arrays of data.

Very few languages fit with Windows as well as APL. Those APL users who are already developing under Windows are enjoying these benefits now.

Listing of the APL Programs for the *WinTime* Application

We have named our little Windows application *WinTime*. The main function *WinTime* repeats the main instructions (used in calculator mode in this article) that create our application's windows.

Note that in real application development you would not need to write a function like this, with the position and properties of every object coded by hand. Instead you would design the form with the visual editor *Jwed* and interactively develop and test the code that works with it. Then, you would generate this main function automatically using the utility command *Jwmake* that is part of the built-in utility library of APL*PLUS III.

Following the main function are all the functions in the *WinTime* application. Note also that all the lines in certain functions have been commented to make them easier to read. Other programs have been abundantly explained in the article.

An excellent practice that we recommend to everyone is systematically to include a comment on each line of code and to align the comment symbols in column 40 (that is, in the middle of the screen). This facilitates maintenance of the code and, moreover, forces one to write short lines of code.

```

▽ WinTime:w
[1]  av Sample Windows Application written in APL+PLUS III v1.1
[2]  av Eric Lescasse 18-02-1994
[3]
[4]      'Time' □wi 'Delete'                a deletion of the form
[5]
[6]  w+ 'Time' □wi 'New' 'Form'            a creation of the form
[7]      w □wi 'onDestroy' 'KillTimer'
[8]  w+ 'Time.Time' □wi 'New' 'Button'
[9]      w □wi 'where' .5 28 1.5 10
[10]     w □wi 'onClick' 'Time'
[11]  w+ 'Time.Font' □wi 'New' 'Button'
[12]     w □wi 'where' 2.5 28 1.5 10
[13]     w □wi 'onClick' 'Font'
[14]  w+ 'Time.Color' □wi 'New' 'Button'
[15]     w □wi 'where' 4.5 28 1.5 10
[16]     w □wi 'onClick' 'Color'
[17]  w+ 'Time.OK' □wi 'New' 'Button'
[18]     w □wi 'where' 7.5 28 1.5 10
[19]     w □wi 'style' 2
[20]  w+ 'Time.lTime' □wi 'New' 'Label'
[21]     w □wi 'caption' ''
[22]     w □wi 'border' 1
[23]     w □wi 'where' .5 1 2 25
[24]  w+ 'Time.fTimer' □wi 'New' 'Frame'
[25]     w □wi 'where' 3 1 6 20
[26]  w+ 'Time.fTimer.Seconds' □wi 'New' 'Option'
[27]     w □wi 'caption' 'Active (seconds)'
[28]     w □wi 'onClick' 'Timer'
[29]     w □wi 'where' 1.25 1.25 1.27 17.5
[30]  w+ 'Time.fTimer.Hundredths' □wi 'New' 'Option'
[31]     w □wi 'caption' 'Active (hundredths)'
[32]     w □wi 'onClick' 'Timer'
[33]     w □wi 'where' 2.75 1.25 1.27 17.5
[34]  w+ 'Time.fTimer.Inactive' □wi 'New' 'Option'
[35]     w □wi 'caption' 'Inactive'
[36]     w □wi 'onClick' 'Timer'
[37]     w □wi 'where' 4.25 1.25 1.27 17.5
[38]
[39]  w+ 'Time.File' □wi 'New' 'Menu'        a menu definitions
[40]     w □wi 'caption' '&File'
[41]  w+ 'Time.File.Exit' □wi 'New' 'Menu'
[42]     w □wi 'caption' '&Exit'
[43]     w □wi 'onClick' 'Close'
[44]  w+ 'Time.Options' □wi 'New' 'Menu'
[45]     w □wi 'caption' '&Options'
[46]  w+ 'Time.Options.Font' □wi 'New' 'Menu'
[47]     w □wi 'caption' '&Font'
[48]     w □wi 'shortcut' 'F' 2
[49]     w □wi 'onClick' 'Font'
[50]  w+ 'Time.Options.Sep1' □wi 'New' 'Menu'
[51]     w □wi 'separator' 1
[52]  w+ 'Time.Options.Color' □wi 'New' 'Menu'
[53]     w □wi 'caption' '&Color'
[54]     w □wi 'shortcut' 'C' 2
[55]     w □wi 'onClick' 'color'
[56]  'Time' □wi 'Wait'                    a call the window

```

▽

```

v Time;A;B;C;D
[1] A←'Time.lTimer.Seconds' [wi 'value'
[2] :if A
[3]   B←[ts[3 2 1 4 5 6]
[4]   C←'G<99-99-9999 99:99:99>'
[5]   D←100 100 10000 100 100 100 1B
[6] :else
[7]   B←[ts[3 2 1 4 5 6 7]
[8]   C←'G<99-99-9999 99:99:99 999>'
[9]   D←100 100 10000 100 100 100 1000 1B
[10] :end
[11] 'Time.lTime' [wi 'caption' (,C [fmt D)
[12] :if A
[13]   :select B[6]
[14]     :case 0
[15]       A←[wcall 'WinExec' 'c:\vb\work\mm.exe' 'sw_show'
[16]       :caselist 15 30 45
[17]       A←[wcall 'MessageBeep' 0
[18]     :end
[19] :end
v

```

```

v Font;A;[wself
[1] [wself←'Time.lTime' a be on the object lTime
[2] :if ~0=A←WChooseFont [wi'font' a call the font dialog box
[3]   [wi 'font' (1>A) a apply font to lTime
[4] :end
v

```

```

v Color;A;[wself
[1] [wself←'Time.lTime' a move to form
[2] :if ~0=A←WChooseColor [wi'color' a color choice dialog box
[3]   [wi 'color' (1>A) a apply to label lTime
[3] :end
v

```

```

v Timer;w
[1] 'tTimer' [wi 'Delete' a clear existing timer
[2] :select [wi ',name' a which radio button?
[3]   :case 'Seconds' a if it's the first...
[4]     w←'tTimer' [wi 'New' 'Timer' a we create a timer ...
[5]     w [wi 'interval' 1000 a with a 1sec interval
[6]     w [wi 'onTimer' 'Time' a exec Time when fired
[7]     Time a and update the time
[8]   :case 'Hundredths' a if it's the sec button
[9]     w←'tTimer' [wi 'New' 'Timer' a we create a timer
[10]    w [wi 'interval' 10 a with a 1/100s interval
[11]    w [wi 'onTimer' 'Time' a run Time each 1/100s
[12] :endselect
v

```

```

v Close;A
[1] A←'Time' [wi 'Close' a close the form
v

```

```

v KillTimer
[1] 'tTimer' [wi 'Delete' a 'kill' the timer
v

```

Chaotic Behaviour Revisited

by Gérard A. Langlet

What is Chaotic Behaviour?

Chaotic behaviour occurs when one is unable to predict what will happen at some distance from here or from now. It has long been known (Poincaré, Lorenz with the butterfly-effect) that a small variation in the initial conditions may lead to huge differences in the behaviour of dynamical systems, which, in general, are described by differential equations as a function of time.

Hundreds of books and papers are devoted to chaos which is supposed to appear even when one iterates very simple nonlinear equations such as the "logistic equation", proposed by Verhulst more than 130 years ago in order to model and explain population ratio (alternate growth and decay) in ecological systems.

If X is a variable which may vary in the interval $(0,1)$, then Verhulst's **nonlinear** formula:

$$X_{n+1} = 4 X_n (1-X_n)$$

will give the next population ratio at step $n+1$ if one knows the ratio at step n .

Although this formula is completely deterministic (ALL equations are deterministic), the succession of $X_i, X_{i+1} \dots X_{i+p}$ (terms of a series) exhibits a "positive Lyapunov exponent", **proof** of its chaotic behaviour.

When the behaviour is not chaotic, this exponent is simply 0.

It is clear that iterations of linear functions may NEVER lead one to observe any chaotic behaviour.

Hic jacet Chaos (a non-syllogistic mathematical proof)

So, let us consider the iterations of one of the most simple linear formulas one can imagine:

$$\omega_{n+1} = 2\omega_n \quad \text{with } \omega \text{ an angle (e.g. expressed in radians).}$$

By no means would the iterations of such a formula lead to chaos.

As an example, if ω_0 is 1 radian, one may immediately predict that ω_N shall exactly equal 2^N radians.

Then, in order to obtain a variation in the closed interval $\{0,1\}$, let us choose variable X as $\sin^2\omega$ and replace the series involving powers of 2 by a series in X .

Any value of X is still predictable, as the squared sine of the corresponding ω .

For any value of $X_n = \sin^2\omega_n$, the next value will be $X_{n+1} = \sin^2\omega_{n+1}$ i.e. $\sin^2 2\omega_n$.

For any value of ω , then $\sin^2 2\omega_n$ may be written as a function of ω_n simply as: $(2 \sin\omega_n \cos\omega_n)^2$ which is equivalent to: $4 \sin^2\omega_n \cos^2\omega_n$.

Knowing that $\sin^2\omega_n$ is X_n , and that $\cos^2\omega_n$ is $1-X_n$, any undergraduate student finds the "logistic" formula:

$$X_{n+1} = 4 X_n (1-X_n)$$

which, consequently, MAY NOT be chaotic anymore.

Quod erat demonstrandum.

The True Origin of Chaos for Iterated Applications

Successive iterations of the logistic equation are ALWAYS computed ... with computers.

In all computers, precision is limited. (On paper, with a slide rule or a calculator, precision is also limited.)

If any initial value of ω is coded with B bits, every new iteration would require ONE new bit on the left of the internal representation of the old ω . Doubling ω_n simply appends a new 0 to the right of the previous representation of ω_n .

So, the internal representation of ω_{n+1} is the same as the one of ω_n with a 1-bit left shift; one can also say that a 0 on the left of ω_n is transferred to the right with a 1-position circular shift, in order to produce ω_{n+1} .

Then, in order to reach the N^{th} iteration with NO loss in precision, a record with $B+N$ bits is necessary. With double-precision floating-point arithmetics, e.g. with the usual IEEE standard, only 64 bits are available to code:

- a) the sign of the constant (1 bit),
- b) the exponent (11 bits, hence the maximum value 10^{308} because 308 is 2^{10} (i.e. 1024) divided by $\log_2 10$, then floored – rounded to the inferior integer –, knowing that one bit is also reserved for the sign of the exponent),
- c) the *mantissa* in 52 bits.

So, it is possible to predict that if the initial value of ω is 1 (radian), then coded in 1 bit, the behaviour will become completely chaotic after the 51st iteration, although the function may no longer be chaotic according to the preceding mathematical proof.

The situation becomes worse when one tries to compute the *Lyapunov exponent* of any series (which may be either experimental, or obtained by computer iteration). There is absolutely NO control of the result; even if the Lyapunov formula is good, the fact that a long series of constants is integrated or averaged in a computer is able to produce a truncated then wrong result, because the Lyapunov exponent is obtained as an average of differences, then is itself hypersensitive to floating-point arithmetic truncations.

One may consider that “proofs of chaotic behaviour”, based on iterated formulas or numeric integration should not be accepted as scientific results, unless the authors can prove that their calculations have been performed at least with $B+N$ bits for the mantissa of the floating-point representation, given B as the number of necessary bits for the “accurate” binary encoding of the initial value(s), and N as the number of performed iterations: all results, taken for granted, which do not respect this condition, would have to be computed again; then, perhaps, some rapidly-drawn conclusions, namely about the behaviour of physical systems, after 1,000 iterations (and, sometimes, after more than 1,000,000) would have to be discussed again, if not completely revisited.

Exercises for Tests in APL

A) Write an *ECHO* function which will iterate, with the maximum available precision, e.g. $\square PP+17$ in APL*PLUS, the function $X_{n+1} \leftarrow X_n$ near the limit of significance of the last digits of a large integer: you enter a large integer from the keyboard; the program shall display what it has understood; then, you enter exactly the same number as the one displayed by the computer, and "wait" for the answer of the computer. Iterate the dialogue until the computer echoes the same number as the one which was entered. Increment the last digit by 1 and do the same thing again and again: you will detect alternate zones of chaos (with strange attractors) and non-chaos (when the computer echoes exactly what you have typed). So you will have the proof that $X \leftarrow X$ is a chaotic function, with *intermittencies*, won't you?

B) Iterate $\omega_{n+1} \leftarrow 2 \times \omega_n$ starting from any value for ω_0 , e.g. 1 (radian). Form the rotation matrix:

$$\begin{pmatrix} \cos \omega & \sin \omega \\ -\sin \omega & \cos \omega \end{pmatrix}$$

- 0) Compute its determinant and display it together with the iteration number, at each iteration.
- 1) Take the initial rotation matrix M (e.g. for 1 radian). Iterate $M \leftarrow M + . \cdot M$ (this squares the matrix, then also doubles ω). Display the computed determinant together with the iteration number, at each iteration.

Knowing that all rotation matrices should have a determinant that IS 1 and nothing else, imagine first what you will obtain. Then, and only then, try (if possible with different implementations on various computers, or with languages other than APL, or on a programmable calculator, or with Excel, etc...).

Will you have proven that application "**1 becomes 1**" is chaotic?

☐ NO ☐ YES

C) Write a letter to Mr Feigenbaum (or to Mr Gleick) and expose your chaotic conclusions ...

Fractals RYI

by Clifford A. Reiter
reiterc@lafcol.lafayette.edu

Fractals are often visually attractive and thus can provide an interesting topic for discussions with mathematics students. They also provide a wonderful sense of satisfaction to students learning to program and provide a rich source of examples in computer graphics courses in addition to providing connections to the study of dynamics. Some of the most popular fractal images, Julia sets and the Mandelbrot set, see [10] or [12], are generated with escape time algorithms that are highly iterative and creating them requires a great deal of time. Great efforts have been made to create incredibly fast fractal image code. For example, FRACTINT is often hundreds of times faster than straightforward compiled Pascal or C code [17]. While APL is a wonderful language, implementations tend to be very slow at running such highly iterative experiments. Nevertheless, some fractals admit simpler constructions and APL can well be graceful at handling such fractals.

I have used APL for teaching from time to time. The uses range from assigning isolated experiments in a given course to teaching entire courses roughly following [16]. I also often find myself using APL for research because it pervades my thinking. Another teaching favourite is a course on computer graphics and mathematical visualization. It seems important for students to be skilled at visually representing information so I have taught such a course using Pascal a couple of times. These are exciting times for APL. Major innovations have been made providing excellent GUI interfaces and bringing graphics to the PC level. Control structures are being added to major vendors' products and J offers ASCII spelling and rich methods for defining pure functions. This has provided a fabulous opportunity for me: I was able to combine my enthusiasm for APL and visually representing information by teaching a course on mathematical visualization using J. Several topics in that course were described in a talk at the Toronto SIGAPL [9].

This note describes some of the visual things that I have had success doing in APL related visual research projects or that course. While any APL ought to facilitate the projects I will use as examples, I was hooked by J and will use J for most of my examples. Of course, an introduction to mathematical visualization could easily be book length, but here we will consider a few divergent examples to sample some of the depth and breadth.

J beginnings: Raster Fractals

At APL '91 [0] there were a few talks utilizing J, the new version of APL developed by K. Iverson [5,6]. While I avoided J talks to some extent, I did hear a couple of lectures on J and I remember some facts about J and my reactions: it was ASCII based (how sad to lose those APL glyphs), it allowed simple definition of pure functions (interesting but confusing) and inner products could use left-function arguments which were not reductions (that could come in handy someday). During January 1993 I taught an APL course using APL2 [1] that was a wonderful experience. We ended the course at the New York SIGAPL Tool of Thought VIII conference. At that conference Iverson spoke about J and Langlet [8] described various Boolean array constructions in APL2. A colleague asked if I could create a transparency showing Sierpinski's Triangle for a talk describing some of Langlet's ideas. The request was so simple that I sought an APL construction that was direct and could be generalized to give colours. After a bit of thought and experimenting I realized that if the indices of the rows and columns are given in binary in the columns of a matrix X, then the Sierpinski triangle can be produced with an "and" dot "nand" inner product:

```
⊖IO←0
```

```
⊖←X← 2 2 2 2 2 ⍒ 16
```

```
0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1
0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1
0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1
0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
```

```
(⊖X) ^.* X
```

```
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0
1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0
1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0
1 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0
1 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0
1 1 0 0 0 0 0 0 1 1 0 0 0 0 0 0
1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0
1 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0
1 1 0 0 1 1 0 0 0 0 0 0 0 0 0 0
1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0
1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

If one thinks of $(\ominus X) \wedge . \star X$ as giving the "and" reduction of pairwise "nand"s of the binary addresses of the rows and columns, then all sorts of generalizations

suggest themselves. One could take sums of pairwise “nands”. In fact, the ordinary matrix product $+. \times$ is a complement to those Boolean inner products giving a multivalued result that measures how far one is from being on the Sierpinski triangle (try it!).

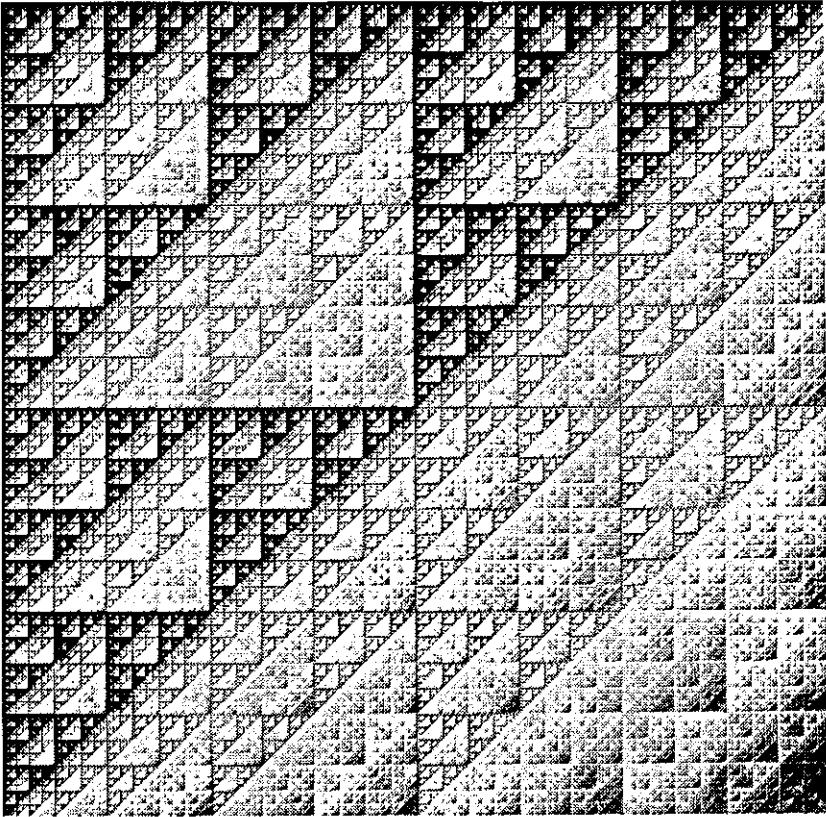


Figure 1: Generalised Sierpinski triangle from a matrix product

Figure 1 shows a grayscale image produced using the ordinary matrix product that way. Another generalization to colour (multivalues): find the position of the first 1 in the pairwise “nand” of the row and column addresses; notice this is sort of an inner product, but the left function is not a reduction, so this is not a classic inner product. Nonetheless, it is easy to implement this in modern APLs. APLers can easily imagine the rich possibilities for experimenting with different primitive functions and changing the addresses from base two to other bases.

After making several dozen images using those variations, I decided to try to describe these ideas in a journal that publishes fractals. While others have previously considered fractals constructed from bit operations, I thought the APL inner product point of view was worth sharing. I wanted to include the APL, but the audience wouldn't know APL, so I also wanted to keep the presentation dead simple. I wanted to use WordPerfect for the writing since the word processor that I use to do APL is ancient. Trying to get the APL symbols in WordPerfect was an aggravation (I know an APL font is available, but I have never managed to get it). I only wanted to use a dozen APL symbols but how was I going to make a transpose symbol? Well, I could have switched to the ancient word processor, it would have worked fine, but I happened to remember that one of my images was a reductionless inner product. J would solve the font nuisance and give me a uniform way of doing the reductionless inner products without going to nested arrays. This would simplify the presentation for a general audience.

I grabbed J electronically from the APL archive at Waterloo, and installed it under MS Windows. Readers interested in getting J may consult Appendix 0. I found J quite frustrating for a couple of days, but it was soon clear that J could do the matrix constructions mentioned above and the file I/O required to save large bitmaps for printing. The paper was finished about a week after I first obtained a copy of J and it has since appeared [13]. I pushed J a bit and noticed I could create individual arrays with millions of entries (requiring dozens of megabytes) on my PC. I was hooked — on my PC, I had an APL, that with 32M of memory plus virtual memory effectively gave me 100M workspaces. I ordered the reference materials [6,7] which, of course, made learning more J far easier.

Functions that allow for reading and writing Windows 256-colour bitmaps, which automated the process used for the fractal images mentioned above, are given in Appendix 1. There are also a couple of functions useful for creating 8-bit graphics images. APL for doing some of these tasks has been described [2,3].

Now a few comments about duplicating Figure 1. The J function #: creates binary representations and the addresses are put into rows.

```
      ]x=.:i.8
0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1
```

```

      x +/ . * |: x
0 0 0 0 0 0 0 0
0 1 0 1 0 1 0 1
0 0 1 1 0 0 1 1
0 1 1 2 0 1 1 2
0 0 0 0 1 1 1 1
0 1 0 1 1 2 1 2
0 0 1 1 1 1 2 2
0 1 1 2 1 2 2 3

```

The function `hue`, given in Appendix 1, takes a number between 0 and 1 and results in an RGB triple giving a pure hue. For example,

```

      hue 0                      NB. red
255 0 0
      hue 1r3                    NB. green
0 255 0
      p=.hue"0]2r3*(i.%<:)9
      P                          NB. 9 hues between red and blue
255 0 0
255 127 0
255 255 0
127 255 0
0 255 0
0 255 127
0 255 255
0 127 255
0 0 255

```

A 256 by 256 bitmap giving Figure 1 can be created and saved as a file. Graphics image formats vary, but a common format, often called an 8-bit graphics image, is essentially two arrays. One array is a palette which is an m by 3 array of numbers giving RGB triples (m is bounded by 256 for 8-bit graphics). The other array, the bitmap, is an array of numbers from $1..m$ where each entry is an index into the palette that gives the colour of the pixel corresponding to the entry. Below, `(p;b)` describes a boxed array giving the palette and bitmap.

```

x=.#:1.256                      NB. 256 row and column
b=.x +/ . * |: x                NB. the bitmap array
(p;b) writebmp8 'figure1.bmp'

```

The file `figure1.bmp` can now be viewed, edited or printed with applications that can handle Windows bitmaps.

Object Graphics: Koch Snowflake

In the visualization course mentioned in the introduction we considered both raster graphics (pixel arrays) and object based graphics. Several utilities were defined to let students create windows and show polygons and other objects on the screen without directly using very much of the J/Windows graphics interface. Some of these are given in Appendix 2; this includes `sogwin`, a function that starts an object based graphics window. For example, to open a window and plot a red triangle we can do the following. Note that the coordinates of the window run from 0 to 1000.

```

''[ sogwin 'sample'           NB. opens a window
]p=.200 300,500 820,:800 300  NB. a 3-gon
200 300
500 820
800 300
255 0 0 spoly p              NB. plots the 3-gon p in red

```

One of the students in the class asked about constructing the Koch Snowflake [12]. This is a triangle with $1r3$ scale bulges on each edge, each edge then has a bulge on it and so on. The basic step in the iterative process is take each edge and replace it with four edges introducing a bulge. If the given edge is trisected, then the two end thirds are used and the centre third is replaced by the other two edges of an equilateral triangle placed on the centre third. So the edge from 0 0 to 1 0 is given by two vertices:

```

0 0
1 0

```

and this should be replaced by the vertex sequence (polygon):

```

0 0
1r3 0
1r2 y
2r3 0
1 0

```

where y is $\%12\wedge 0.5$. In order to implement this subdivision it is convenient to define functions that find the midpoint and the points dividing the segment in thirds.

```

tri1=.2r3&*@ [ + 1r3&*@]  NB. one third point
tri2=.1r3&*@ [ + 2r3&*@]  NB. two thirds point
mid=. -:@+                  NB. midpoint
nor=.1 _1&*@|.              NB. right normal vector
0 0 tri1 1 0

```

```

0.333333 0
  0 0 tri2 1 0
0.666667 0
  0 0 mid 1 0
0.5 0
  nor 1 2
2 _1
  bulge=. mid + (:%12)&*@nor@:- NB. bulge point
  0 0 bulge 1 0
0.5 0.288675

```

So notice that bulge is just the midpoint with a displacement in the opposite of the right normal direction added. Below, segdiv divides up the given segment into pieces. Notice the last point is not included in the result since it will be included as the first point in the next segment to be divided. The function step applies the division to neighbouring pairs of vertices.

```

segdiv=. [ , tri1 , bulge ,: tri2
0 0 segdiv 1 0
  0 0
0.333333 0
  0.5 0.288675
0.666667 0

step=.,/@[ segdiv"1 (1&|.))

p NB. a 3-gon
200 300
500 820
800 300

<.step p NB. a 12-gon (floored to integers)
200 300
300 473
199 646
400 646
500 820
600 646
800 646
700 473
800 300
600 300
500 126
400 300

#step^:4 p NB. iterate subdivision 4 times
768
  0 0 0 spoly step^:4 p NB. plot the 768-gon

```

That image is shown in Figure 2. Can you decide what the following would look like?

```
255 0 0 spoly step^:4 p
255 255 0 spoly step^:3 p
0 255 0 spoly step^:2 p
0 255 255 spoly step^:1 p
0 0 255 spoly step^:0 p
```

Try it! What would happen if the order of the statements was reversed? How about if the bulges were in instead of out? Inward bulges on a 5 pointed star create a nice variation. Can you make that?

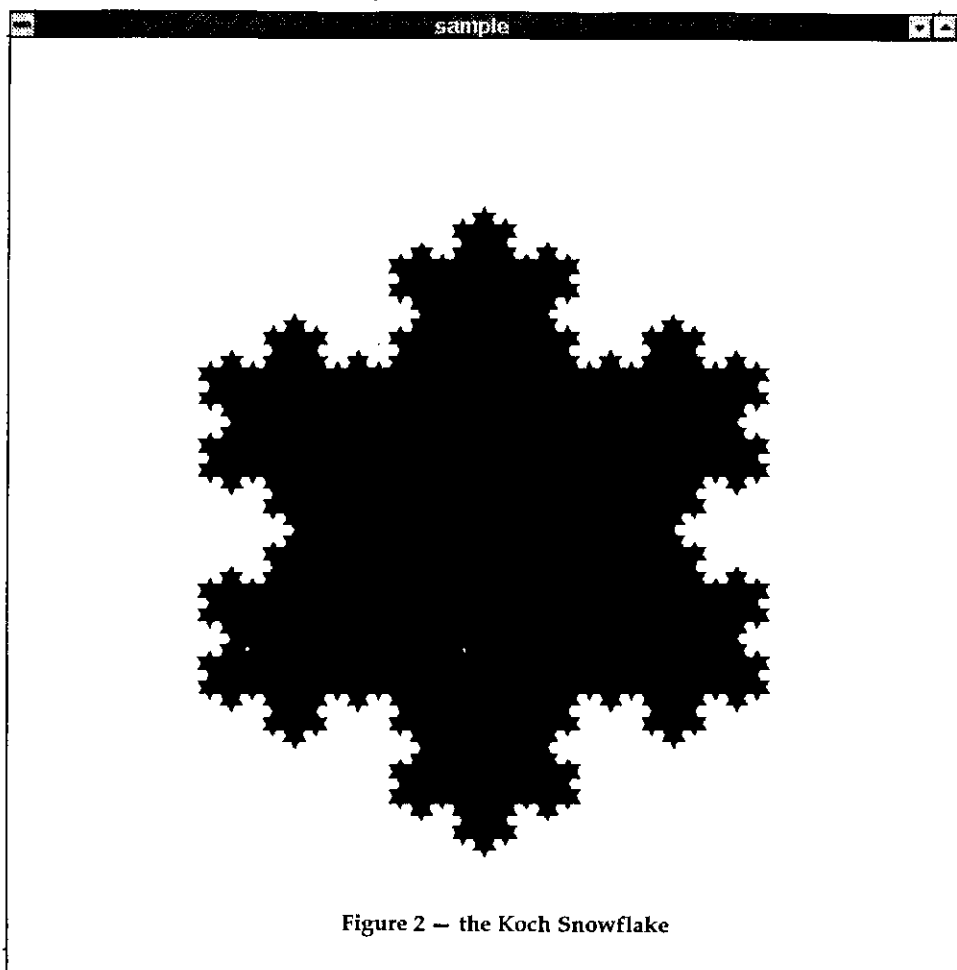


Figure 2 — the Koch Snowflake

Image Processing

One of the class exercises that was a lot of fun was to do some elementary image processing. The students were asked a sequence of questions and worked toward some primitive edge detection. The first step is to read in a digital image of a photograph. The example here was a colour image of two bears by a lake. The result of `readbmp8` is a two element boxed vector. It is convenient to immediately assign its elements names.

```

      ('p';'b')=.readbmp8 'bears.bmp'
      $p
256 3
      $b
480 640
      5{.15}.p      NB. some palette entries
167 138 105
122 118 117
138 129 109
207 161 112
222 186 127

```

Here `p` is a palette of 256 colours and `b` is a 480 by 640 array corresponding to the pixels. The first question was to find a way to turn this colour image into a grayscale image. The simplest answer is to replace each palette entry by a grayscale — the average of its red, green, and blue components is an obvious choice.

```

      avg=.+/% #
      avg 1 3 4
2.66667
      p1=.<.(3&#)@avg"1 p
      5{.15}. p1      NB. some averaged palette entries
136 136 136
119 119 119
125 125 125
160 160 160
178 178 178
      (p1;b) writebmp8 'temp1.bmp'

```

This image is indeed a gray scale version of the original. See Figure 3.

The next question is how to turn this into a black and white image. The simplest answer is to select a threshold for the gray scale. If the gray level is higher than half, consider the palette entry white and otherwise consider it black.



Figure 3 — Grayscale Bears

```

p2=.255*p>128
5{.15}.p2      NB. Some black & white palette entries
255 255 255
  0   0   0
  0   0   0
255 255 255
255 255 255
(p2;b) writebmp8 'temp2.bmp'

```

The result of temp2.bmp is shown in Figure 4. It is a black and white image, but it is "cheating" in that there are still 256 palette entries and the bitmap entries are from 1..256. We really want a black and white two colour palette and a Boolean bitmap. Below pbw is a black and white palette and b3 remaps the bitmap as a Boolean by indexing the entries from 1..256 into a Boolean array with appropriate entries. Thus, temp3.bmp is a true Boolean image, though its appearance is identical to that in Figure 4.



Figure 4 — Black and White Bears

```

]pbw=.0 0 0,:255 255 255
0 0 0
255 255 255
5{.15}.pbw i. p2 NB. test if palette entries are white
1 0 0 1 1
b3=.b{pbw i. p2
(pbw;b3) writebmp8 'temp3.bmp'

```

Lastly, we consider a primitive edge detector. A simple idea is to check whether neighbouring entries are the same. The fork `}.=}` drops the first item, the last item and then tests equality of the resulting arrays. Thus some edge detecting:

```

ed=.}.=}:
ed 0 0 0 1 1 1 1 1 0 0 NB. detect on a vector
1 1 0 1 1 1 1 1 0 1
(pbw;ed b3) writebmp8 'temp4.bmp'

```

The result is shown in Figure 5. Certainly the image is primitive, but `}.=}` is a short program. Moreover, students can quickly run through an exercise like this and get a feeling for algorithms for edge detection. In the class I taught, the students actually implemented (without being asked) better ideas for picking the threshold as well as creating row, column and 45° edge detection. They also suggested general ideas for better strategies for getting clearer definition of the edges of the bears as well as contrast enhancement.



Figure 5 – Primitive Edge Detection

Words in J

One of the ways in which APL can be useful is for organizing data for use by independent application packages. The construction of 3-dimensional objects directly in J was a topic in the visualization course, but it was also useful to use J to organize the description of 3-dimensional fractals for further processing by POV-Ray. POV-Ray is a ray-tracer that can be used to render “realistic” 3-dimensional images. I have used J this way on several projects including [11,14,15]. This note describes the J construction of a 3-dimensional Cayley graph from [11].

Consider all the words that can be written in the letters from the alphabet ‘aAbBcC’ with the understanding that upper and lower case letters are inverses and c commutes with everything. That is, aA, Aa, bB, Bb, cC, Cc are the same as the empty word and ac is ca and bc is cb. Thus, for example, abcBca is the same as aacc but abab is not aabb since a and b do not commute. It is convenient to write the words in the form wcn1 where w is a word in a, b and their inverses, some power of c (possibly negative) appears, and 1 is a letter from the alphabet. Constructing all such words of a given length is straightforward in J. The relationship between group elements (really classes of equivalent words) can be represented graphically by putting an edge between two words if you can get from one to another by multiplying by a single letter. Thus, if wcn1 is a word of the form described, we draw an edge between wcn and wcn1.

We turn for a bit to the construction of words of the given form. First, we will represent each word by a boxed character string. If we have two lists of words and want the concatenation of all possible pairs, this is an outer product:

`]L1=. '' ; 'a' ; 'b'`

a	b
---	---

`]L2=. '' ; 'A' ; 'B' ; 'AB'`

A	B	AB
---	---	----

`L1 ,&.>/ L2`

	A	B	AB
a	aA	aB	aAB
b	bA	bB	bAB

`adjw=. ,&.>/`

Here we have given the name `adjw` to the outer product. The next step is to create all words in given letters up to some specified length. This is just iterating `adjw` on a list of letters. This can be accomplished by taking the `adjw` reduction of the replication of the list of letters.

`]s=. '' ; ; 'aAbB'`

a	A	b	B
---	---	---	---

`2(#, :) s`

a	A	b	B
a	A	b	B

`mkallw=.adjw/@(#, :)`
`2 mkallw s`

	a	A	b	B
a	aa	aA	ab	aB
A	Aa	AA	Ab	AB
b	ba	bA	bb	bB
B	Ba	BA	Bb	BB

~.,2 mkallw s

a	A	b	B	aa	aA	ab	aB	Aa	AA	Ab	AB	ba	BA	bb	BB	Ba	BA	Bb	BB
---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Notice ~., removes duplicates from the ravelled list. Without commenting on each detail, we can construct all the words of the form wc^{n1} using some additional functions defined in Appendix 3. The steps required are: excluding words with letters next to their inverses, then padding with some powers of c or C, followed by adjoining a single letter at the end.

```
maxl=.4
abw=.( 'aA'; 'Aa'; 'bB'; 'Bb' ) exclw ~.,(maxl-1) mkallw ' ';/'aAbB'
abcw=.padc abw          NB. pad with c and C
normw=~.,abw adjw ;/'aAbBcC'
```

One of the important steps is to describe the edges. These are formatted as cylinders in three-space with given endpoints, radius and colour in a syntax that is used by POV-Ray. Here are some simple examples of the use of the function that formats vectors in a notation used by POV-Ray and an example of creating a cylinder description for the word abc. Here, the colour of the cylinder is determined by the last letter of the word.

```
fntvec 1 _2 3
<1,-2,3>
pos 'ab'
0.5 0.25 0
pos 'abc'
0.5 0.25 0.5
radius 'abc'
0.0140625
fntcyl 'abc'
object{cylinder{<0.5,0.25,0>,<0.5,0.25,0.5>,0.0140625}
pigment{rgb<0,0,1>}}
```

The result of graphing these cylinders is shown in Figure 6. Easy variations on this construction allow for finite order elements and different number of generators and commuting generators. Appendix 3 gives a complete script for generating the required POV-Ray file including light sources and position of the camera. Running:

```
povray -iffz.pov -offz.tga +ft -h200 -v200
```

would create a viewable targa file.

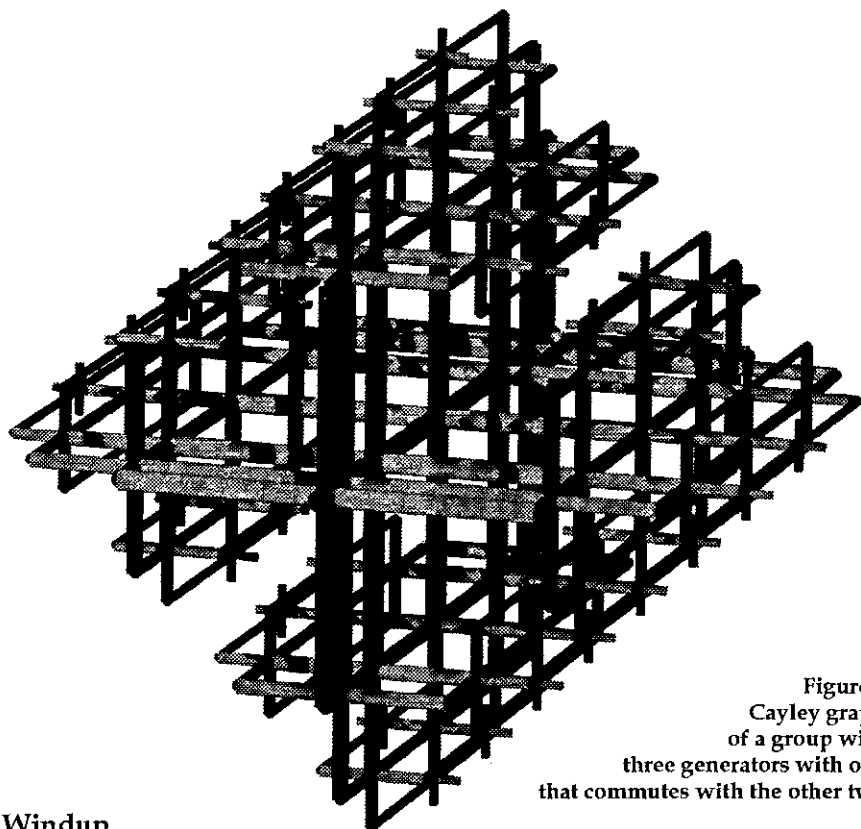


Figure 6
Cayley graph
of a group with
three generators with one
that commutes with the other two

Windup

We have seen that it is possible to use J to experiment effectively with ideas from mathematical visualization. Examples of bit-based fractals, object fractals, image processing, and 3-dimensional fractals were described here. Other topics from mathematical visualization and fractals, such as iterated function systems, transformations of the plane, projection from 3-space to 2-dimensions, estimating fractal dimension, contour plots, colour models, and escape time algorithms can all be readily implemented. Classic highly iterative escape time algorithms, like the Mandelbrot and Julia sets, run quite slowly, but students were even able to make a few 512x512 Mandelbrot images in J. It is wonderful that APL in general, and J in particular, has progressed to the point that mathematical visualization is simple and practical.

Clifford A. Reiter
Department of Mathematics, Lafayette College,
Easton, PA 18042 USA

References

- [0] *APL 91 Conference Proceedings*, APL Quote Quad, 21 4, (1991).
- [1] *Instructor uses APL*Plus Innovatively to Teach Math*, APL*PLUS News, C. Piersma, Editor, 5 1 (1993) 11.
- [2] J. Barman, *Dyalog APL Version 6.3, review*, Vector, 10 1 (1993) 59-69.
- [3] R. Cannon, *Windows BMP files and APL*, Vector, 10 1 (1993) 80-96.
- [4] D. Holt, *How to become an APL/J Sysop...and Why*, APL Quote Quad, 24 3 (1993) 11-12.
- [5] K. Iverson, *The Dictionary of J*, Vector, 7 2 (1990).
- [6] K. Iverson, *J Introduction & Dictionary*, Iverson Software Inc., (1993).
- [7] K. Iverson, *Programming in J*, Iverson Software Inc., (1993).
- [8] G. Langlet, *New Mathematics for the Computer*, APL as a Tool of Thought VIII, Conference Proceedings, E. Shaw, Editor, Y/SIGAPL, (1993) 1-17.
- [9] C. Linton, *From Pentagons and Plasma Clouds to Cliff Dwellers (Part 1) Gimme Arrays!* 1 6 (March 1994) 1,7-10.
- [10] B. Mandelbrot, *The Fractal Geometry of Nature*, W.H. Freeman and Co., (1982).
- [11] J. Meier and C. Reiter, *Fractal Representations of Cayley Graphs*, manuscript.
- [12] H-O. Peitgen, H. Jürgens, and D. Saupe, *Chaos and Fractals*, Springer-Verlag, (1992).
- [13] C. Reiter, *Fractals and Generalized Inner Products*, Chaos Solitons & Fractals, 3 6 (1993) 695-713.
- [14] C. Reiter, *Sierpinski Fractals and GCDs*, Computers & Graphics, to appear.
- [15] C. Reiter, *Synthetic Coloring of Fractal Terrains and Triangular Automata*, manuscript.
- [16] C. Reiter and W. Jones, *APL with a Mathematical Accent*, Brooks/Cole Publishing Co. (1990).
- [17] T. Wegner and M. Peterson, *Fractal Creations*, Waite Group Press (1991).
- [18] T. Wegner, *Image Lab*, Waite Group Press (1992).

Appendix 0: Getting J

See Product Guide for details. J is also available by ftp from archive.uwaterloo.ca in languages/apl/j and from Dick Holt's bulletin board [4] at 703-528-7617. The hardcopy documentation from Iverson Software is recommended.

Most of the functions mentioned in this note can be found in reiter.zip at Waterloo, on the bulletin board, or from the author. The functions in Appendix 1 are in bmp8.js and those in Appendix 2 are in gwin.js.

Appendix 1: Bitmap Utilities

These functions allow writing and reading uncompressed 8-bit Windows bitmaps. The function pwlin is a piecewise linear function that is used to construct the function hue that generates pure hues.

```
readbmp8=. 0 : 0
head=.256#.}. "1 a.i. 13 4$2).54{x=. 1!:1 <y.
pal=. |."(1) 3(. "1 (m,4)$ a.i. 54).(54+4*m=.11{head){.x
sbmp=.5 4{head xsbmp=.sbmp+(i.2)*4|-sbmp
bmp=. a.i.|.sbmp{xsbmp$(2{head)).x
pal;bmp
)

writebmp8=. 0 : 0
:
('spal';'sbmp')=.$@>"0 x.
h=.524289 0,(* /sbmp),0 0,2#spal=.0{spal
h=. (54+(4*spal)+* /sbmp),0,(54+4*spal),40,({.sbmp),h
head=. 'BM',,a.{~,. "1 (4#256)#:h pal=. ,(0,"1~|. "1 >{.x.){a.
xsbmp=.sbmp+(i.2)*4|-sbmp
bmp=. |.(xsbmp{.>{:x.){a.
(head,pal,bmp) 1!:2 <y.
)

pwlin=: 0 : 0
:
p=.>{:x.
i=.i.&0 p<y.
((1-r),r)=(y.--i{p)%~/(i-0 1){p)+/ .* (i- 0 1){c=.>{:x.
)
hue=:<.@((((i.7)%6);255*|. "1#;7|3^i.7)&pwlin)
```

Appendix 2: Some Windows Object Graphics Functions

These functions allow for opening a graphics window and plotting polygons of a given colour. The graphics window may be closed by exiting J or with wd 'reset;' and it may be cleared with wd 'gclear;'.

```
wd=.111:0          NB. Windows driver
sogwin=. 0 : 0      NB. Start Object Graphics Window
3 3 500 500 sogwin y.
:
z=. 'pc ',y.,';xywh ',(": x),';cc g isigraph;pas
',":2{x=.<.x.%2.5
wd z,';pcenter;pscale;pcloseok;pshow sw_showna;'
)

sfill=. 0 : 0      NB. Set Fill Color
wd ((_1 e.y.){('grgb ',(":y.),';gbrush'),:'gbrushnull'),';gshow;'
)

spoly=.0 : 0       NB. Show polygon
wd 'gpolygon ',(, ' ',":y.),';gshow;' : sfill x. spoly y. )
```

Appendix 3: Script for Creating Figure 6.

```
outfile=. 'ffz.pov'
'' write outfile          NB. Create empty output file
output=.append&outfile@, @(&eol"1)

NB. ***** Create Povray header *****
povheader=.0 : 4
camera{
  location <15.0,12.0,-50.0>
  direction <0.0 0.0 24.0>
  up <0.0,1.0,0.0>
  right <1,0,0>
  look_at <0.0, 0.0, 0.0>}
object{light_source{<150.0,125.0,-200.0> color rgb<1,1,1>}}
object{light_source{<-150.0,125.0,-200.0>color rgb<1,1,1>}}
object{light_source{<150.0,-125.0,-200.0>color rgb<1,1,1>}}
background {color rgb<1,1,1>}
)

NB. ***** end of povray header *****
output povheader
alf=. 'aAbBcC'          NB. alphabet of letters
fml=.,/(,;-)"1=1.3      NB. direction of full motion of letters
rml=. 'aAbB'            NB. reduced motion letters
r=.0.5                  NB. reduced length ratio
rr=.0.75                NB. reduced radius ratio
```

NB. pos gives the position in 3D of word with scaling

```
pos=. {&fml@(&alf &i.)+/@:*((&r@ -.)+(* r&^@(+/\)))@(&e.&rml)
standr=.1r40          NB. standard radius with no reduction
radius=.standr&*@(&rr&^@(+/\))@(&e.&rml)@: NB. radius of word
c=.|fml              NB. color table
color=.{&c@(&alf&i.)@: NB. select color by last letter
adjw=. ,&.>/         NB. adjoin words
mkallw=. adjw/@(&#,:) NB. make all words of given length
chsw=. ' ' : '(a.i.y.){({:x.) (a.i. {x.})a.' NB. character switch
fmtvec=.,&'>'@(' '<'&,)@((2 2$' _,-')&chsw)@": NB. vector format
```

NB. Format a cylinder for POV-Ray corresponding to word

```
fmcyl=.0 : 0
z=. 'object{cylinder{' , (fmtvec pos ):y.),' , (fmtvec pos y.)
z=.z , ' , (":radius y.),' pigment{rgb' , (fmtvec color y.),' }'
```

```
exclw=. *./@:({-@(+/"1)@ E.&>"0 _)#] NB. exclude words
maxl=.4          NB. maximum word length
wc=.}.@, @(<@#"0/&'cC')@i.          NB. small words in c and C
padc=.;@](<,&.>wc@(&maxl&-@#))&.> NB. pad with c and C
```

NB. ***** Construct the words and format for POV-Ray *****

```
abw=.('aA';'Aa';'bB';'Bb') exclw ~.,(maxl-1) mkallw ' ' ;/ 'aAbB'
abcw=.padc abw
#normw=.,abcw adjw ;/alf
#output fmcyl&> normw
```

One Font to Rule Them All

by Adrian Smith (Vector Production)

*'Three .DINs for the Elven-Kings under the sky
Seven for the Dwarf-Lords in their halls of stone ...*

*One font to rule them all, one font to find them
One font to bring them all, and in the darkness bind them.*

with apologies to JRR Tolkien

Introduction

Two recent items in Vector prompted me to write this. Gfeller and Kromberg [1] make the point that under Windows, it is meant to be easy to move stuff between applications. With Dyalog-7 (which now uses the clipboard sensibly) and APL*PLUS III, I should be able to move $\uparrow$ 2 from one session to the other via the clipboard, and have it execute just the same in either interpreter.

Then we have Jan Karman's letter [2] complaining at the near impossibility of getting an $\uparrow$ into Word for Windows using the Dyalog APL font.

Finally, I was goaded into action by the Finns, who got really upset that the APLPLUS font omitted most of the Scandinavian characters!

Objectives

1. To be able to cut and paste code between Dyalog-7, APL*PLUS III, WinWord, Write etc. If it arrives (by any route) into an APL interpreter, it should run.
2. To have one and only one TrueType APL font loaded in Windows for use by all the Windows applications on my machine.
3. To have (at most) a couple of bitmap fonts for use in APL development, which support exactly the same character set as the TrueType equivalents.
4. To leave as many as possible of the 'special' characters (such as the £ sign and all the Scandics) where Windows users expect to find them.
5. To avoid known bear-traps, such as the ch(160) location which Dyalog ill-advisedly occupied with the afore-mentioned $\uparrow$.

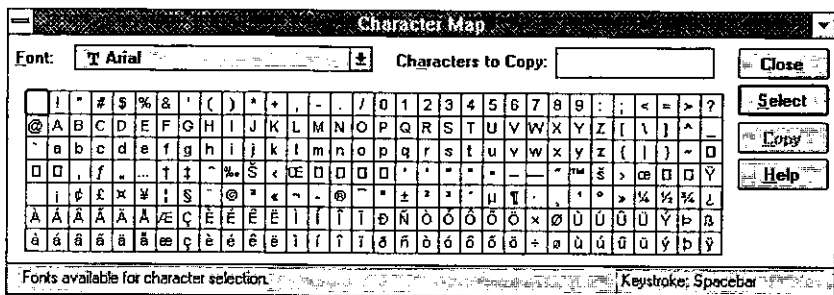
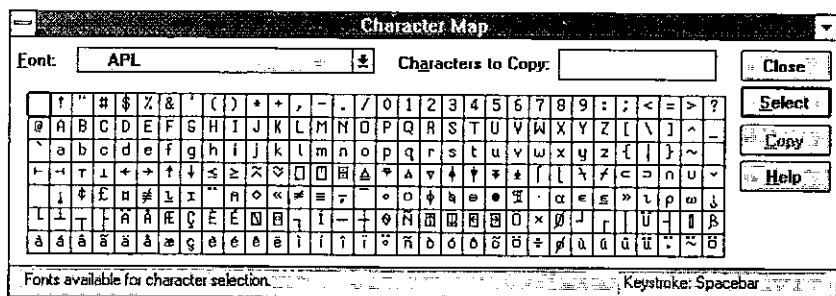
Compromises Required

1. Goodbye A — Z. There are simply not enough holes in the 256-character set to put these in.
2. Goodbye some of the accented capitals.
3. Hello APL*PLUS font encoding. Dyalog suffers for being too public-spirited! It is easy to change their output mapping, and impossible to change STSC's, so APL*PLUS it is. Actually, the Maryland mapping is very sensible, and makes it quite easy to remember things like 'abc' >> Δābc∇ if you want proper paired quotes in your text. The Copyright © symbol is an APL ⌈ which is nice.

However, I do insist on keeping the basic line-drawing set (I like the Dyalog *DISP* function) and all known APL symbols, including the oddballs which APL2 defined and never used.

What Does it all Look Like?

You might like to compare the CharMap prints of the bitmap font and standard Arial to see which characters I have stolen!



All the Scandics have been checked out by the Finnish APL group (thanks, Kimmo), so it remains to see which other outposts of Europe I will have left bereft of some crucial part of their alphabet.

The Complete APL2741 Font as Currently Proposed

	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
p	q	r	s	t	u	v	w	x	y	z	{		}	~	
⌈	⌊	⌋	⌌	⌍	⌎	⌏	⌐	⌑	⌒	⌓	⌔	⌕	⌖	⌗	⌘
⌙	⌚	⌛	⌜	⌝	⌞	⌟	⌠	⌡	⌢	⌣	⌤	⌥	⌦	⌧	⌨
	;	⌫	⌬	⌭	⌮	⌯	⌰	⌱	⌲	⌳	⌴	⌵	⌶	⌷	⌸
∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘
⌊	⌊	⌊	⌊	⌊	⌊	⌊	⌊	⌊	⌊	⌊	⌊	⌊	⌊	⌊	⌊
⊖	⊗	⊗	⊗	⊗	⊗	⊗	⊗	⊗	⊗	⊗	⊗	⊗	⊗	⊗	⊗
à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
ö	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	·	·	ö

This uses exactly the same character designs as DyalogAPL and the VectorAPL font supplied with the Vector APL typewriter. The original artwork is in CorelDRAW! and the final production was done with FontMonger. It is available in TrueType and Adobe Type-1 format. All variants may be freely distributed with no restrictions of any kind.

What Happens Next?

Someone needs to kick whatever remains of the APL Standards group to start the ball rolling towards a generic font mapping for APL under Windows. I can see scope for plenty of argument over the details, but at least if everyone accepts the principle, then the smaller fish will quickly fall into line, and interpreters like APLIWIN will also adopt enough of the standard mapping to be acceptable citizens. The APL*PLUS III font has to be the obvious starting point, and the APL2741 scheme is totally compatible with it.

Someone needs to find out (and publish) more about what is going on in the Unicode world [3]. When is this likely to hit us (I don't see any signs of a mass migration to Windows NT or OS/2) and is it just being driven by IBM?

I need to plan a move from Word-for-DOS technology (Vector is still produced with the old APL*PLUS/PC mappings) to a full Windows platform. I may well go to PageMaker (which supports styles properly) rather than WinWord (which remains less functional than Word 5.0 for DOS, even in the new resource-hogging version). Either way, I will need a bi-directional mapping from VectorAPL to APL2741 without loss of information.

Vector OnLine?

This looks very attractive, and would be very easy to do, but to be useful, it would need to be in a generic format that anyone could read, and extract code from (hence the requirement for a standard font across all interpreters). I favour Windows Write — suggestions welcome.

References

- [1] Gfeller and Kromberg, *Standardisation beyond the Language*, Vector Vol.10 No.4 pp58 - 63
- [2] Karman J, *Take it or Leave It*, Vector Vol.10 No.3 page 129
- [3] Brown J, *Extending the APL2 Character Set*, APL93 Conference Proceedings, page 41

Understanding Statistical Theory Through Simulation

by David R Appleton

The central limit theorem is of tremendous importance to statistical theory. Do students really understand it? How can we help them to? We present here a small revision project which makes students use simulation, most easily in APL, to investigate how addition of variates from the same distribution usually leads to a more nearly Normal distribution. The project is not suitable as an initial approach to the central limit theorem as it brings in too many other statistical ideas, though the simulation programs could be used by a lecturer to produce an informal graphical introduction to the theory. Many teachers probably already do just that; this is for the others. Theoretical results are summarised, not because they are unfamiliar but so that readers may the more quickly assess how much knowledge students must have to complete the project.

The Central Limit Theorem

The central limit theorem states that if $\xi_n = \frac{1}{n} \sum_{i=1}^n \zeta_i$ where ζ_i are identically and independently distributed with mean μ and variance σ^2 then

$$\lim_{n \rightarrow \infty} P\left\{t_1 \leq \frac{\bar{\xi}_n - \mu}{\sigma/\sqrt{n}} \leq t_2\right\} = \Phi(t_2) - \Phi(t_1)$$

where $\Phi(\cdot)$ is the cumulative distribution function of the standard Normal distribution.

The Project

- For each of the following five values of α , generate 10 random samples, each of size 1000, where observations are independently distributed as the sum of α standard exponential variates. ($\alpha = 1, 2, 5, 10$ and 20). Calculate the skewness and kurtosis of each sample, and plot them (separately) against a suitable power of α . Plot the theoretical values of the skewness and kurtosis for all $\alpha \geq 1$, remembering what distribution is formed by the sum of independent exponential variates with the same mean.
- Repeat the exercise using rectangular variates. Superimpose the results on the graphs for the exponential distribution, adding again the theoretical values for all α . (You may find it easiest to use the rectangular distribution on $[-1,1]$ for the theoretical part of this question, and to obtain the moments using Maple.)

- (c) Now generate 50 samples of 1000 and 50 samples of 100 variates which are the sum of 20 rectangular variates. Compare the standard deviations of your two sets of 50 estimates of skewness with the theoretical standard error of the estimate of skewness from a Normal sample. Do the same for kurtosis.
- (d) Repeat part (a) using Cauchy variates.
- (e) Discuss all of your results with reference to the central limit theorem.

APL Functions

These are all very easy – indeed trivial – though to execute them you need to be able to use quite large workspaces. Since we are measuring tendency to Normality by skewness and kurtosis we need functions to evaluate these. Let us assume that the replicate samples are contained in the rows of a matrix: one possible function is:

```

▽Z←SKKUR X;N
[1] X←X-⊖(⊖ρX)ρ(+/X)÷N←~1+ρX
[2] X←X+⊖(⊖ρX)ρ((+/X×X)÷N)*0.5
[3] Z←((+/X*3),[1.5](+/X*4))÷N
▽

```

We need to be able to generate rectangular, exponential and Cauchy variates:

```

▽Z←RECT N
[1] Z←(Nρ2147483647)÷2147483648
▽

▽Z←EXP N
[1] Z←-⊖RECT N
▽

▽Z←CAUCHY N
[1] Z←3∘∘~0.5+RECT N
▽

```

The last two of these programs use the inverse of the appropriate distribution functions, which we give here for completeness. For the exponential distribution with unit mean

$$F(x) = 1 - e^{-x}, \text{ and for the Cauchy distribution } F(x) = \frac{1}{2} + \frac{\tan^{-1}x}{\pi}.$$

Finally we need to be able to add the random variates to form the matrices for evaluation of skewness and kurtosis:

```

▽Z←REPSNalpha SUM distn
[1] Z←+/⊖distn,' ',▽REPSNalpha
▽

```

where the left-hand parameter contains the number of replicates (10 in most of the project), the sample size (1000 or 100 in the project) and the number of variates to be added (the 5 values of α). The right-hand parameter is the name of one of the functions which generate the random variates. The calculations for part (c) may therefore be performed by calling

```
SKKUR 50 100 20 SUM 'RECT'
SKKUR 50 1000 20 SUM 'RECT'
```

Theoretical results

The sum of α independent unit exponential variates is an Erlangian (or gamma) variate with distribution function $f(x) = a^{a-1}e^{-x} / \Gamma(\alpha)$ and characteristic function $(1-ix)^{-\alpha}$. Since the distribution is asymmetric, it is necessary for students to be able to calculate the central moments using the formulae $\mu_2 = \mu_2' - \mu_1'^2$; $\mu_3 = \mu_3' - 3\mu_1'\mu_2' + 2\mu_1'^3$; $\mu_4 = \mu_4' - 4\mu_1'\mu_3' + 6\mu_1'^2\mu_2' - 3\mu_1'^4$. The variance, skewness ($\mu_3/\mu_2^{3/2}$) and kurtosis (μ_4/μ_2^2) can be shown to be α , $2/\sqrt{\alpha}$, and $3(1+2/\alpha)$ respectively. All these formulae may conveniently be found in Kotz and Johnson.¹

The characteristic function of the rectangular distribution on the interval $[a-b, b+h]$ is $\frac{e^{i\theta b} \sin(b\theta)}{b\theta}$. As suggested in the project, therefore, it is easiest to work with $a=0$, $b=1$, so

that the characteristic function for the sum of α such rectangular variates is $\left(\frac{\sin \theta}{\theta}\right)^\alpha$.

Maple² gives the Taylor expansion of this function as $1 - \frac{1}{6}\alpha x^2 + \left(\frac{1}{120}\alpha + \frac{1}{72}\alpha(\alpha-1)\right)x^4 + \dots$

which can be manipulated into the form $1 + \frac{1}{3}\alpha \frac{(ix)^2}{2!} + \frac{1}{15}\alpha(5\alpha-2) \frac{(ix)^4}{4!} + \dots$. From this we

can see not only that the variance is $\frac{\alpha}{3}$, which is obviously true since the variance of the rectangular distribution on $[-1, 1]$ is $\frac{1}{3}$, but also that the kurtosis is $\frac{1}{15}\alpha(5\alpha-2)/\frac{\alpha^2}{9}$ or

$3(1 - \frac{2}{5\alpha})$. The skewness is of course zero. The Maple command which gives this result is

```
taylor( ((sin(x)/x)^alpha), x= 0, 6 ); .
```

though in fact working with $m = 1/\alpha$ makes the algebra easier.

For large samples of size n the variance of the skewness of a Normal distribution is approximately $6/n$, and that of the kurtosis $24/n$.

Simulation Results

Figure 1 shows graphs of a set of simulations for the rectangular and exponential distributions. The student has to choose what power of α to plot the skewness and kurtosis against. Since the rectangular distribution is symmetrical it does not matter what power is chosen for skewness, but for the exponential distribution $\alpha^{-1/2}$ is best as it shows the relationship as a straight line. For either distribution α^{-1} is best for kurtosis for the same reason.

Figure 2 gives the same information for the Cauchy distribution. Note the considerable change of scale.

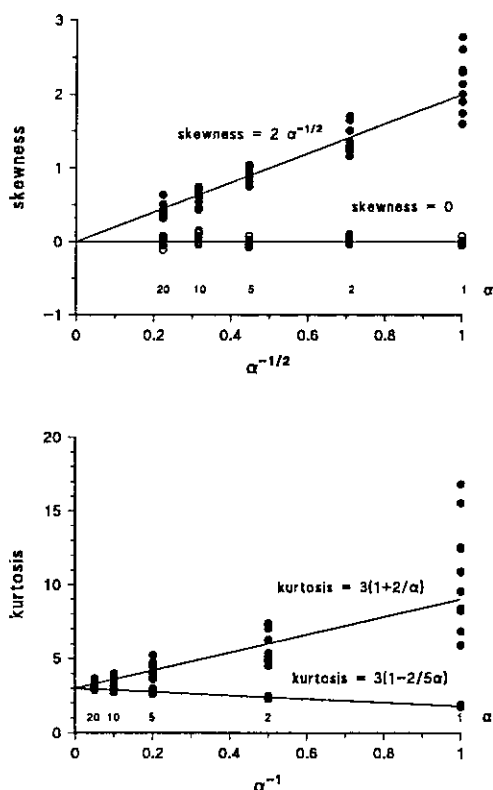


Figure 1. (a) skewness and (b) kurtosis of the sum of exponential (●) or rectangular (○) variates. Each point represents a sample of 1000 variates which are formed as the sum of α such variates, for $\alpha = 1, 2, 5, 10$ or 20.

The theoretical results are superimposed.

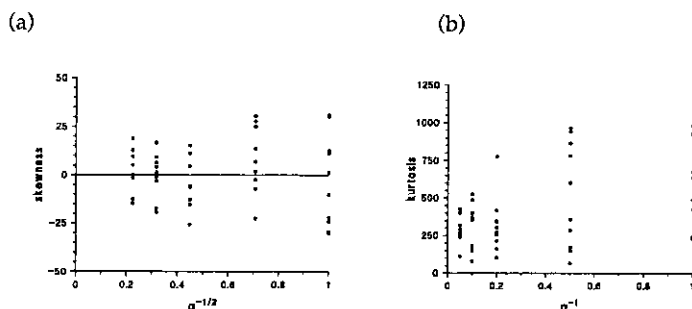


Figure 2. (a) skewness and (b) kurtosis of the sum of Cauchy variates. Each point represents a sample of 1000 variates which are formed as the sum of α Cauchy variates, for $\alpha = 1, 2, 5, 10$ or 20 .

What are students to make of such results? They ought to be struck by the closeness of the simulations to the theoretical results, and by the small amount of variability when rectangular variates are involved as compared to exponential variates. The greater variability when fewer variates are summed should also be clear. It should be obvious to them that the Cauchy results are totally different from those of the other two distributions. That the skewness tends to zero and the kurtosis to the value 3 as α gets large (though it does not need to be very large for the values to be close) should be clear evidence that the distributions are tending rapidly to Normality.

For the particular simulations carried out here for the sum of rectangular variates, when $\alpha = 20$, we have results as in Table I.

	Sample size					
	100			1000		
	Mean	Standard	Theoretical standard error	Mean	Standard	Theoretical standard error
Skewness	-0.032	0.230	0.245	0.001	0.073	0.077
Kurtosis	2.87	0.42	0.49	2.93	0.13	0.15

Table I. Values of skewness and kurtosis of samples of size 100 and 1000 of the sum of 20 rectangular variates; means and standard deviations are given for 50 samples of each size. Theoretical values are also given for the standard errors of estimates of skewness from samples of size 100 and 1000 from a Normal distribution.

Students should not only see that the simulations give values for skewness and kurtosis which agree well with theoretical values; they should also note that the standard deviations of their sample values for both skewness and kurtosis agree well with the theoretical values of the standard errors of the estimates.

The best students might look more closely at the individual results, rather than merely summarise them in this way. They might notice that for the smaller sample size the distribution of values of kurtosis is somewhat skew.

Discussion

The central limit theorem is the basis for much of the methodology of applied statistics. If it did not hold, the pre-eminence of methods based on the Normal distribution would not exist; for example the calculation of confidence intervals for sample means would not take place as it does, nor would the method of least squares be so important. It is necessary that students of statistics understand it, not merely as a statement but also in terms of its consequences.

What we have tried to do here is present a short project which puts the theorem in perspective. It also tries to tie together various other aspects of a statistics course. Most students will find the moments of the gamma distribution either by looking them up in a reference book or by remembering that they can integrate $\int_0^\infty x^\alpha e^{-x} dx$.

The slightly unusual choice of notation (using α instead of k perhaps) was meant to guide them gently to this integral. For the rectangular distribution they have been pointed towards the characteristic function, and unless they are very adept at handling series they will almost certainly require to demonstrate some ability with a computer-algebra package. To examine the sampling variance of skewness and kurtosis they will have to perform some straightforward descriptive statistics, and they must also show that they can produce simple computer graphics; here we have used the Fig.P package³. And of course they must be able to write the short programs necessary to generate the random variates from the appropriate distributions and calculate the skewness and kurtosis. No single part of the project is difficult, but it tests a variety of important statistical and mathematical skills.

The use of the sum of variates from an asymmetric distribution such as the exponential is quite important, as the approach to Normality is more impressive than from a symmetric distribution, but of course other distributions may be used. The fact that the distribution of the sum of variates from the logistic distribution approaches a value of 3 for kurtosis no faster than those from a rectangular distribution might be of some interest. Maple will show this quite easily:

```
readlib(coeftay1);
```

```
f:=(Pi*I*x/sin(Pi*I*x))^(1/m);  
c2:=-2*coeftayl(f,x=0,2);  
c4:=24*coeftayl(f,x=0,4);  
simplify(c4/c2^2);
```

Any project involving random numbers is by its nature unpredictable. The teacher cannot know exactly what data will be produced unless he specifies the programs for random number generation and insists on the seed to be used. The first of these actions takes away a good deal of the learning process the project is designed to engender, and the second will justly be viewed as extremely suspicious. It is therefore important that simulation projects should not rely on small experiments where strange outcomes are more likely to occur. However, even when large numbers are involved it is important to make sure that results are not over-interpreted; in the graphs related to the Cauchy distribution, for example, there is the suspicion that the kurtosis may be slightly less varied for larger values of α .

If a poor student suggests this, it may indicate that he is not fully aware of the ramifications of the Cauchy distribution, though a good student may wonder if it could be something to do with standardising by a variance which exists only for a sample. The results for the other distributions are more important than such points; the reason for introducing the Cauchy distribution into the project is to show that the conditions under which the central limit theorem can be proved really matter: the mean and standard deviation must exist. This is sometimes regarded by students as rather frivolous, but the vast difference in the results between the two 'proper' distributions and the Cauchy distribution should make the point clearly.

There are, of course, many places in statistical education where simulation can aid understanding, and APL is a most convenient language in which to perform such exercises. It is to be hoped that use of the language will continue to help students to understand more fully these essential concepts.

References

1. Kotz S & Johnson NL (1988). *Encyclopedia of Statistical Sciences*. Wiley, New York.
2. Waterloo Maple Software (1993). *Maple V*. University of Waterloo, Canada.
3. Fig.P Software Corporation (1992). *Fig.P (version 6.0)*. Biosoft, Cambridge, UK.

Advance Warning: Change to *ALL* UK Phone Numbers

For the benefit of overseas subscribers, please note that as from **16th April 1995** there will be a complete revision of the telephone dialling codes in Great Britain.

In most cases, simply add an extra '1' to the code, for example:

+44-439-788385
becomes
+44-1439-788385

The exceptions are:

Bristol: +44-272-730036 >> +44-117-9730036

Leeds: +44-532-xxx >> +44-113-2xxx

Leicester: +44-533-xxx >> +44-116-2xxx

Nottingham: +44-602-xxx >> +44-115-9xxx

Sheffield: +44-742-xxx >> +44-114-2xxx

You can start using the new numbers now.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers' Corner: Making Noises with APL	Adrian Smith	118
Technical Correspondence		123
Causeway: a Technical Architecture	Duncan Pearson	126
No SX please, we're APLers	Ray Cannon	143
Bracket Axis Simulation for APL*PLUS II/386	Allan Gay	150

Hackers' Corner: Making Noises in APL

by *Adrian Smith*

Introduction: What is Sound Anyway?

Ignoring (for the moment) the odd fact that we have two ears, sound is a strictly one-dimensional phenomenon. All we 'hear' when listening to a room full of people talking (or to Beethoven's Fifth in the Albert Hall) is a sequence of positive and negative variations about the normal air pressure in the room. It is easy to get confused and think of a sound wave as some kind of complex pattern of vibration going up-and-down and side-to-side, travelling towards us from each person in the room (or instrument in the orchestra) as a separate channel of information. Wrong! It is the brain which somehow buffers the simple vector of amplitude values and deduces what the source of each must be. You may be able to **listen** to the second flute over the background of strings and brass, but you certainly don't **hear** it!

Once you understand how simple sound waves are, you will appreciate how easy they are to model in APL! Take a moment to think about it, and read on.

How are Sounds made by Windows?

This also turns out to be easier than expected! Windows can play 'wave-forms' held in files called .WAV — several are supplied with your system (such as DING.WAV) and many more turn up on various games disks. I would recommend buying the *Microsoft Entertainment Pack Vol.4* just for the noises! Unusually, these files are held in a really simple format — essentially a 44-byte header structure (which is pretty standard) and then a simple sequence of byte values in the range 0-255 giving the pressure variation around a 'normal' value of 128. The rate these values are sent to your sound card (or PC Speaker driver) is determined by the 'sampling frequency', typically set at 11,000 per second. I use 14,567 for no good reason other than that I can hear frequencies up to near this value through my cheapo speakers (Goodman's Active 65 — £34 the pair from Dixons, highly recommended). Younger readers with Hi-Fi may need to go above this, but *WS FULL* awaits the unwary!

Playing existing .WAVs from APL is really easy (Vector 10.3, page 99) ... but what we are about here is making new ones with only some basic physics to

guide us. Let's start with a pure tone (a sine wave) of constant amplitude and work up from there:

```

V {r}←Tone arg;⌈IO;sam;freq;ampl;data;size
[1]  A Pure tone at given frequency (amplitude 0 ← 128)
[2]  A Sounds for exactly 1 sec at Hi-Fi sampling rate.
[3]  ⌈IO←0 ◊ sam←14567
[4]  freq ampl←2+arg ◊ ampl←128|ampl
[5]  data←|ampl×sam⌈1002×freq×(1sam)÷sam
[6]  A Adjust zero point to 128 +/- data
[7]  r←128+data
V

pTone 200 128
14567
80+Tone 200 128
128 139 149 160 171 181 191 200 209 217 225 232 238 243 247 251
253 255 255 255 254 252 249 245 240 234 228 220 212 204 195 185
175 165 154 143 132 121 110 99 89 78 68 59 50 41 33 26 20 14 10
6 3 1 0 0 0 2 5 9 13 18 25 31 39 47 56 66 76 86 97 107 118 129
140 151 162 173 183 192

```

The next stage is to append a valid header structure to the front of this:

```

V r←AddHdr snd;hdr;sam;size;⌈IO
[1]  A Add header structure to sound.
[2]  A      R I F F {size}      W A V E f m t
bl      fm ch sam      bps      blk bps      d a t a
[3]  hdr←82 73 70 70 6 192 0 0 87 65 86 69 102 109 116
32 16 0 0 0 1 0 1 0 17 43 0 0 17 43 0 0 1 0 8 0 100 97 116 97
226 191 0 0
[4]  ⌈IO←0 ◊ sam←14567 ◊ size←ρsnd
[5]
[6]  hdr[24 25 28 29]←4ρ⌈256 256⌈sam
[7]  hdr[3, /4 40+⌈14]←8ρ⌈256 256 256 256⌈size
[8]
[9]  A Return completed structure, ready to be played ...
[10] r←hdr,snd
V

```

If you really want to know what is going on on line [3], you will have to read the SDK manuals! The comment on line [2] is a rough translation of the byte values directly below, and indicates where values must be inserted into the constant data. The tricky bit is getting the encoding right — as well as expressing our APL integers as short-integer pairs, we must reverse the byte ordering (lines [6] and [7]). Here is the tone with its header stuck on the front:

```

48+AddHdr Tone 200 128
82 73 70 70 231 56 0 0 87 65 86 69 102 109 116 32 16 0 0 0 1 0
1 0 231 56 0 0 231 56 0 0 1 0 8 0 100 97 116 97 231 56 0 0 128
139 149 160

```

... hopefully, you can see where the data got inserted! All we need to do now is to write this lot to file (with a .WAV extension) and Windows will play it like any other sound. I suggest using *rput* from Vector 11.1 page 127. A more interesting challenge is to play the sound directly from an image in memory — this saves the overhead of file I/O, and avoids cluttering your disk with lots of failed attempts.

```

v PlaySound sound;GlobalAlloc;GlobalLock;GlobalUnlock;
  GlobalFree;handle:pointer;sink;hmemcpy;sndPlaySound
[1]  a Add header to sound and play from memory image.
[2]  sound+AddHdr sound
[3]  [NA'I kernel.P16|GlobalAlloc U U4'
[4]  [NA'I4 kernel.P16|GlobalLock U'
[5]  [NA'I kernel.P16|GlobalUnlock U'
[6]  [NA'I kernel.P16|GlobalFree U'
[7]  [NA'kernel.P16|hmemcpy I4 <U1[] I4'
[8]  [NA'I mmsystem.P16|sndPlaySound I4 I'
[9]  handle+GlobalAlloc 0(psound)
[10] pointer+GlobalLock handle
[11] sink+hmemcpy pointer sound(psound)
[12]  a The <4> is the flag to play from memory at <pointer>
[13] sink+sndPlaySound pointer 4
[14]  a Don't forget to free the memory!!
[15] sink+GlobalUnlock handle
[16] sink+GlobalFree handle
v

```

What a lot of Microsoft programmers it must take to change a lightbulb!! All we are actually doing is moving the completed sound into memory, and telling Windows to play it from there! If you were to execute:

```
PlaySound Tone 500 128
```

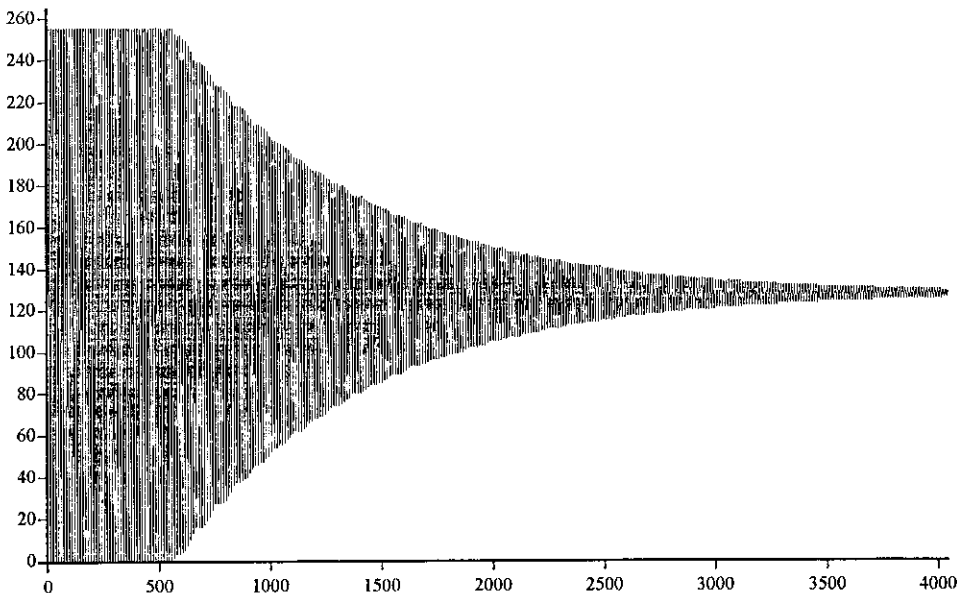
You would hear a gentle but pure tone through your speakers. I just did!

Adding Texture and Realism

At this point, I should really stop and let you experiment on your own — I have only spent an hour or two messing around with different functions to hear the effect. I would suggest 'over-driving' the data by using an amplitude above 128 for the early values and clipping the tops to give a much squarer waveform, and

using a simple decaying exponential to model 'die-away' of the sound. Here is a first attempt, plotted as a line-graph by the *RAIN* workspace:

```
chset 'nomark' ◊ chplot Sound 1200 128 ◊ PG+chclose
View PG
```



```

V {r}+Sound arg;□I0;sam;freq;ampl;dur;data;size;dieaway
[1]  Reasonable 'electronic piano' sound.
[2]  freq ampl+arg ◊ dur+2\400÷freq ◊ sam+14567
[3]  □I0+0
[4]  size+└dur×sam
[5]  dieaway+freq÷1000000
[6]  ampl+2×ampl×*-dieaway×\size
[7]  data+└ampl×sizep1002×(freq×dur)×(\size)÷size
[8]  Reasonable 'electronic piano' sound.
[9]  data+(ampl>2)/data
[10] Reasonable 'electronic piano' sound.
[11] data+255\0└128+data
[12] r+data

```

V

Notice how the die-away starts by making the sound 'purer' at the leading edge, and then cuts in to reduce the amplitude until there is so little left we can trim off the tail completely.

You can improve this further (and the effect of this minor change is quite surprising) by adding random noise to the very beginning:

```

v {r)+Square arg;[10;sam;freq;ampl;dur;data;size;dieaway;vol
[1]  freq vol+arg * dur+2[400÷freq * [10+0 * sam+14567
[2]  size+[dur*ampl
[3]  dieaway+freq÷1000000
[4]  ampl+1000*+dieaway*size
[5]  data+[ampl*sizep1002*(freq*dur)*(size)÷size
[6]  • Add random noise to leading edge (percussive effect)
[7]  (200+data)+÷7200p1000
[8]  data+[data*vol÷0.1*[|data
[9]  data+(ampl>2)/data
[10] data+255[0[128+data
[11] r+data
v

```

... which comes out as a pretty acceptable Xylophone.

```
chset'nomark' * chplot 1200+Square 2000 128
```

... which I don't have space to include, but if you try it you will see how the random noise messes up the regularity of the waveform (even more over-driven in this case) to give the effect of a hammer hitting an instrument.

That should be enough to get you started — making silly noises is all good educational fun, but I can see a lot more uses for this stuff in analysing recorded sound, for example comparing Chaffinch dialects across the UK or making sense of Dolphin-speak. Once you know how to read the .WAV format, you can get the data into APL and start doing something seriously interesting with it.

TECHNICAL CORRESPONDENCE

From: Norman Thomson

July 25, 1994

This is in response to Gordon Sutcliffe's letter in which he asks how in J the results of combinations of & and @ can be "derived analytically". The answer is there is no problem — simply follow the rules stated explicitly in the *J Introduction and Dictionary*.

There appears to be a mistranscription of some sort in the second paragraph of the letter, however assuming that the problem is to predict the results of

1. x u & v & w y 2. x u & v @ w y
3. x u @ v & w y 4. x u @ v @ w y

start by observing the monadic and dyadic rules for & and @, namely

$$\begin{array}{ll}
 u \& v & : \quad u(v \ y) \\
 & (v \ x)u(v \ y)
 \end{array}
 \qquad
 \begin{array}{ll}
 u @ v & : \quad u(v \ y) \\
 & u(x \ v \ y)
 \end{array}
 \begin{array}{l}
 \text{Monadic} \\
 \text{Dyadic}
 \end{array}$$

Only conjunctions and verbs appear to the left of the colon, nouns are introduced to the right. (As an aside the monadic cases of @ and & are identical, a point made explicitly in the Dictionary.)

Now extend the above rules to pairs of conjunctions:

$$\begin{array}{llll}
 u \& v \& w & = & (u\&v)\&w & : & (w \ x)(u\&v)(w \ y) & = & (v \ w \ x)u(v \ w \ y) \\
 u \& v @ w & = & (u\&v)@w & : & (u\&v)(x \ w \ y) & = & u \ v(x \ w \ y) \\
 u @ v \& w & = & (u@v)\&w & : & u(w \ x)v(w \ y) & = & u(w \ x)v(w \ y) \\
 u @ v @ w & = & (u@v)@w & : & (u@v)(x \ w \ y) & = & u \ v(x \ w \ y)
 \end{array}$$

proving conclusively that cases 2 and 4 are identical. The derived expressions for 2 and 4 demonstrate well how @ should be thought of as "in sequence", a point elaborated at greater length (as well as much more on conjunctions) in *J-ottings* 3.

More on Compositions ...

From: Roger Hui (Iverson Software Inc)

20 Sept 94

In Vector Volume 11 Number 1, Gordon Sutcliffe posed several questions on compositions in J. First, how is the following behaviour derived?

```
x u&v&w y <--> (v w x) u (v w y)
x u&v@w y <--> u v (x w y)
x u@v&w y <--> u (w x) v (w y)
x u@v@w y <--> u (w x) v (w y)
```

The conjunctions @ and & are defined in the *J Introduction and Dictionary*, release 2:

```
u@v y <--> u v y      x u@v y <--> u x v y      p.172
u&v y <--> u v y      x u&v y <--> (v x) u (v y) p. 176
```

A conjunction (or adverb) applies to any verb, whether primitive, derived, or "user-defined" (p. 74, p. 78, and in examples throughout).

The parsing rules specify how sentences are interpreted: a consequence of the parsing rules is that (quoting from p. 79) "the left argument of an adverb or conjunction is the entire verb phrase that precedes it". Thus, `u@v&w` is parsed (`u@v`) &w, and

```
x u@v&w y
x (u@v) &w y      parsing rules
(w x) (u@v) (w y)  & dyad
u (w x) v (w y)    @ dyad
```

The other three cases can be derived similarly. The analysis of such compositions is the same in other APL dialects.

Mr Sutcliffe then proceeded to comment on trains of verbs (hooks and forks). A fork is a sequence of three verbs; a hook is a sequence of two verbs. Parsing from right to left and preferring forks over hooks (a consequence of the trident rules preceding the bident rule, p. 80), a unique meaning is assigned to a sequence of verbs of any length.

For example:

x (p q r s t u v) y	
x (p q (r s (t u v))) y	parsing rules
(x p y) q x (r s (t u v)) y	fork dyad
(x p y) q (x r y) s x (t u v) y	fork dyad

(x p y) q (x r y) s (x t y) u (x v y) · fork dyad

x (q r s t u v) y	
x (q (r s (t u v))) y	parsing rules
x q (r s (t u v)) y	hook dyad
x q (r y) s (t u v) y	fork monad
x q (r y) s (t y) u (v y)	fork monad

Roger Hui
Iverson Software Inc.
Toronto, Canada

Causeway: a Technical Architecture

by Duncan Pearson

Background

Causeway is the amalgamation of the ideas from two embryonic GUI management systems developed by Adrian Smith and myself through most of last year. Adrian was working on the logical successor in the Windows environment to his FSM/WIN utility set (see Vector 10.1 page 67) and I was building a new GUI development environment under Dyalog APL for the OR Systems team at Nestlé. In November last year we sat down together and planned an amalgamation of the two systems as material for the BAA GUI Workshop and subsequently for use within Nestlé. The new system was based on the structure of Adrian's system DBX, but incorporated ideas from my system such as a table of events and behaviours, local variables on forms and automatic update of all controls linked to any variable.

The Aims of this Article

In the course of this article I will discuss how the elements of the dialogue box definition determine each object's appearance and behaviour; how the system is managed internally to enable the effect of localisation at form level; and finally how the contents of the object prototype table enable the hard-core techie to create new controls or modify those that come free with the workspace.

There are in effect three levels of user of the Causeway system.

- The **application user** is the ultimate beneficiary of the Causeway system. He is the one who is using the application developed under Causeway to do a job of work. He might be a production planner, an accountant or perhaps a managing director.
- The **application developer** puts together a number of forms with controls on them, some of the controls being simple, such as edit fields, and some being quite complex, such as that for PostScript graphics display. He links the controls to variables and defines the application specific behaviour of the controls, putting actions on events. He produces the applications used by the user. He need know nothing about the way in which the underlying APL interpreter handles the graphical user interface.

- The control prototype (or "class" for short) developer is concerned with enriching the variety of controls available for the application developer to use. He defines the inherent (or default) behaviour of the controls, the way they appear, the type of data they display and the way in which they display it. He may build a new control class from scratch (as we have done in many cases) or he may change aspects of an existing control. He has a detailed knowledge of the underlying interpreter's GUI facilities.

Underneath all of these users you have me and Adrian (and hopefully Marc Griffiths for APL.68000) writing the Causeway code to enable all of this to happen and making sure that it happens in the same way regardless of the APL interpreter that the user is running.

Compatibility

One of the major aims of the Causeway system is that if you as an application developer follow certain guidelines you should be able to develop forms that work unchanged on whatever interpreter you use, provided of course that we or someone else has implemented Causeway in that environment. We have a full version in Dyalog, a working but not yet fully featured version for +III (evlevel 2) and a stated intention from Marc Griffiths to implement it under APL.68000 for the Apple Mac.

While all the writers of different interpreters are fighting to present their approach and their set of quad functions as the sensible way to write GUI systems, not only do they give you too much feature and not enough organisation, but also they give you incompatibility. It is possible to take a Causeway form definition in Dyalog, convert it to a text vector, pass it through the clipboard to +III, reconstitute it on the other side, and it will run. The form definitions will constitute much of an application written under Causeway and it is possible for an application developer to write a very good system without ever touching a GUI quad function, nor even knowing under what interpreter he is running.

In order that form definitions be portable a number of things need to be considered. Firstly all control types used in the form definition must behave in the same way. They must occupy the same screen real estate and most importantly they must expect and return data in exactly the same way. It does not matter as much if a radio button group has a 3D appearance in one system but not in another, provided that both accept a vector of options and return the index of the selected button.

Secondly the events mentioned in the behaviour tables for the controls on the form must be the same on both systems. This is why we prefer the use of our two-

character event codes. We ensure that they will be available on both systems, or at least we document which are compatible. Under +III you can use any event name that you like and so have access to such +III specific events as `KeyDown`, but you should be aware that if you use them *your form definition is not portable*. Not only must the events appear on both systems but their arguments must be the same. It is no good having the `MouseDown` event in both systems if on one system the position is reported in pixels and on the other in character units. We jump through hoops to ensure that, for what we term "compatible events", the value you see on execution of an action is the same no matter what implementation of Causeway you are using.

Modules

An important part of Causeway, but one mainly unconnected with the GUI, is the idea of *modules*. A module is a way of containing functions and data associated with a particular task. There are two main reasons, to remove the necessity of prefixing each function in the set with some distinguishing letters to separate it from the other sets in the workspace, and to allow effectively static local data. In the Dyalog APL implementation of Causeway the designer, *Dbx*, is a module.

A module is nothing more than a form (typically iconised) being used as a container for function and variable definitions.

The idea is that you can call a function in the module and while it is running it will have access to all the other functions and variables of the module. In the same way that the environment is prepared before executing an action on a normal form, so the function to execute a command in a module, *Gui_do*, creates an environment in which to run the command that you specify. The data property of the form contains a list of function names, a corresponding list of definitions, a list of variable names and a list of values for those variables. *Gui_do* simply shadows the two name lists, defines the functions and variables, runs the function specified in the first element of the right argument on the remainder of the right argument, saves the new values of all local variables and then exits.

If you want to run the same functions on two different sets of local variables then you just create two instances of the module, each contained in its own form and each with its own set of local data. I can have two instances of my Oracle module, each connected to a different database and each acting independently.

What is Causeway?

The main ideas behind the current system have been described in Adrian Smith's introduction to Causeway in Vector 10.4, pages 64 – 76.

Briefly they are:

An object on the screen should be the visual representation of a variable. For example a text vector would normally be associated with an edit field and a numeric scalar would be associated with a special edit field that only allowed numeric input and which furthermore would put a numeric scalar back in the variable when the user changed it. The developer should not have explicitly to change the screen representation, nor should he have to read it back into a variable when it is changed. Instead changes to the data on the screen should automatically update the variable, and changes to the variable should automatically update the screen representation.

The variable to which a control is linked should have the option of being entirely contained within the form. It should be able to be a "local" variable and a change in it should not affect the global environment.

For any event that could happen to a control the developer could define a line of APL to execute, an "action" of the control. A typical event for a push button is "being pressed". Most buttons have just one action on the press event (called in Causeway SL, for select). On any control that is linked to a data variable it is also possible to put an action that is run before the variable is updated to validate the new value.

If a variable should change during the execution of an action for a control on a form, then all controls linked to that variable, whether on that or on other forms, should be updated. In order for this to happen the application developer must tell the Causeway system in some way, what variables could change as a result of any action.

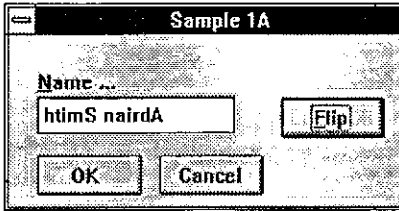
Any action on a control of a form should execute in the context of that form. Any variable defined as local to the form should have its locally defined value on execution of the expression. When the expression has been executed the new value should be saved back "in" the form.

If, during execution of an action on a form A, a new form B is created, then any variables local to A should be global to actions executed on B, in the same way that a function *GOO* called from a function *FOO* has access to the variables local to *FOO*. In effect we would have a form stack in the same way that we have a function stack. The way in which it differs from a function stack is that instead of having a linear structure it would have a tree structure. A form might have more than one child form open at one time.

If the APL interpreter does not provide a control that the application developer needs in his application, such as a spin box, but its appearance and behaviour could be duplicated by a composition of various basic controls (a spin box is just

an edit field and two buttons) then it should be possible for an experienced developer to encapsulate the code and state variables necessary to build and manage that control, in such a way that the normal application developer need not know that it is not a basic control, and that the code and state variables should not be visible, either from the workspace, or from actions executed on the object.

The Form Definition Matrix



As you will have seen from a couple of the diagrams in the previous article (Vector 10.4, page 70), a form is defined by a ten-column matrix with the top row giving the characteristics of the form itself and each subsequent row defining a control on the form.

show sample1a

Ty	Caption	Pos	Size	Sty	Ref	Data	Listen	Event-Action Table	Id
FM	Sample 1A	320 156	120 272	{Dbx}					
TX	&Name ...	44 20	25 136			name	name 0		
AC	&Flip	44 188	28 72	{ }				SL name+<name {name}	
EN	OK	84 20	28 72						
QT	Cancel	84 104	28 72						

For a control the first element defines its type, is it a button, an edit field, a spin box or what? The next four elements define its appearance (caption, position, size, style); the last five define its behaviour: with what variable(s) it is associated, what lines of APL will run as a result of which events on it, and by what name it is known to the application developer.

Type (Element 1)

This is a two-element text vector such as FM for form or TX for text field. It corresponds to the first element in a row of the class table. From this the system knows what functions to use to create the control, to read data from the control and to put data back into it, to "refresh" it. The definitions of these functions are held in the class table and they are only defined as needed and expunged once they have done their job.

Appearance (Elements 2 - 5)

Caption (2)

This is text vector which will normally appear above the top left corner of the control, but on the face of a button and in the title bar of a form. The management of the caption (the initial positioning and any subsequent movement) is handled by the system. Having a caption on each control means that the number of labels is greatly reduced. In other systems it is necessary to define a label for each edit field and then to have to re-align them whenever the field is moved in the design process.

In the caption of the form (as in the header of a function) the name is optionally followed by a list of local variables separated by semicolons. On creation of the form these variables are given the value 0 by default.

Position (3)

This is a two-element integer vector giving the position of the control in pixels relative to the form's client area (the bit in the middle). For a form it is the position relative to the screen. The first element is the vertical position and the second is the horizontal position.

If the first element is positive it is the distance from the top of the control to the top of the form. If it is negative then the absolute value gives the distance from the bottom of the control to the bottom of the form. The horizontal position is defined similarly with positive from the left and negative from the right. This gives the ability to anchor the control to any corner of the screen.

Size (4)

This is again a two-element vector giving in order the height and width of the control in pixels. If an element is negative then the absolute value gives the number of pixels by which the size in that dimension is less than the size of the form. If a control has negative height and width and the user makes the form larger, then the control will grow to match and retain the same borders.

Style (5)

This is a vector of style names, each of which modifies the appearance of the control in some way. Each name identifies a low level specification which will be used in the creation of the control to modify it. The style might specify the colour of the control or the relative position of the caption, amongst other things. The techie might wish to modify or add to the style table to produce his own effects. It can be used to implement a "house style", as shown here :

Disp Astyletab[1 9 10 20:] a Style tag, Gui-code, Allowed objects

<i>Dbx</i>	{{Border 2} {Accelerator 112 0}}	{FM RU}
<i>Password</i>	{{Password *}}	{TX NM}
<i>Dragable</i>	{{DRAGABLE 1}}	{LB IC BM TC}
<i>Arial14b</i>	{{font arial 20 0 0 0 800} {fcol 0 0 128}}	

Behaviour (Elements 6 - 10)

Reference (6)

This is an APL expression. How the result of the expression is used by the control varies from control to control as will be explained later. It generally specifies aspects of the control that may be changed by the application code during the lifetime of the control, but that cannot be changed explicitly by the user. For example the reference expression for a multiple selection list should give the elements of the list. The reference expression for a group of radio buttons would give the labels of the buttons. When a control is created and when it is subsequently refreshed (see below) the expression is executed and the result passed to a special function defined in the control prototype which updates the visual representation of the control.

Data (7)

This is another APL expression. It too is executed whenever the control is refreshed and the data passed to the "refresh" function in the same way as the result of the reference expression. The data expression for a list would give a vector of the indices of the selected elements of the list. For a group of radio buttons the data expression would give the index of the selected button.

In the case of the radio button group a change in the selected button should not affect the labels on the buttons. So if only the data has changed there should be no need to execute the reference expression and pass its value to the refresh function. On the other hand if the reference information has changed then it is essential that the data should be updated as, in the case of a list or radio group, the number of elements or buttons might have reduced and the old selection might give an index error. As you will see there is a mechanism to enable only the data to be refreshed.

The data expression has a special restriction as it is used for another purpose. If the user changes the data in the control the data expression is used to specify where it should be put. As such it must be possible to use the expression as the left hand side of an assignment. It must therefore be one of three things. It can simply be a variable name. It can be an indexed variable, in which case the expression must end with a "I". Finally it can be a selective specification. To explain better the restriction I will show what happens when the user changes the data in a control.

- A "read" function is defined which queries the control and returns the new data into a temporary variable.
- If the data expression is a selective specification then it is surrounded by parentheses.
- The data expression and the name of the temporary variable are then joined by an assignment arrow and the resulting expression is executed.

If the data is not a variable name, an indexing of a variable or a selective specification then this last execution stage will fail. Originally in my system I only allowed variable names, which made the job of tracking what things needed to be updated much easier, but the advantages of being able to use indexing outweigh the extra hassle it causes.

Interests (8)

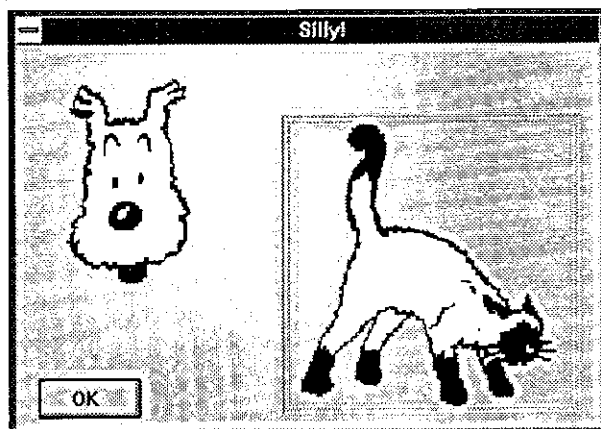
As mentioned in the preceding paragraphs, if a control is to be updated automatically then the system needs to know upon what variables it depends for its information. Then if it is told that one or more of these has changed it can arrange for the reference and data expressions to be executed and the results passed to the "refresh" function which will update the control. The eighth element of the definition is what tells the system what variables are used in the two expressions.

It is a two-column matrix with the first column a list of variable names and the second column boolean. For each variable the boolean tells the system whether it is used in the reference expression or not. If it is not (denoted by 0) then when the variable changes only the data expression is run and the refresh function may have much less work to do. This can save considerable time. If a number of the variables in which the control has an interest change, then the reference expression will be run if *any* of them is marked with a 1.

It is important to note that this matrix is the *only way* in which the system can know of the interest of a control in a variable. We considered analysing the expressions to find all referenced variables but it is quite possible for one of the expressions, especially the reference, to contain function calls. Moreover the combination of this element and the next are useful as a record of the dependencies within the system. The traditional function calling diagrams with which we used to document APL systems are inappropriate to this new way of working but by putting all the interdependencies within well defined structures, i.e. form definitions, we force a form of system documentation.

Actions (9)

The ninth element of the definition is where the functionality of the control is defined. It is a four-column matrix. For an event specified in the first column there will be an APL expression to execute in the second column. That expression may change the values of some variables and in order that any controls linked with those variables can be updated the third column contains a vector of the names of all variables that might have changed. All controls that have the names of any of these variables in the eighth element, the interests table, will be updated. As with the interests table, an entry in the third element of an action row is the *only means* by which the system can know if a particular variable has changed. If this section or the interests table are not completed correctly the automatic updating of controls from changed data, which is one of the main advantages of the system, cannot take place. If in a system things are not being updated as planned then these are the first places to look.



Within the APL expression the developer may use the characters alpha and omega which will be replaced with the control name and the event argument respectively.

On creation each control or form is given an arbitrary name by the Causeway system which will identify it to the underlying interpreter.

There are two ways of getting this name: through the use of alpha in an action on the control and through the control id described in the next section. The event argument is a means by which the system gives more information about the event that has happened, more often than not quantifying it. For example if the event is a resize of a form then the argument will contain the new size. If the event is a "mouse down" then the argument will contain among other things which button has been pressed.

show dbx_silly

Ty	Caption	Pos	Size	Sty	Ref	Data	Listen	Event-Action Table	Id								
FM	Silly!	312 376	272 412	{Dbx}					DAFT								
BM		20 28	30 30	{Dragable}		'snowy'		<table><tr><td>MD</td><td>PLAY 'dog'</td><td>{}</td><td>2*3=ω</td></tr><tr><td>MU</td><td>a Null</td><td>{}</td><td></td></tr></table>	MD	PLAY 'dog'	{}	2*3=ω	MU	a Null	{}		
MD	PLAY 'dog'	{}	2*3=ω														
MU	a Null	{}															
BM		96 192	30 30	{cutout}		'cat'		<table><tr><td>MD</td><td>PLAY 'meow'</td><td>{}</td><td></td></tr><tr><td>DD</td><td>PLAY 'growl'</td><td>{}</td><td></td></tr></table>	MD	PLAY 'meow'	{}		DD	PLAY 'growl'	{}		
MD	PLAY 'meow'	{}															
DD	PLAY 'growl'	{}															
EN	OK	236 16	28 72														

The dog barks on left-mouse, but the right-mouse gives standard (Dyalog) behaviour, and allows the user to drop the dog on the cat, causing a growl.

The final column contains an optional condition that will be executed before the main expression and, if it returns singleton 0, will inhibit the execution of the main expression. For any event there may be as many rows in the behaviour table as are required; the system will execute all that have an empty condition expression or whose condition expressions return 1. If for example you wish to put different actions on the left and right mouse buttons then you

should put two rows in the behaviour table, each with 'MD' in the first column. For the right button action the fourth element would be an expression querying the appropriate element of the argument, represented by omega, and returning 1 if it were "right". For the left button action the condition would return 1 on a left button press. The condition-action pairs are executed in turn from the top down; any changes to the environment caused by an expression will be in place during the execution of all subsequent expressions.

The use of two-letter codes to denote events may seem an added complication when the events now have names both in Dyalog APL v7 and in APL*PLUS III, but it is because these names are not always the same that we use our own names. Not only are the names sometimes different but what one interpreter might report as one event another might report as two, an example being "configure" in Dyalog and "Move" and "Resize" in APL*PLUS III. If you use the Causeway code you are guaranteed the closest compatible event and furthermore the arguments will be the same on both systems. It will be possible to use interpreter-specific names in place of the two-letter codes but at the cost of portability of form definitions.

Another reason that we use our own codes is that the Causeway system generates its own events. An example is the validate event "VL" which is generated just before the updating of a variable. The argument of the VL (reported in omega) is the new value for the variable. If you change this value then what you assign to omega is what gets placed in your variable (or rather the result of your data expression). Another is the create event, "CR". This is generated immediately after the creation of a control and before it is refreshed for the first time. It is particularly useful on a form, to set up any local variables that will be linked to controls on the form.

Id (10)

The final element gives a means by which the application developer can refer to a control or form. The function *Gui_id* will, for a given form and id, return the internal names of all controls bearing that id. As discussed these internal names can be used by certain Causeway functions or can be used by the more technically inclined application developers to perform low-level interpreter-specific operations on the control. As the same form definition may be used to define many instances of a form so an id might identify the same control on many different forms only one of which was intended. This is why it is necessary to specify the form as well as the id. The internal name of a form will be returned on creation of the form. If the form is omitted when calling *Gui_id* then it is taken to be the current form, the form on which the current action is being executed. This concept of current form will become more important when considering localisation. It is worth pointing out that the current form need not be the form with the focus, although that will often be the case. There is a mechanism by which the application can cause arbitrary events to be registered against controls on other forms. During execution of any action the current form is the form on which the event that caused the action was registered.

Localisation

As mentioned in the introduction, each form can have its own set of local variables which are listed in the caption to the form and which reside in the form itself. These are especially useful if the same form definition is to be used to define many independent instances of a form. Suppose you had a large document represented in the workspace as a vector of text vectors, each subvector representing a chapter, and you wished to edit many different chapters at the same time, each in a separate window. You could define a form with a notepad control for which the data expression was the document indexed by some integer variable representing the chapter. If that variable were only able to exist in the workspace then all the windows would look at the same chapter. If that index variable were to be local to the form, then each window would have its own version of it and could look at its own chapter. The underlying document would be the same and if two windows had identical index numbers then they would look at the same chapter and a change in one would be reflected in the other.

Associated with each active form there is a set of data specific to it. In both Dyalog APL and APL*PLUS III (and I presume by implication in APL.68000) this is held in the "data" property of the form¹. Part of the data associated with a form is a table of name/value pairs representing the local variables.

The way that Causeway handles local variables is closely linked with the concept of a current form, already described as the form from which the action currently being executed was taken. Simply, before executing an action Causeway prepares an environment for it by localising the names in the local variable table of the current form (either by using `⌈SHADOW` in Dyalog APL or by creating a temporary function with an appropriately generated header line in APL*PLUS III) and then assigning the values in the table to the names. After execution of the expression the new values of the named variables are saved back in the table before they are lost as we come out of the localisation. In this way the variables are not only local but also *persistent* across execution of different actions on the same form. If we follow the analogy between the form and the function it is as though the variables were local statics, something we do not yet have in APL.

If in the execution of an action a new form is created then it becomes the child of the current form. In its data property is an entry containing the name of its parent,

¹If we needed to implement Causeway in an interpreter that did not have data properties in its forms then we would just have a large table in the workspace keyed by form name.

the current form when it was created. In the same way that the local variables of a function are global to any called function, so the local variables of a form are global to any child forms. Thus, when the environment is prepared, the names and values of the locals are collected from a number of different forms, the value from the child form always taking precedence over the value from a parent (or its parent).

The process is as follows:

1. The system creates a large list of name/value pairs, going from each form to its parent adding the form's local variable table to the bottom of the list and stopping when it reaches a form with no parent.
2. It then finds the first occurrence of each name (which will be the most local) and uses the corresponding value in its definition, noting where in the master table it was found.
3. After the execution the new values are replaced where they were found in the table.
4. Finally each part of the table is returned to the form from whence it came.

Thus we simulate the way a function stack has localisation of variables at different levels and the way that a higher level definition shadows the definition at a lower level in the stack.

Registration and Notification

When a variable is changed by an action all controls linked to that variable are refreshed so that what they display is based on the new value. In order that the system knows what controls to refresh, that information must be saved in a table somewhere, keyed by variable name. When a variable is changed then the table is searched for all references to the variable name and all the corresponding controls updated. Initially it was this easy. There were no local variables and so all changes would be to global variables and there need only be one table.

The fact that the variable linked to a control might be localised somewhere down the form stack adds a level of complication. What needs to happen is that if a variable local to a form is changed, then all controls *that are in that form or its children*, and that are linked to that variable, should get updated. The way that this is managed is that each form has, in addition to the set of local variables, a table showing the controls in the form and its children (and their children) that are interested in those variables. We call this the registration table. If a variable in a form is marked as changing as a result of an action, then the registration table

for the form is searched for controls interested in the variable. Each one is then refreshed regardless of the form in which it is found.

The registration table is created when the form is created and whenever a new control is created with an interest in a variable on the form, another row is added to the table. For any control there could be many variables in which it was interested, which might be localised in different forms. On creation of the control the list of variables in which the control is interested (column one of element eight of the control definition) is compared with the local variable table on the form in which it is created. If there are any matches then the appropriate rows are added to the registration table of the form. The matched variables are removed from the table of interests and the process is repeated with the form's parent. This continues until we run out of forms or the interests table is depleted. If we have reached the base form and there are still names left in the interests table, then they must refer to global variables and so entries are made in the registration table of the root or system object, the parent of all first level forms.

A similar process happens when an action changes some variables. We have a list of the variables that might have been changed (from the third column of the behaviour table) but we don't know in which forms they are held. We work down through the form stack finding where they have been localised and search the corresponding registration tables for controls with an interest in the change. Any that we find we refresh.

Control Prototypes

Ultimately the set of controls provided with Causeway will prove insufficient and there will be a need to create new ones. Within Causeway all the types of controls, from the form down to the humble edit field, are defined by a few function definitions and a behaviour table, all of which are generally held in a table in the workspace. These functions are used by the system whenever it needs to do anything to a control that it does not know how to do. In a number of places we talk of *creating*, *refreshing* and *reading* controls. For different controls these things are done in different ways. The Causeway system itself does not know how to read data from an edit field, it doesn't know which property to query. However, because for each control there is a function in the prototype designed to read data from that particular type of control, the Causeway system does not need to know. We have separated the things fundamental to Causeway, such as the running of actions, the building and re-saving of environments and the registration and notification, from the things specific to particular controls. When a control is created the Causeway system fetches and defines a "create" function from this table and passes it the control definition together with the internal name to use

when creating it. If the "create" function succeeds in creating the control then the generic Causeway code will add event handlers from the class table to enable it to work and then link it into the system, making entries in the registration table of its form and of its form's parents. When subsequently we need to refresh the control we fetch and define the "refresh" function which we pass the name of the control and the data with which to refresh it.

As the implementation of the same control is generally different under different interpreters, so the structure of the prototype table may differ also. However the principles are the same. I will explain the purpose of the common parts of the prototype but for detailed information about the construction of new prototypes you will need to refer to the documentation in the version for your own interpreter.

The most fundamental function, which must always be provided is the "create" function. This, as mentioned above, is generally given the definition and the intended name of the control as arguments. It will create whatever basic interpreter-defined control or controls are necessary to implement the intended behaviour of the control. Here is a Dyalog APL example:

```
r←nm cr def;cap;ps;sz;acc;kc
a Create AC (action button) object as <nm>
a Splits out 'Hint' from caption, and activates
a 'Accelerator' from 'Label=Ctrl+A' if given
r←0 ∘ cap ps sz←def[2 3 4]
nm □WC'BUTTON' 'ps sz,Gui_style 5def
a Remove hint, leaving visible part
cap←nm Gui_hint cap
a Anything after '=' should be activated ...
nm Gui_accel(cap1='')+cap
nm □WS'Caption'cap
```

If the control has a separate caption it will create it² and if the control is composed of more than one basic control then it will attach the necessary event handlers (functions to be called on particular events) to all the component controls. It needs to do this because the generic Causeway code only knows the name that it passes to the "create" function, which will in general be used for the main component control. An example would be the spin box. The main control might be the edit field and its event handlers would be applied by the system, however there

²In Dyalog and +III there is a function Gui_caption that will do this in a consistent fashion using a label. The reason that this is not used on all controls is that the caption might already, as in the case of the form or the push button, be incorporated within the control anyway

would also be two buttons which would be created by the "create" function with names chosen by it (generally derived from the main control name) and with event handlers applied also by the create function.

If the control is to display data that can be varied by the application, given by the results of the reference and data expressions, it will need a function to apply the data to the control. This may be simple, as in the case of an edit field where the data is assigned to the "text" property of the control, or it may be complex, as with the numeric matrix object that I developed for Nestlé in which, as the shape of the linked matrix could have changed, the refresh function recalculated the cell size to fit the cells optimally within the area for the whole control, and moved the focus to the nearest cell in the new matrix corresponding to the focus position in the old. As mentioned in the section on the form definition matrix, the refresh function can be passed simply the result of the data expression or passed the results of both data and reference expressions. If for any control there is no refresh function then the reference and data expressions and the interests table are effectively ignored.

```
r+nm rf arg;ref;val;flg
a Refresh LB (label).
a This just uses DATA at the moment
r+0 0 flg val ref+arg 0 →(0ερ,val)+0
val+,*val
a Check what was already there (saves flicker)
→(val=nm □WG'CAPTION')+0
nm □WS'CAPTION'val
```

For controls in which the user can change the data there must also be a read function to take the changed data out of the control for assignment into the data expression. As with the refresh, if there is no read function then no assignment will take place, regardless of the contents of the data expression.

```
val+msg rd dum
a Read from NP (notepad) object
val+(>msg)□WG'TEXT'
```

The thing that triggers the read, that activates Causeway to define and run the read function, is defined in another behaviour table, this time in the class table. Each row of this table links an event with either with a function to run or with an instruction for the system to execute the "read" process. These call-back functions are also held in the prototype table and are defined on demand by the Causeway system. This whole process will be simplified considerably in Dyalog v7 with the use of namespaces. There the control specific functions will be defined in the

namespace of the control by the create process and the interpreter will execute them directly on generation of the event.

Currently in both the Dyalog 6.3 and the +III versions we have two other mandatory control specific functions.

In order to implement the re-sizing and re-positioning behaviour described above it is necessary to have a function to move the control, any of its component parts, and its caption if that is defined separately. We call this the *resize* function. We anticipate that in the Dyalog v7 version of Causeway this function will be unnecessary as Dyadic have provided an 'attach' property which implements the behaviour at the interpreter level. Here is a typical *resize* function from the current version of Causeway for Dyalog:

```

r←nm rs def;ps;sz;flg;adj;⌵TRAP
a Reconfigure (notepad). Designer sets flg←0
a Caption and [cutout] must follow.
ps sz flg←def ◊ r←0
adj+ps-nm ⌵WG'posn'
a Refuse to go below 1-char high ...
sz+24 72[ sz
nm ⌵WS('posn'ps)('size'sz)
nm Gui_cutout flg ps sz
+(adj^.=0)◊0 ◊ ⌵TRAP+6 'C' '←0'
a Bump caption to the right place ...
nm←nm,'caption' ◊ nm ⌵WS'posn'(adj+nm ⌵WG'posn')

```

Also we prefer in our form designers to use simpler versions of the controls than in the real system. This is mainly to reduce visual clutter. To implement this we require that for each class there be a "design" function which creates a dead control of the same size, position and general appearance as the intended control. In many cases this "design" function will be identical to the "create" function.

In Dyalog APL we have an operator called *Gui_apply* that takes as its function argument the ⌵OR (the definition) of a function and applies it to the arguments of the derived function. This avoids all manner of name clashes as during execution the function has the name assigned to it in the header of the operator.

Under +III, which does not have ⌵OR or defined operators, I have implemented *Gui_apply* as a function that takes, instead of an OR, a VR, replaces the given name with one of my own choosing (that incidentally appears in the header line of *Gui_apply*), defines the function with ⌵DEF and then runs it. Thus I have the same effect with a little more work.

Causeway for DOS?

We have noticed that systems written in Causeway seem to be quicker to write and, more importantly, more robust than the equivalent systems written under DOS. This is in stark contrast to the experience of our colleagues writing in C, Powerbuilder and other Windows development tools. It could be that the encapsulation of system code and the management of the control and screen update logic is at just the right level. Whatever it is, we are aware that most of what we have done could have been done just as easily under DOS. None of it relies on any revolutionary features of the GUI that could not be simulated easily in a DOS APL. While neither of us have the time at the moment to spend on a project like this, if there is interest in a DOS version we might be persuaded. Alternatively if anyone out there enjoys a challenge then either of us will be only too glad to provide advice and to go into the code and principles in more detail.

The Future

There are two things that we would like for Causeway in the future, less code and more platforms.

The first objective of less code is one that is not really in our hands. We have noticed that, by coincidence, some of the things that we worked quite hard to implement in Dyalog 6.3 have been made considerably easier in version 7. The implementation of Causeway for v7 will have a lot less code as a result. Furthermore with support for VBX controls it will be possible to implement complex controls in C once they have been prototyped in APL, refined, and properly specified. It is a satisfying trend and I would like more. Top of the list of things that would make my life easier is a "Variable Changed" event. Instead of all the mess with registration tables and searching through form stacks I would like to put a call-back on a control that said "refresh me if this variable changes" and for the system to handle it. It will know when it is changing any particular variable so it should be able to tell me.

Finally we would like to see Causeway implemented in all the major APL interpreters that support a GUI. Primarily this means in APL2 under OS/2, in APL68000 on the Mac and for all four of the main APLs under UNIX. One way in which this aim might be furthered would be for the BAA or SIGAPL to sponsor a project to this end. The version for Dyalog APL/W has been put in the public domain through the extremely far-sighted and public spirited approach of Nestlé UK on whose time much of it was developed. They wish to see the use of APL grow in the world, thus ensuring the survival of the interpreter developers and protecting the considerable investment that they have in APL systems. Adrian and I have also done a great deal of work in our own time on this project. We can probably manage the development and support of one version each but no more.

We need help.

No SX please, we're APLers

*What your mother never told you about the naming of the Intel chip set
by Ray Cannon*

During a recent meeting a question arose about which Intel 486 chip was best for running APL, a 486DX or 486SX. In reply, I will attempt to answer by explaining the Intel chip-set naming scheme.

In the beginning, Intel created the 4040, but that was before my time.

Once upon a time, there were three famous 8-bit chips, the Zilog Z80, the Motorola 6502, and the Intel 8080. They all had a 16-bit address bus (which could access 64K bytes of memory), an 8-bit data bus (each read or write to memory consisted of 8 bits of data), and a mixture of 8 and 16-bit registers. (Both the Zilog and Intel had register pairs that could be used for some 16-bit operations.)

In the business market, the Z80 was king and ruled through the CPM operating system. It also saw the start of the home market with the Sinclair ZX80 and ZX81. However, the 6502 soon became popular in home computing (Apple, BBC, Commodore). The 8080 gained support in industrial control applications.

The Genesis of the IBM PC Range

And someone said "Let there be 16-bit computing" and there was the Intel 8086 and IBM thought it was good, but the Intel 8088 was cheaper (and 20% slower), so they put that in their new PC instead.

Both the 8086 and the 8088 have the same instruction set, so they look identical to the programmer, but the 8086 has a 16-bit data bus (and a 6 byte queue) while the 8088 has a 8-bit data bus (and only a 4 byte queue), so the 8088 is slower. They could both run at a clock rate of 5MHz. (A newer member of the family, the 8086-2, could run at 8MHz.)

IBM used the 8088 in all their original PCs, XT's and portable PC (the PCjr). The first Compaq Deskpro used the 8086.

Both the 8086 and 8088 had a 20-bit address bus which could access up to 1Mb of memory. (Internally this 20-bit address is composed of 2 overlapping parts, a 16-bit segment address and a 16-bit offset. This leads to fast processing of data within a 64K address range, but real problems when trying to deal with data larger than 64K.)

Intel next produced the 80186 and 80188 (sometimes the leading "80" was dropped from the name). These were bigger, requiring less external circuitry, better as they had a few more instructions and most importantly faster as they ran at 8MHz. IBM did not bother with them, but Tandy sold lots of compatible PCs with them in.

Intel's next processor was the 80286 (which by now almost everybody called the 286). This was designed to address up to 16Mb of memory (24-bit address bus) and could be fooled into believing it had up to one gigabyte via "virtual memory". It could also multi-task and ran at 10Mhz. Like the 186 it has a 16-bit data bus and a 6 byte instruction queue.

IBM fell in love with the 286 and used it in their PC-ATs.

STSC, knowing a good thing when they see it, developed an APL for the PC and called it "APL*PLUS/PC".

Moving to 32-bit Computing

In the meantime Dyadic were developing Dyalog APL under UNIX (on a Zilog Z8000 I think). When asked if they could port their APL to the IBM, they said "No, it needs a 32-bit machine." Now Intel heard about this, and not wanting to lose the IBM PC contract to Zilog embarked on producing the 32-bit chip and called it the 386 *[Ed: A likely story!]*.

The 386 is upwards compatible from the 8088 but has 32-bit registers and a 32-bit data path. Its address bus can access up to 4Gbytes of memory. It was designed to handle both "multi-tasking" and "multi-user". Each user can have up to 64 terabytes of virtual-address space.

Now, producing circuit boards with 32-bit wide data paths is a lot harder than producing ones with 16-bit paths. So Intel came up with the idea of having a 32-bit width internally and a 16-bit external path (just like the old 8088 compared to the 8086). So, if they had had any sense, not to confuse anyone, they would have used the old naming standard and the 388 would have been born, except they didn't. They actually called it the 386SX, and just in case anyone was confused, they started calling the 386 (with the full 32-bit data path) the 386DX.

The advantage of the 386SX was that you could take a basic 16-bit AT type board and plug in a 386SX chip to get (a substandard variety of) 32-bit computing at 16-bit prices.

Adding Floating-point Maths

Now let's "float" back to a "point" in time shortly after the 8086 first appeared. The 8086 only supports integer maths in its instruction set. It did not support "floating-point" maths. So, Intel produced a "8087 Maths Co-processor" which could be plugged in alongside the 8086, 8088, 80186 or 80188, to do the hard work. Intel also call the co-processor an NPX — Numeric Processor Extension, hence the "Intel 8087 NPX".

To allow for the addition of a co-processor chip, most PC motherboards came equipped with an empty co-processor socket.

With an NPX, a machine instruction to multiply two real numbers could be processed. Without an NPX, this hardware instruction has to be emulated in software. The NPX hardware can perform this (and other maths) operations many times faster than the software. Also the NPX "can improve the speed of multiple-precision software algorithms for integer operations by 10 to 100 times" (Intel 387DX User's Manual). To keep the 8086/8087 chips in step with each other, each NPX instruction had to be preceded with a "WAIT" instruction. Intel's assembler ASM86 did this automatically.

With the advent of the 286, Intel produced a new co-processor, the 80287 NPX. The 80286/80287 operating in "Real-Address mode" will execute 8086/8087 programs without major modifications. However differences between the way 8087 and 80287 handle numeric exceptions may require different exception handling routines. Also, the 80287 does not require "WAIT" instructions. So reassembling using Intel's ASM286 assembler will produce faster and more compact code.

For the 386DX they produced the 387 DX NPX which Intel now tend to call the "387 DX Math Co-processor" and for the 386SX the "387 SX Math Co-processor" (and of course a new assembler ASM386). Although compatible, the 387 differs from the 8087/80287 in some minor ways mainly to bring it into line with the IEEE Standard 754 for Binary Floating-Point Arithmetic.

Since the SX version was much cheaper to produce than the DX, IBM compatible 386SX PCs sold in their millions.

Now APL does a lot of high precision maths (both integer and real), so a 32-bit 386 PC with NDX is just what Dyadic needed for its Dyalog APL, and STSC needed for its new second-generation APL "APL*PLUS II".

486 — the Next Stage

The next stage in Intel's plan for global domination was to merge the 386DX and 387DX into a single chip. This they called the 486DX or sometimes "i486", but NEVER (by Intel) the 80486. (Intel's internal code name for the range is P4.) These were sold to run at 20 (rare), 25, 33 and 50Mhz.

Just a quick note on processor speed. All these 486s are produced to the same specification, and are subsequently speed rated via testing, so a 486DX-25 is a 486DX which failed the 33MHz test but passed the 25MHz test.

When someone discovered you could run some of these chips at twice the external clock speed, the 486 DX2 was born. DX2s are available as 40, 50 and 66 MHz versions but not 100Mhz. When sold as the "OVERDRIVE" version they still use the DX (not DX2) name. The Overdrive socket is 486's equivalent to the co-processor socket found on earlier PCs. (Note, Overdrive sockets now come in at least two colour-coded flavours, blue and white, but that's jumping the gun a bit.)

Now, legend has it that one batch of 486DXs failed to pass Intel's quality control, because the NDX part was not working. Rather than just binning them the marketing men at Intel came up with the idea of selling them under the name of 486 SX, under the impression that SX would sell anything.

An example might help here. My computer has an "80486" mainboard, which can (according to the "User's Guide") run at 25, 33 40 and 50 MHz. It can be configured with the following "169 pin PGA CPU" micro-processors:

80486SX-25, 80487SX-25, 80486DX2-50 CPU running at 25MHz
80486DX-33, 80486DX2-66 CPU running at 33MHz
80486DX-40 CPU running at 40 MHz
80486DX-50 CPU running at 50 MHz

It can't run the rare 486DX-20 or 486DX2-40.

What, you may ask, is an 80487SX-25? It took me some time to find out. It is a full i486DX-25 designed to be used in the OVERDRIVE socket of a motherboard fitted with an i486SX-25. It's a co-processor with an SX name!

Onward to the Pentium and P6

After the P4 comes P5 the "Pentium". This Intel chip has 2 on-board 486s, with 64-bit addressing. Because Intel could not copyright the number 586, they coined

the name Pentium. As you might expect, a Pentium runs about twice as fast as a 486 with the same clock speed.

Next comes the i486DX4. Knowing how well Intel have named their chips, you won't be surprised to discover this is a clock-tripled 486, with a larger (16k) processor cache. Hands up all of you who thought Intel could count.

And for the future, the P6 which, as they say, will be history.

Oh, I forgot. The moral of this story is: if you want your APL to run fast on a PC with "Intel Inside", get a Pentium, and if you can't afford a Pentium get an i486DX4. Or of course you could wait for Intel's next chip, the P6. But please, no SX, we're APLers.

Other Great Titles in this Series:

"Stop the Greenhouse Effect" Using a low power consumption i486SL

"Clone Wars" The ADM invasion

"Growing up with Cyrix" Replace your Intel 386SX or 386DX with Cyrix Cx486SRX2

"Blue Lightning" IBM strikes back with the clock-tripled 486SX3-99

"Anything Blue can do, 'i' can do better" Intel's latest chips, the clock-tripled i486DX4

Appendix: Power Saving Management System

To reduce the total power used in portable PCs, all Intel and Cyrix 486 chips now incorporate "System Management Mode" (SMM). This progressively reduces the power used by a processor by powering down parts of the chip which are not being used at the time.

When SMM was first introduced by Intel, the chips using it were designated as "486SL". However, this feature has been incorporated in all Intel 486 chips since the start of 1994, and the SL named chips are no longer made.

All Cyrix 486 chips have SMM as standard.

486SLC and 486DLC

The IBM chip 486SLC is logically the same as an i486SX, (it has a cache controller but no co-processor) but has 386DX pin out. This means that it can be put onto boards designed for 386DX chips, providing PC manufacturers with a cheap 386 to 486 upgrade path.

The Cyrix 486DLC chip is like the IBM 486SLC but DOES have a co-processor. I have been told that Cyrix don't actually make chips, they just design them and IBM actually produces them for Cyrix.

Intel Math Co-processor Identification Notes

Every Intel Math Co-processor is marked to indicate its product number and maximum operating speed. Currently, Intel marks the top of each chip with the math co-processor product number, a dash, then a number representing the maximum speed of the chip. Usually this number represents the speed in megahertz (MHz), but Intel's previous marking scheme was slightly different. The numbers in parentheses in the following chart are from this previous marking scheme.

Product Number	Rated Speed	Intel Model #
8087 (-3)	4.77MHz	BOX8087
8087-8 (-2)	8MHz	BOX8087-2
8087-10 (-1)	10MHz	BOX8087-1
80287 (-6 or -3)	6MHz	BOX287
80287-8	8MHz	BOX287-8
80287-10	10MHz	BOX287-10
80C287A	12MHz	DBOXC287A-12
387DX-16	16MHz	BOX387DX-16
387DX-20	20MHz	BOX387DX-20
387DX-25	25MHz	BOX387DX-25
387DX-33	33MHz	BOX387DX-33
387SX	16MHz	BOX387SX-16

The Intel 486/Pentium & Cyrix ranges:

(This will be incomplete and out of date by the time you read this article)

INTEL Chips

Processor	MHz	Volts	Watts	Co-Processor	Intel code
i486 SX	16	5/3.3	1.7	i487SX-16 overdrive	P23-16
i486 SX	20	5/3.3	2.3	i487SX-20 overdrive	P23-20
i486 SX	25	5/3.3	2.8	i487SX-25 overdrive	P23-25
i486 SX	33	5/3.3	3.0	i487SX-33 overdrive	P23-33
i486 DX	25	5/3.3	2.8	Internal	P4-25
i486 DX	33	5/3.3	3.5	Internal	P4-33
i486 DX	50	5	4.0	Internal	P4-50
i486 SX2	50	5/3.3	3.5	i487SX2-50 overdrive	P23-50
i486 DX2	40	3.3	1.4	Internal	P24-40
i486 DX2	50	5/3.3	3.9	Internal	P24-50
i486 DX2	66	5	4.9	Internal	P24-66
i486 DX4	75	3.3	3.0	Internal	P24C-75
i486 DX4	100	3.3	4.0	Internal	P24C-100
Pentium	60	5	12.0	Internal	P5-60
Pentium	66	5	13.0	Internal	P5-66
Pentium	90	3.3	4.0	Internal	P54C-90
Pentium	100	3.3	4.0	Internal	P54C-100
???????	133-200	?	?	Internal	P6

CYRIX chips

Processor	MHz	Volts	Watts	Co-Processor
Cx486DX-33	50	5	3.05	Internal
Cx486DX-V33	33	3	1.38	Internal
Cx486DX-40	40	5	3.42	Internal
Cx486DX-50	50	5	3.82	Internal
Cx486SLC-33	33	5	2.47	None?
Cx486SLC2-50	50	5	3.40	None?
Cx486DX2-50	50	5	3.82	Internal
Cx486DX2-V50	50	3	1.63	Internal
Cx486DX2-66	66	5	4.30	Internal
Cx486DX2-V66	66	3	1.85	Internal
Cx486DX2-V80	80	3	2.06	Internal

Bracket Axis Simulation for APL*PLUS II/386

by Allan Gay, Cocking & Drury Ltd

Introduction

A recent migration of mainframe APL2 applications to APL*PlusII/386 highlighted some language differences which had to be addressed.

One difference which proved particularly interesting concerned axis specifications. In APL2, some functions — mostly scalar dyadics — can take an axis spec which causes the data to be processed in ways which are unfamiliar to APL*PlusII/386 aficionados.

Because the target interpreter does not support these additional axis specifications, it was necessary to simulate them. This article describes how this was done, using APL*PlusII/386 Release 4 (or higher) in Evolution Level 2 mode. The examples in this article assume that $\square IO$ is 1.

Time constraints enforced a selective approach. Basically, nothing was simulated unless it was used by the applications we were migrating. However, each simulation was programmed to handle all documented (APL2 *Programming: Language Reference*, IBM, SH20-9227-0) repertoire rather than just the application's specific requirements.

Concept

To explore the concept, we'll talk about the *Add* function.

When we add two numeric arrays together, they must be conformable. In APL*PlusII/386, ignoring the multi-rank singletons, all of which conform with each other, that means that they must be identically shaped or that one of them must be a scalar.

```

      2 + 3
5
      2 + 3 4
5 6
      2 3 + 4 5
6 8
      (4 3p10 20 30) + 3 4p12
11 12 13 14
25 26 27 28
39 40 41 42

```

In APL2, conformability also takes into account the fact that we can perform axial addition. So, in addition to the above forms, we can code

```

      20 30 40 +[1] 3 4p12
21 22 23 24
35 36 37 38
49 50 51 52

```

In this example, the arguments are conformable because the shape of the left argument is the same as the shape of the right argument indexed by the axis specification. Arguments are conformable if $(\rho L) = (\rho R)[X]$ or $(\rho R) = (\rho L)[X]$.

We are not restricted to specifying a single axis, so we can code

```

      A←2 3p1 2 3 7 8 9
1 2 3
7 8 9

      A←2 2 3p12
1 2 3
4 5 6
7 8 9
10 11 12

      A←[2 3]W
2 4 6
11 13 15
8 10 12
17 19 21

```

Simulation

We couldn't code the axis specifications verbatim in APL*PlusII/386, so the code had to be rearranged somewhat. In the event, it proved simplest to write a function for each axial use, combining the axis specification with one of the function arguments.

Here is how the APL2 expression in the previous example is recoded for the simulation

```

      A plus_with_axis (2 3) W
2   4   6
11  13  15
8   10  12
17  19  21

```

The processing technique adopted was to propagate the input array which was to be applied axially, so that it was conformable with the other input array.

Thus, returning to an earlier example

```

      20 30 40 +[1] 3 4p12
21 22 23 24
35 36 37 38
49 50 51 52

```

the left argument was propagated to become

```

20 20 20 20
30 30 30 30
40 40 40 40

```

after which the addition could take place normally.

A glance at the APL2 Language Reference Manual (page 91) shows there to be twenty-one scalar dyadic functions. If their axial forms are to be simulated, their arguments are going to need this treatment. A utility function was therefore written to do this task. Each simulation function then called this conformer before doing the relevant APL*PlusII/386 operation on the results. Here is a listing of the conformer.

```

Vdz+Δa conformer Δw;ΔA;ΔE;ΔQ;ΔX;ΔW;[]IO
[1]  A Conformer a and 2>w for arithmetic on axes 1>w.
[2]  A (See "APL2 Programming: Language Reference" pp95-96)
[3]
[4]  A a is target array (addend|subtrahend|multiplicand|dividend).
[5]  A w is 2-item vector:
[6]  A 1) integer axis-number[s]
[7]  A 2) modifier array (increment|decrement|multiplier|divisor).
[8]
[9]  A NB: Conversely, 2>w may be the target and a may be the modifier.
[10]
[11] A ← is one of:
[12] A simple textvector errormsg OR
[13] A 2-item nested vector:
[14] A 1) conformed copy of a
[15] A 2) conformed copy of w.
[16]
[17] []IO←1
[18] ΔA+Δa                                A Get target.
[19] (ΔX ΔW)+Δw                            A Get axis spec & modifier.
[20] ΔX+(-,ΔX)[Δ,ΔX]                      A Work with vector of axis nos.
[21] ΔE+0                                  A Start exchange-args flag.
[22] +((ρρΔA)>ρρΔW)/AAA                  A If 2>w is target, and a is
[23] (ΔW ΔA)+ΔA ΔW                        A modifier, swap them over
[24] ΔE+1                                  A and note the exchange.
[25]
[26] AAA:
[27] Δz+'AXIS ERROR'
[28] +((ρΔX)*ρρΔW)/ZZZ                    A Axis for each axis of modifier?
[29] +(0∈{ΔX,ΔX}=1ρΔX)/ZZZ                A No duplicate axes?
[30] +(0∈ΔX∈1ρρΔA)/ZZZ                    A All axes present in target?
[31] Δz+'LENGTH ERROR'
[32] +(-(ρΔW)=(ρΔA)[ΔX])/ZZZ              A Check the modifier will map
[33]                                         A to the stated target axes.
[34] ΔQ+((1ρρΔA)~ΔX),ΔX                  A Put declared axes last.
[35] Δz+ΔQ&((ρΔA)[ΔQ])ρΔW                A Generate enough data.
[36] Δz+ΔA Δz                              A Make result on the assumption
[37] +(~ΔE)/ZZZ                            A we didn't swap the inputs.
[38] Δz+(2>Δz) ΔA                          A If we did, swap back.
[39]
[40] ZZZ:
V

```

Coding invocations

In APL*PlusII/386, it was necessary to recast axial invocations of the scalar dyadics to use the simulation functions. Thus, for example,

```
20 30 40 +[1]3 4ρ112
```

became

```
20 30 40 plus_with_axis 1,←3 4ρ112
```

The same form was used regardless of which argument was to be extended. Thus, to invert the above example,

```
(3 4 p112) +[1] 20 30 40
```

became

```
(3 4 p112) plus_with_axis 1, c20 30 40
```

Using this coding paradigm, manually recasting applications' axial uses of the scalar dyadics became a fairly pedestrian affair.

Repertoire

Altogether, the migrated applications required two sets of simulation functions to be written. The first set comprised some scalar dyadic functions: *and_with_axis*, *divide_with_axis*, *multiply_with_axis*, *or_with_axis*, *plus_with_axis* and *subtract_with_axis*.

The second set comprised the monadic mixed function *ravel_with_axis*, and two dyadic mixed functions *drop_with_axis* and *take_with_axis*.

All the functions listed in the first set above were very similar. They all made use of the conformer. Here, as an example, is *plus_with_axis* again.

```

VAZ+Δa plus_with_axis Δv;DIO
[1]  a Return the sum of a and 2Δw on axes 1Δw  ← PUBLIC
[2]
[3]  a a is a numeric vector: the addend.
[4]  a w is 2-item vector:
[5]  a 1st item is integer scalar axis-number
[6]  a 2nd item is the increment.
[7]  a + is the sum.
[8]
[9]  DIO+1
[10] Δz+Δa conformer Δw          a Conform a and w axially.
[11] ΔERROR (1z=Δz)/Δz         a Report any error.
[12] Δz+Δw/Δz                  a Add.
v

```

The three functions in the second set were not amenable to having their arguments conformed in this way. It was necessary to engineer a separate solution for each of them.

Ravel With Axis

APL2's Ravel With Axis function is monadic, so the simulation function was written to take the axis specification as its left argument. The axis specification may be in one of four forms: empty, integer vector, integer scalar, and fractional scalar. The simulation function, after validating the arguments, checks which form applies, and then branches to the appropriate logic, which begins at line [25].

```

VΔZ+ΔA ravel_with_axis ΔW;ΔQ
[1]  A Ravel w on axes α.
[2]
[3]  A α is integer scalar or vector axis specification.
[4]  A w is variable to be ravelled.
[5]  A + is the result of the ravel.
[6]
[7]  +(1<ρpΔA)/XXX           A α must be rank-0 or rank-1.
[8]  +(v/0>ΔA)/XXX           A α must all be positive.
[9]  +(0=ρpΔA)/CCC           A α ok if scalar.
[10] +(0=ρΔA)/CCC           A α ok if empty vector.
[11]
[12] +(-^/0=1|ΔA)/XXX        A Bad α if fractions in vector.
[13] +(-^/ΔA∈\ρpΔW)/XXX      A Bad α if any axis nonexistent.
[14] +(1=ρΔA)/CCC           A α ok if only one axis stated.
[15]
[16] +(v/ΔA*ΔA[⊆ΔA])/XXX      A Bad α if axes out of sequence.
[17] +(v/ΔA*ΔA[1]+~1+ρΔA)/XXX A Bad α if axes not contiguous.
[18]
[19] CCC:
[20] +(0<ρpΔA)/EEE           A CASE: empty axis spec
[21] +(1=ρpΔA)/VVV           A CASE: integer vector spec
[22] +(0=1|ΔA)/SSS           A CASE: integer scalar spec
[23] +FFF                     A CASE: fractional scalar spec
[24]
[25] EEE: ΔZ+((ρΔW),1)ρΔW    ◊ +ZZZ A EMPTY SPEC: append an axis
[26] FFF: ΔZ+((1,ρΔW)[⊆ΔA,⊆ρpΔW])ρΔW A FRACTION SPEC: insert new axis
[27] +ZZZ
[28] SSS: ΔZ+ΔW              ◊ +ZZZ A SCALAR INTEGER SPEC: w axis
[29] VVV:                    A VECTOR INTEGER SPEC:
[30] ΔQ+ρΔW                  A combine indicated axes.
[31] ΔQ+((~1+ΔA[1])+ΔQ).(*ΔQ[ΔA]).(ΔA[ρΔA])+ΔQ
[32] ΔZ+ΔQρΔW
[33] +ZZZ
[34]
[35] XXX: [ERROR 'AXIS α ERROR']
[36]
[37] ZZZ:
V

```

Some examples of use follow.

An empty axis specification appends another axis

```

      []←Q←(10) ravel_with_axis 1 2 3
1
2
3
      ρQ
3 1

```

A fractional axis specification inserts another axis

```

      A←2 3ρ16
      []←Z←1.1 ravel_with_axis A
1 2 3
4 5 6
      ρZ
2 1 3

```

A vector of axis numbers causes the indicated axes to be combined as one axis

```

      M←2 3 4ρ124
      []←N←2 3 ravel_with_axis M
1 2 3 4 5 6 7 8 9 10 11 12
13 14 15 16 17 18 19 20 21 22 23 24
      ρN
2 12

```

Drop With Axis

To simulate Drop With Axis, the drop specification is expanded to include a zero for each unreferenced axis and then an APL*PlusII/386 Drop (i.e. without axis) is done. This paradigm has proved serviceable for the migrated applications, and has produced correct results for the handful of examples in the manual (pages 197-198), although its ultimate correctness in the production of empty arrays is moot.

```

      []←D←1 2 drop_with_axis (2 3)(3 4 5ρ160)
8 9 10
13 14 15
18 19 20
28 29 30
33 34 35
38 39 40
48 49 50
53 54 55
58 59 60
      ρD
3 3 3

```

Take With Axis

Take With Axis is quite a challenge, it being necessary to generate the correct prototype with which to extend each axis during over-taking. Again a simulation was produced which met the requirements of the migrated applications and ran the examples in the manual correctly.

```

]DISPLAY T←2 3 4p'HOME',1 1 1 1,'FAIR',2 2 2 2,'MARK',3 3
3 3
+---+
|H O M E|
|1 1 1 1|
|F A I R|
|2 2 2 2|
|M A R K|
|3 3 3 3|
+---+

]DISPLAY Z←5 6 take_with_axis (2 3) T
+---+
|H O M E|
|1 1 1 1 0 0|
|F A I R|
|0 0 0 0 0 0|
|0 0 0 0 0 0|
|2 2 2 2 0 0|
|M A R K|
|3 3 3 3 0 0|
+---+

pZ
2 5 6

```

As this example demonstrates, when an over-take occurs on a given axis, the fill item is of the same type as the first item of each sub-array along that axis.

Conclusion

The absence of certain APL2 axial forms from the APL*PlusII/386 repertoire need be no barrier to a successful migration from mainframe APL2. What's more, writing functions to simulate them is an entertaining exercise.

Of course, there were other APL2-isms besides these. Simulating `⌈TF` was a mini-system in its own right. Simulating `⌈EA` was rendered awkward by the applications' inclusion of branch statements in `⌈EA` arguments. Disclose was an interesting function which shrank as it improved, and then there were simulations of the APL2 Assembler functions such as CAN and DAN besides. But I think we'd better leave all these amusements for some future article.

Guidelines for Vector Authors

by Adrian Smith (Vector Production)

Small Stuff (SusMemb News etc.)

This can come over the mailbox just as plain text embedded in your message. Use [] or quad- as a placeholder for quad, and some sensible transliteration of the APL symbols such as rho. If you want your formatting to survive (*italic* and **bold** for example) save the text from Word in *Rich Text Format* (file extension .RTF) and just append this to the message. I can quickly swipe the obvious bit with the mouse, copy it into an empty file, and open the result in Word. This seems quite reliable, but I believe there are limits on message length, so don't try it with a complete article!

Articles which are mostly text (or J)

I still prefer these with as little formatting as possible. I normally start by stripping stuff back to plain text anyway, as otherwise I get unexpected side-effects, such as your italics appearing in *Century Schoolbook Italic* (because that was the font you started in) rather than *Palatino*. J authors can help a lot by surrounding J code with obvious markers such as <<a =. 1. 12>> — I can quickly knock up a Word macro to reformat the bits in between, and then remove the markers. APL is much easier to spot, and I am quite happy to read the text through and format this by hand.

Heavy Windows stuff with lots of Bitmaps

This really has to be Write or Winword (2 not 6 at the moment please). If possible, ask me for a copy of VECTOR.DOT before you start, and write the article based on Vector's template. Please check that the page size is A4. This way you have full control over page breaks (so your pictures go where you want them) and your special effects (tables, bordered paragraphs, sections in double-column and so on) will survive.

Failing this, please keep it simple! Use **Heading1** for top headings, **Heading2** for subheads, **Normal** for everything else. **DO NOT USE FORMAT CHARACTER TO CHANGE THE NORMAL FONT**. Use format style define and set it there! I can cope with all known APL TrueType fonts, but please be consistent if possible, or mark clearly what is from Dyalog and what is from PLUS and what is from ISWIN. CompuServe seems totally reliable, but for anything large, a disk in the post is cheaper and can be wrapped in a paper copy 'just in case'.

Index to Advertisers

Dyadic Systems Ltd	12
IBM	2
MicroAPL	4
Sungard Capital Markets	6
Soliton	40
APL Booklist	32
Vector Back Numbers	37

All queries regarding advertising in VECTOR should be made to Gill Smith, at 0439-788385, CompuServe: 100331,644.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Anthony Camacho,
11 Auburn Road, Redland,
BRISTOL, BS6 6LS
Tel: 0272-730036
Email: acamacho@cix.compulink.co.uk

Authors wishing to use Windows Write should contact Vector Production for a copy of the Vector APL TrueType font and Vector APL typebox.

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO6 4JJ.

Tel: 0439-788385 (any time)
Compuserve: 100331,644.

BAA: Membership Application / Renewal

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: _____
 Address: _____

 Post or zip code: _____
 Country: _____
 Telephone Number: _____

Membership category (please tick box):		
UK private membership	£12	☐
Overseas private membership	£14	☐
Airmail supplement (not needed for Europe)	£4	☐
UK Corporate membership	£100	☐
Corporate membership overseas	£135	☐
Sustaining membership	£430	☐
Non-voting UK member (student/OAP/unemployed only)	£6	☐

PAYMENT — in Sterling only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard or Visa number.

I authorise you to debit my Visa/Mastercard account

Number: Expiry date: |

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

*Data Protection Act:
 The information supplied may be
 stored on computer and processed
 in accordance with the registration
 of the British Computer Society.*

Signature: _____ Send the completed form to:

British APL Association, c/o Rowena Small, 8 Cardigan Road, LONDON, E3 5HU

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1994/95 Committee

Chairman:	Dr Alan Mayer 0792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Secretary:	Sylvia Camacho 0272-730036	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Treasurer:	Nicholas Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU
Journal Editor:	Anthony Camacho 0272-730036 acamacho@cix.compulink.co.uk	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Activities:	Duncan Pearson 0483-33329 100265.1564@compuserve.com	7 Lynne Court Chesham Road GUILDFORD, Surrey GU1 3LR
Education:	Dr Ian Clark 0388-527190 100021.3073@compuserve.com	9 Hill End, Frosterley Bishop Auckland Co. Durham DL13 2SX
Technical:	Jonathan Barman 0488-648575 100116.1030@compuserve.com	Hill Top House, East Garston, NEWBURY, Berks RG16 7HD
Projects:	George MacLeod 0442-878065 100412.1305@compuserve.com	Greymantle Associates Ltd., Bartrum House, Ravens Lane, BERKHAMSTED, Herts HP4 2DY
Publicity:	David Eastwood 071-922 8866 microapl@applelink.apple.com	MicroAPL Ltd., South Bank Technopark, 90 London Road, LONDON SE1 6LN
Recruitment:	Mark Harris 0438-310155	Kestrel Consulting, Business & Technology Centre, Bessemer Drive, STEVENAGE, Herts SG1 2DX
Administration:	Rowena Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Anthony Camacho	0272-730036
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (0439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (0439-788385)
Support Team:	Jonathan & Bridget Barman (0488-648575), Ray Cannon (0252-874697), Richard and Adam Weber (0302-539761), Sylvia Camacho, Duncan Pearson, John Searle (081-858 6811), David Ziemann (071-267 8032), Jon Sandles (0904-411635)	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost:

Compass R&D Ltd
5 & 7 Frederick Sanger Road
Surrey Research Park
CULFORD, Surrey GU2 5MD
Tel: 0438 302249
Fax: 0438 302279

Dyadic Systems Ltd
Riverside View, Basing Road
Old Basing, BASINGSTOKE
Hants, RG24 0AL
Tel: 0256 811125
Fax: 0256 811130

FMW Trading Systems Ltd
Hamilton House
1 Temple Avenue
LONDON EC4Y 0JA
Tel: 071 853 4242
Fax: 071 853 3625

MicroAPL Ltd
South Bank Technology
90 London Road
LONDON SE1 6UN
Tel: 071 922 8866
Fax: 071 928 1000

Soliton Associates Ltd
Strat Blankenberg 59
1082 AG Amsterdam
Netherlands
Tel: +31 20 646 4275
Fax: +31 20 644 1206
Email: jhr@soliton.com

Dutch APL Association
Postbus 1041
8450 BH Nieuwegein
Netherlands
Tel: 08474 2287

Manugistics
2415 East Jefferson St
Rockville
MARYLAND 20852 USA
Tel: (301) 984-5312
Fax: (301) 984-5094
Email: apsales@manu.com (US)
Email: mti@manu.com (International)

Insight Systems ApS
Nordre Strandvej 119A
DK-8450 Halden
Denmark
Tel: +45 42 10 70 22
Fax: +45 42 10 75 74
Email: info@insight.dk